



IBM Granite Edge AI Optimisation using TurboQuant on Intel Windows PCs

Candidate Number: VQVZ6

Internal Supervisor: Dr Graham Roberts

Deputy Supervisor: Dr Dean Mohamedally

External Supervisors: IBM - John McNamara Intel - Costas Stylianou

MSc Computer Science

Submission Date: 14/09/2026

Disclaimer:

This report is submitted as part requirement for the MSc Computer Science degree at UCL. It is substantially the result of my own work except where explicitly indicated in the text.

The report may be freely copied and distributed provided the source is explicitly acknowledged.

Department of Computer Science
University College London

Abstract

Running language models on a personal computer can reduce the need for cloud services, but limited memory and different model formats can make them difficult to set up. This project developed a Windows 11 application for running selected IBM Granite models on consumer Intel hardware. It also investigated whether TurboQuant could reduce cache memory use while maintaining acceptable speed and output quality.

The application guides users through choosing a model, inspecting it, checking hardware fit, selecting a configuration and starting local chat. It supports GGUF and OpenVINO models, with runtime operations running in separate processes. Available configurations depend on supporting evidence and current memory and storage checks. Experiments on one Intel computer with approximately 16 GB of RAM compared selected runtime and cache configurations, recording memory use, speed, output quality and failures. A separate study explored TurboVec for compressed vector indexes.

Selected configurations ran successfully, and matched TurboQuant tests showed lower key-value cache memory use, although speed and quality did not always improve. These savings applied to the cache, not to total application memory. TurboVec produced faster searches and smaller indexes than the Exact baseline, but none of its compressed formats met the retrieval-quality criteria, so it was kept as a separate demonstrator. Automated inspection checks and manual offline use supported parts of the application workflow, but memory-estimator calibration, a formal user study and clean-machine installation testing were not completed. The results therefore apply only to the hardware, models and runtime versions tested. [Application demonstration video folder.](#)

Contents

Abstract	i
List of Abbreviations	viii
1 Introduction	1
1.1 Project Context and Motivation	1
1.2 Problem Statement	1
1.3 Project Aim and Objectives	2
1.4 Research Questions	2
1.5 Individual Contribution	3
1.5.1 Third-Party Technology	3
1.5.2 Software Development Contribution	3
1.5.3 Testing and Experimental Contribution	3
1.6 Development Approach and Report Structure	3
2 Background and Related Work	5
2.1 Local Large Language Model Inference	5
2.2 Memory Use During Local Inference	5
2.3 Numerical Representations and Quantisation	6
2.4 IBM Granite Models	7
2.5 GGUF and the llama.cpp Runtime Route	8
2.6 OpenVINO Intermediate Representation and the Intel Runtime Route	8
2.7 KV-Cache Compression and the TurboQuant Route	9
2.7.1 Standard KV-Cache Quantisation and Baselines	9
2.7.2 Other KV-Cache Compression Approaches	9
2.7.3 PolarQuant, QJL and TurboQuant	10
2.7.4 TurboVec as a Separate Vector-Compression Route	10
2.8 Existing Local AI Applications, Inspection and Hardware-Fit Tools	11
2.9 Technology Alternatives and Selection Rationale	11
2.10 Chapter Summary	11
3 Requirements, Users, and Project Method	12
3.1 Stakeholders and Intended Users	12
3.2 Operational Context and Assumptions	13
3.3 Functional Requirements	13
3.4 Non-Functional Requirements	14
3.5 MoSCoW Prioritisation	15
3.6 User Journeys and Use Cases	16

3.7	Requirements Traceability and Verification Plan	17
3.7.1	Experimental Evidence Coverage	18
3.8	Project Management and Change Control	18
4	Experimental Methodology	19
4.1	Evaluation Design and Research-Question Mapping	19
4.2	Evidence Authority, Provenance and Data Grain	20
4.3	Hardware, Software and Repository Identity	20
4.4	Controlled Execution Protocols	20
4.5	Metrics and Calculations	21
4.5.1	Performance and Memory	21
4.5.2	Generative Output Quality	22
4.5.3	TurboVec Retrieval Quality	22
4.6	Status, Safety and Comparison Rules	23
4.7	Analysis Boundaries, Ethics and Reproducibility	23
5	System Architecture and Design	24
5.1	Architectural Drivers	24
5.2	Architecture Alternatives and Decisions	24
5.3	System Context	25
5.4	Component Architecture	25
5.5	User Interface and Navigation Design	26
5.6	Runtime Processes and Deployment	28
5.7	Lifecycle, Cancellation and Recovery	29
5.8	Security, Privacy and Ethical Design	29
5.9	Design Evolution	29
6	Implementation	31
6.1	Development Environment and Repository Structure	31
6.2	WinUI Application Shell and Navigation	31
6.3	Model Import Workflow	32
6.4	GGUF Validation and Metadata Inspection	32
6.5	Hardware Analysis	33
6.6	Memory Estimation	34
6.6.1	Experimental Validation Status	35
6.7	Configuration and Optimisation Selection	35
6.8	Runtime Integration	36
6.8.1	llama.cpp Integration	36
6.8.2	OpenVINO Integration	37
6.8.3	TurboQuant Integration or Feasibility Investigation	37
6.8.4	TurboVec Feasibility Harness	38
6.9	Inference User Interface	38
6.10	Reliable Programming and Observability	38
6.11	Implementation Status	39

7	Software Verification and User-Experience Evaluation	40
7.1	Verification Approach	40
7.2	Automated Testing and Results	40
7.3	Application Journeys, Packaging and Recovery	41
7.4	Offline Use and Safety Checks	41
7.5	Interface and Accessibility Checks	41
7.6	Overall Findings and Remaining Limitations	41
8	Experimental Results and Discussion	42
8.1	Experimental Completeness and Evidence Coverage	42
8.2	Runtime Feasibility by Route	42
8.3	Inference Performance	43
8.4	Memory and TurboQuant Trade-Offs	44
8.5	Generative Output Quality	45
8.6	Context, Stability and Failure Analysis	46
8.7	Application Evaluation Status	46
8.8	TurboVec Feasibility	46
8.9	Relationship to Prior Claims	47
8.10	Threats to Validity	47
9	Critical Evaluation and Conclusion	48
9.1	Summary of Achievements	48
9.2	Final Answers to the Research Questions	48
9.3	Evaluation of the Software Product	49
9.4	Evaluation of Architecture and Technology Choices	49
9.5	Evaluation of the Research Contribution	50
9.6	Limitations	50
9.7	Prioritised Future Work	50
9.8	Final Conclusion	50
A	Requirements Catalogue and Traceability Matrix	56
B	User Journeys and Detailed Use Cases	72
B.1	Stakeholder Guidance and Source Records	72
B.2	Role-Based Proto-Personas	72
B.2.1	P1: Non-Specialist Local-Inference User	73
B.2.2	P2: Technical Troubleshooting and Reproduction User	73
B.3	Detailed Use Cases	74
C	Architecture Documentation and Decision Records	78
D	Experimental Protocol and Environment Manifest	86
E	Prompt Set, Reference Answers, and Scoring Rubric	88
F	Detailed Experimental Results	90
F.1	How to Read the Experimental Evidence	90
F.2	Inference Campaign Outcome Summary	91
F.3	Cross-Route Comparison Boundary	92

F.4 TurboVec Gate Audit	92
G Software Test Catalogue and Reports	94
G.1 Scope and Recorded Results	94
G.2 Automated Test Catalogue	95
G.3 Manual and Package Checks	95
G.4 Remaining Issues and Coverage Limits	96
G.5 Experiment-Associated Validation Catalogue	97
H User Manual	98
H.1 Before Starting	98
H.2 From Model Selection to Chat	98
H.3 Stopping and Getting Help	99
I Developer, Build, and Reproduction Manual	100
I.1 Source and Build Setup	100
I.2 Code and Test Locations	100
I.3 Experiment Reproduction	101
J Code Source References	102
K Risk, Assumption, Licence, and Known-Issue Registers	103
K.1 Risk Register	103
K.2 Experiment-Derived Known Issues	104
K.3 Assumption Log	104
K.4 Dependency and Licence Inventory	105
L Release and Demonstration Material	106
L.1 Recorded Application and Release Status	106
L.2 Demonstration and Evidence Backup	106

List of Figures

5.1	System context and main interactions of Granite Edge AI.	26
5.2	Logical components and their main responsibilities.	27
5.3	Local process groups and storage responsibilities.	28
5.4	Intended lifecycle, validation decisions and recovery paths.	30
6.1	GGUF vocabulary-only inspection settings.	32
6.2	Collecting and checking installed, OS-usable and available RAM.	33
6.3	Preventing conflicting readings from becoming accepted memory facts.	33
6.4	GGUF KV-cache estimation with head padding and block rounding.	34
6.5	Checking candidate admission before issuing an optimisation plan.	36
6.6	Settings passed to the packaged GGUF runtime.	36
6.7	Checking the applied OpenVINO key and value cache precision.	37
6.8	Preserving response text while recording its completion state.	38
6.9	Preventing duplicate final messages.	39
C.1	User actions and application responsibilities from model selection to chat, including preparation and recovery.	79
C.2	Model acquisition and import	79
C.3	Protected runtime sequences	80
C.4	Hardware evidence and compatibility	82
C.5	Optimisation custody and publication	83
C.6	Trust boundaries and data flows	84

List of Tables

2.1	Different quantisation targets in local LLM inference	7
2.2	Comparison of selected existing applications and tools, reviewed 14 August 2026	11
3.1	Grouped summary of Active functional Must Have requirements	13
3.2	Active non-functional Must Have requirements and planned checks	15
3.3	MoSCoW v1.2 scope by priority and lifecycle	15
3.4	F-M21: planned trace from requirement to application evidence	17
4.1	Operational mapping of research questions to controlled evidence	19
5.1	Main architecture choices and trade-offs	25
6.1	OpenVINO Raw 3B preference mappings, subject to current checks.	35
6.2	Implementation status and main limitations.	39
8.1	Terminal status of the five inference-route campaigns	42
8.2	Observed performance ranges within each route	43
8.3	Matched experimental and official OpenVINO results	44
8.4	Selected within-route AtomicBot cache trade-offs	44
8.5	Descriptive quality-score distributions by route	45
8.6	TurboVec results at 10,000 chunks	46
A.1	Baseline identity and selected revisions	56
A.2	Complete catalogue with draft Active acceptance criteria	57
A.3	Active requirements: source and traceability	66
A.4	Exceptional planned evidence paths	71
B.1	Development of the application and intended-use focus	72
B.2	P1: intended role and assumptions not validated by user research	73
B.3	P2: intended role and assumptions not validated by user research	73
B.4	Detailed use cases derived from the intended workflow	74
D.1	Controlled inference-route identities and principal coverage	86
D.2	Published Granite 4.1 Language architecture and intended project role	87
F.1	Evidence grains for the five inference routes	90
F.2	Inference campaign outcomes	91
F.3	Principal failure, blocking and recovery outcomes	91
F.4	TurboVec supporting results at 1,000 and 10,000 chunks	92
F.5	TurboVec Gate A results at 10,000 chunks	93

G.1	Application test groups and requirement links	95
G.2	Manual checks and evidence limits	96
J.1	Source files and methods for the Chapter 6 code figures.	102
K.1	Main project risks at the report cut-off	103
K.2	Selected final assumption outcomes	104
K.3	Dependency and licence inventory	105

List of Abbreviations

AI	Artificial Intelligence
BF16	Brain floating-point, a 16-bit format
CI	Continuous Integration
CPU	Central Processing Unit
FP16/FP32	16-bit and 32-bit floating-point formats
GGUF	Model file format used by <code>llama.cpp</code>
GPU	Graphics Processing Unit
INT4/INT8	Four-bit and eight-bit integer formats
IR	Intermediate Representation
KV cache	Key-Value cache
LLM	Large Language Model
MiB/GiB	Binary units equal to 2^{20} and 2^{30} bytes
nDCG	Normalised Discounted Cumulative Gain
NPU	Neural Processing Unit
RAG	Retrieval-Augmented Generation
RAM	Random-Access Memory
RTM	Requirements Traceability Matrix
SDK	Software Development Kit
SYCL	C++ programming model for heterogeneous computing
TTFT	Time to First Token
UI	User Interface
XAML	Extensible Application Markup Language

Experiment-Specific Format Labels

The following labels are not interchangeable. Their meaning depends on the repository and on whether a table column describes model weights, the KV cache or a vector index.

Q4_K_M/Q8_0	Q4_K_M is a GGUF weight type; Q8_0 is a <code>llama.cpp</code> quantisation type used for weights or the KV cache as stated by the table column
F16/U4/U8	Conventional 16-bit floating-point, unsigned four-bit and unsigned eight-bit KV-cache formats in the experiment tables
Turbo2/3/4	AtomicBot fork-specific TurboQuant KV-cache formats
TQ3_0	animehacker fork-specific three-bit KV-cache implementation, classified by the source audit as PolarQuant without QJL
TBQ3/TBQ4	OpenVINO TurboQuant three-bit and four-bit KV-cache codecs
TQ2/TQ3/TQ4	TurboVec compressed vector-index modes; these are not language-model KV-cache formats
QJL	Projection-based cache quantisation method using a Johnson–Lindenstrauss transform
Polar3/Polar4	Experimental three-bit and four-bit PolarQuant KV-cache formats

Chapter 1

Introduction

1.1 Project Context and Motivation

Cloud LLM services run models on remote servers, reducing the amount of computation required on the user’s computer but requiring an internet connection and potentially incurring usage charges [1,2]. In contrast, local inference runs the model on the user’s own computer. Once the required model and software are available locally, responses can be generated without a continuous internet connection [3].

Education and healthcare provide motivating contexts for this project because users in these settings may work with sensitive data [4,5] and may have limited or unreliable internet access [5,6]. Processing data locally can limit how much information leaves the device, but it does not guarantee privacy, security or legal compliance. Prompts, responses and logs may still be saved locally and contain sensitive information [7].

The project targets Windows 11 PCs with selected Intel hardware, but the experimental results only apply to the computer tested in this project. IBM Granite was selected as the project’s model family. Model files need storage space, while inference uses memory for the model weights, activations and the KV cache [8]. The memory available for inference also depends on how much memory other running processes are using [9].

These memory limits make the KV cache an important optimisation target for local inference [10,11]. The project investigates TurboQuant as a KV-cache quantisation method, separate from weight quantisation [11]. Experimental results reported in the TurboQuant paper cannot be assumed to apply to IBM Granite models, the selected runtimes or the tested Windows 11 system with Intel hardware [11].

1.2 Problem Statement

Running local LLM inference on a Windows 11 PC with selected Intel hardware depends on more than just the size of the model file. Model format, weight precision, KV-cache format, context length, runtime, execution device and available physical memory affect whether a configuration loads successfully, generates valid output and fits in memory, as well as its performance and output quality. Selecting a suitable configuration may present difficulties for non-specialist users. This is treated as a design assumption because it has not been validated through a completed user study. The project also investigates whether verified TurboQuant configurations reduce KV-cache memory use compared to matched standard-cache configurations used as the baseline on the tested system, and whether there are performance, output quality, and runtime stability differences. The conclusions are limited to the tested models, runtimes, hardware and configurations, and a hardware-fit result is only an estimate.

1.3 Project Aim and Objectives

The aim and objectives below summarise the Project Definition and Scope Baseline (PLAN-PROJ-001). Chapter 3 provides the requirements and their traceability.

The project aims to design, build and evaluate a WinUI 3 desktop application for running selected IBM Granite models locally on Windows 11 PCs with selected Intel hardware. It also investigates whether TurboQuant configurations verified as active can reduce KV-cache memory and what differences are observed in output quality, performance and runtime stability.

The main project plan has eleven detailed objectives. To keep this introduction short, they are grouped below into five main objectives, labelled CO1–CO5.

- CO1.** Develop a guided WinUI 3 application that supports model artefact selection and inspection, hardware profiling, memory estimation, configuration selection and local inference without requiring command-line knowledge.
- CO2.** Integrate and verify local inference routes, including controlled runtime execution, streamed output, cancellation, error handling and reporting the runtime and, where available, the execution device actually used.
- CO3.** Investigate TurboQuant and other optimisation routes, while keeping weight and KV-cache quantisation separate and assessing whether TurboVec is suitable for the project. Only configurations supported by the project evidence are treated as verified; all others remain experimental or unavailable.
- CO4.** Evaluate complete IBM Granite configurations under controlled conditions by measuring memory use, performance, output quality, context-length capacity and stability, while accounting for model size, weight precision, KV-cache format, context length, runtime and execution device.
- CO5.** Verify, package and document the system using traceable requirements, automated testing, continuous integration, reproducible experimental evidence and release documentation.

These objectives summarise the original project plan: CO1 covers O1–O4 and O6; CO2 covers O5, O6 and O8; CO3 covers O7–O9; CO4 covers O8 and O10; and CO5 covers O11.

Lower-priority features and exclusions are recorded in Section 3.5.

1.4 Research Questions

Four main questions guide the research evaluation; a separate exploratory question addresses TurboVec. Chapter 4 defines their methods and measures.

- RQ1 Intel hardware and runtime compatibility:** Which combinations of selected IBM Granite models and inference routes load and generate valid output on the tested Windows 11 system’s Intel CPU or integrated GPU, and what compatibility and fallback limitations apply to each build, backend and execution device?
- RQ2 TurboQuant memory, quality and performance trade-offs:** Compared with matched standard KV-cache baselines, what differences are observed in KV-cache allocation, peak process memory, output quality, context-length feasibility, performance and stability when a TurboQuant configuration is verified as active?

- RQ3 Feasibility under defined memory budgets:** Which complete Granite configurations meet the defined feasibility criteria within 4 GB, 8 GB and 16 GB total system-memory budgets, and how does feasibility vary with model size, weight precision, KV-cache format, context length, runtime and execution device?
- RQ4 End-to-end application usefulness and reliability:** How reliably and clearly does the WinUI 3 application guide a non-specialist user through importing and inspecting a local Granite model artefact, assessing hardware and memory fit, selecting a configuration and running inference without command-line knowledge?
- RQ-TV Exploratory TurboVec feasibility and release decision:** Can the specified TurboVec implementation be reproduced in the target environment, and should it be developed further, retained only as a demonstrator, deferred or excluded?

1.5 Individual Contribution

This was my individual project. My supervisor gave me advice, but I completed the design, development, testing, experiments and analysis.

1.5.1 Third-Party Technology

IBM Granite, `llama.cpp`, OpenVINO, OpenVINO GenAI, TurboQuant, TurboVec, WinUI 3 and the Windows App SDK were created by other people and organisations. My contribution was using and connecting these technologies in the application and evaluating them through experiments.

1.5.2 Software Development Contribution

I built the WinUI 3 application for downloading, importing and checking models. It also checks the computer, estimates memory use, helps users choose supported settings, optimises models and provides local chat. I connected the GGUF and OpenVINO routes and added model-file checks, memory and storage checks, progress updates, cancellation, error handling and automated tests.

1.5.3 Testing and Experimental Contribution

I designed, ran and documented controlled experiments for selected IBM Granite configurations across three `llama.cpp` routes and two OpenVINO routes. The study measured compatibility, memory, performance, output quality and stability. It also recorded failed, blocked and unavailable cases, and compared results only when the configurations were similar enough for a fair comparison.

I also carried out a separate TurboVec study that compared exact and compressed embedding indexes. The models, repositories and compression methods were third-party work. My contribution was the experimental design, test preparation, execution, evidence collection, analysis and release decisions. Chapters 4 and 8 give the full methods and findings.

1.6 Development Approach and Report Structure

The project used an incremental approach instead of full Scrum or strict test-driven development. The methods were chosen to match the size and risks of an individual MSc project. *Systems Engineering Principles and Practice* guided requirements, traceability, risk management, staged integration and verification [12]. *Engineering Software Products: An Introduction to Modern Software Engineering* guided

incremental development, testing, code management and continuous integration [13]. *Fundamentals of Software Architecture: A Modern Engineering Approach* and *A Philosophy of Software Design* guided modular design, separation of responsibilities, complexity management, trade-off analysis and design decisions [14, 15].

Implementation and maintenance were also guided by *Code Complete: A Practical Handbook of Software Construction*, *Refactoring: Improving the Design of Existing Code*, *The Art of Unit Testing: With Examples in JavaScript*, *Why Programs Fail: A Guide to Systematic Debugging* and *Designing Secure Software: A Guide for Developers* [16–20]. These books helped me with coding, refactoring, testing, debugging and secure software practices. *The UX Book: Agile UX Design for a Quality User Experience* guided the user-experience approach [21]. *AI Engineering: Building Applications with Foundation Models* guided AI evaluation and inference optimisation [3], while *Designing Data-Intensive Applications: The Big Ideas Behind Reliable, Scalable, and Maintainable Systems* guided me on reliability, maintainability and technical trade-offs [22].

Development was carried out incrementally: progressing from model inspection and hardware analysis to memory estimation, configuration selection and local inference. Requirements were linked to the evidence produced during development, and early feasibility checks were used to assess whether uncertain routes such as OpenVINO and TurboQuant were appropriate for integration. The upstream `llama.cpp` route was kept as an experimental baseline. The application uses verified GGUF and OpenVINO runtime packages and only shows optimisation choices that have supporting evidence and pass the current memory and storage checks. Detailed testing and verification methods are presented in Chapter 7.

Chapter 2 reviews the background and related work. Chapters 3–4 define the requirements and evaluation method; Chapters 5–7 cover design, implementation and verification. Chapters 8–9 present the results and conclusions.

Chapter 2

Background and Related Work

This non-systematic review focuses on local inference, weight and KV-cache quantisation, and tools relevant to the project’s research questions and design decisions. Different types of claims use different sources of evidence. Primary research papers support algorithmic claims and published experimental results; official model cards and documentation support model characteristics and documented software features. Version-pinned repositories are used for implementation-specific claims. External findings are kept separate from evidence produced by this project. Sources were last checked on 14 August 2026.

2.1 Local Large Language Model Inference

Local LLM inference uses a trained model to generate output without changing its learned weights. In this project, inference runs on the user’s computer. Before generation begins, the runtime prepares the model and memory needed for execution [3].

Inference has two main stages. During *prefill*, the model processes the input prompt and creates the initial state, including the KV cache, needed for generation. In this project, Time to First Token (TTFT) covers prompt processing and production of the first generated token, but excludes model loading. During *decoding*, the model generates one token at a time, adding each new token to the sequence before predicting the next.

Transformer attention creates query, key and value representations when processing tokens [23]. During decoding, keys and values from earlier tokens are needed again. Rather than recalculating them for every new token, they are stored in a *KV cache* and reused [24]. This reduces repeated computation, but the cache requires more memory as more token positions are retained. Section 2.2 explains this memory cost in more detail.

2.2 Memory Use During Local Inference

The stored model size does not necessarily reflect the memory required during inference. The KV cache, temporary data, runtime libraries and the application also consume memory, while Windows and other running processes draw from the same physical RAM [3]. Installed RAM is the total memory capacity of the computer, while available RAM is the amount Windows reports as currently available for use. The amount of available system memory can therefore vary between runs as background activity changes [9]. The Intel integrated GPU shares system RAM, so reported shared GPU memory is not additional memory and must not be counted twice [25].

A useful high-level estimate is:

$$\begin{aligned} \text{estimated peak runtime memory} &\approx \text{model weights} + \text{KV cache} \\ &+ \text{runtime and application overhead.} \end{aligned}$$

The estimate is compared with the available physical memory after accounting for a Windows allowance and safety reserve. These allowances form part of the memory budget rather than the predicted runtime footprint, so they must be applied only once. Since available RAM and runtime allocations can vary between runs, the result indicates whether a configuration is likely to fit but cannot guarantee that it will. Section 6.6 describes the estimation method and its validation status.

For a fixed model, weight memory is mainly affected by parameter count and numerical format, although the amount used after loading may differ from the stored artefact size because the runtime can map, copy, transform or offload model data. The KV cache depends more directly on the inference configuration. Its memory use is affected by the retained context, number of simultaneous sequences, attention layers, KV heads, key and value head dimensions, and cache format [3, 24]. Sharing KV heads can reduce this requirement. Runtimes may also reserve cache capacity in advance or allocate it as the sequence grows, so configured context length and allocated cache memory are not always the same.

A configuration may therefore fit at a short context but run into memory pressure at a longer one. Reducing the KV cache also does not reduce total runtime memory by the same proportion, because the model weights and other runtime costs remain.

Memory measurements depend on what is being measured. A process working set is the part of a process currently held in physical memory, while its peak working set is the highest working-set value recorded for that process [26]. Backend-only and whole-system measurements therefore cover different parts of memory use.

2.3 Numerical Representations and Quantisation

LLMs store their trained weights as numerical values and create other values, including activations and KV-cache entries, while running. The numerical format used for these values affects precision, memory use, data movement and hardware support. Lower-precision formats can reduce storage and memory traffic, but this usually comes at the cost of some approximation error [3].

The formats used in this project range from FP32 and 16-bit formats such as FP16 and BF16 to INT8 or U8 and lower-bit representations. A quantiser replaces higher-precision values with a smaller set of numerical levels. In affine quantisation, a scale controls the spacing between these levels and a zero point can be used to represent real zero. Dequantisation then reconstructs an approximation of the original value. Symmetric quantisation centres the represented range around zero, whereas asymmetric quantisation can shift that range using a zero point. Vector and codebook methods may use different representations.

Quantisation error also depends on how values are grouped. Parameters may be shared across a whole tensor or applied separately per channel, per token or to smaller groups. Smaller groups can handle outliers more precisely, although they also require more metadata and may add processing overhead.

Quantisation can target different parts of inference:

Table 2.1: Different quantisation targets in local LLM inference

Target	What is changed	Main effect
Model weights	Trained parameters stored in the model artefact.	Can reduce weight memory and produce a reusable quantised GGUF or OpenVINO artefact.
Activations	Intermediate values created during execution.	Affects runtime computation and temporary memory without changing the stored model.
KV cache	Key and value vectors retained during generation.	Can reduce cache memory without changing the stored model weights.

Post-training quantisation lowers the precision of weights or activations after the model has been trained and may rely on calibration data. Quantisation-aware training takes a different approach by continuing the training process, allowing the model to adjust to lower-precision computation [3, 27]. Since Granite is not retrained in this project, the evaluation focuses on post-training and runtime quantisation methods.

Weight quantisation can produce a reusable model artefact, such as a Q4_K_M GGUF file or an INT4 OpenVINO model. TurboQuant works in a different way: it changes the KV cache during inference rather than producing a smaller GGUF weight file [11]. Other runtime settings, including context length, batch size, thread count and device offload, can also affect execution without altering the stored weights. Even the precision used at runtime may differ from the stored precision if the runtime transforms model data for the selected device.

Quantisation is generally a lossy process, so converting approximate values back to FP16 cannot recover the information that was discarded. Requantising those values may introduce further error, which is why new weight-quantised artefacts should normally be created from a suitable higher-precision source [28].

Fake quantisation simulates lower-precision values through quantisation and dequantisation while retaining higher-precision storage. This makes it useful for evaluating changes in output quality, but it does not by itself show that physical memory use has been reduced. A claim of memory savings therefore needs evidence that lower-precision data was actually stored or used during execution.

Using fewer bits can reduce the amount of data moved during inference, although conversion and dequantisation may introduce additional work. Performance consequently depends on how the runtime, its kernels and the available hardware handle these operations [3]. A format label provides only a general description of the storage scheme. For example, Q4_K_M does not mean that every tensor is represented using exactly four bits.

2.4 IBM Granite Models

The experiments focus on IBM’s Granite 4.1 Language models for local text generation. The 3B and 8B models are the primary targets, while the 30B model remains a stretch goal for the available Intel Windows hardware. Other Granite families are outside the project scope. IBM offers Granite 4.1 in several sizes and variants, including instruction-tuned and long-context versions, together with official local-inference artefacts released under the Apache 2.0 licence. The instruction-tuned models are used in the chat workflow because they receive additional training for conversation and instruction following [29, 30].

The 3B, 8B and 30B models all use dense decoder-only transformer architectures with grouped-query attention (GQA) [30–32]. With GQA, several query heads share a smaller set of key and value heads, which can reduce KV-cache memory compared with standard multi-head attention. The actual saving still depends on the particular model and runtime configuration.

Table D.2 in Appendix D records the published architecture values used for model selection; 3B, 8B and 30B denote approximate parameter scales rather than file sizes.

Correct generation depends on the tokenizer and chat template remaining consistent with the original checkpoint, including after conversion [30, 33]. Successful model loading alone does not confirm that prompts are being formatted correctly.

The documented checkpoint context does not guarantee that the same context will fit in memory or run reliably on the project hardware.

IBM provides the original checkpoints with BF16 weights in `safetensors` format. The GGUF experiments use BF16, Q8_0 and Q4_K_M weight formats [30, 34, 35]. These weight formats are separate from the KV-cache format selected at runtime.

For OpenVINO testing, the project uses converted IR artefacts with FP16, INT8 and INT4 weight formats. However, producing an IR artefact does not by itself confirm that tokenisation is correct, that the model generates valid output or that execution occurs on the requested Intel device. At the review cut-off, Granite 4.1 was not included in Intel’s list of verified OpenVINO models, although Intel notes that models outside the list may still be compatible [36]. Direct GGUF loading through OpenVINO GenAI was also a preview feature limited to selected architectures, with IR conversion recommended for other models [37]. Compatibility must therefore be assessed for the specific artefact, runtime and device configuration.

2.5 GGUF and the llama.cpp Runtime Route

The GGUF route depends on the interaction between the model file and the particular `llama.cpp` build used to load it. GGUF stores the model data, whereas `llama.cpp` provides the runtime that reads the file and performs inference through one of the backends enabled during compilation. This means that compatibility cannot be judged from the file alone: a GGUF model may load successfully with one build and fail with another if the required backend or operation is unavailable. CPU, Vulkan and SYCL are consequently treated as distinct execution routes, and the corresponding build identity is recorded with each result [3, 38–40].

Before attempting to load the model, some useful information can be obtained directly from the GGUF file. Its contents include the model weights, metadata and tensor records, with the metadata describing properties such as the architecture, context length, tokenizer and chat template. The tensor records provide further information about each tensor, including its shape and data type [38, 41]. Section 6.4 explains the project’s limited metadata scan.

`llama.cpp` is used as the project’s primary GGUF baseline because it can load the required Granite artefacts and includes local command-line tools for both generation and benchmarking. The `llama-cli` executable is particularly suitable here because it does not depend on a web server or an open network port. This fits the offline desktop workflow and gives the TurboQuant experiments a stable point of comparison. To support reproducibility, the baseline is pinned to tag `b9870`; the associated build identity and revision details are documented in the experimental methodology and reproduction material [39, 40].

2.6 OpenVINO Intermediate Representation and the Intel Runtime Route

The OpenVINO route uses Intermediate Representation (IR) models, where the XML file defines the model structure and the BIN file stores constant data such as the weights. The generation workflow also requires the tokenizer, generation configuration and chat template [37, 42].

OpenVINO can manage device placement through `AUTO` or `HETERO`. The `AUTO` mode chooses among the supported devices available on the system, whereas `HETERO` allows different parts of a model to run on different devices when no single device can support all of its operations [43, 44]. Although runtime properties describe the configuration submitted for execution, they do not necessarily show where every operation was carried out. Because of this, the requested device is recorded separately from the available evidence about actual placement during the run.

This preparation process separates the OpenVINO route from the GGUF and `llama.cpp` route described in the previous section. A compatible `llama.cpp` build loads a GGUF artefact without first converting it to OpenVINO IR, whereas OpenVINO first compiles an IR model through Intel’s device plugins. That compilation stage gives the application access to Intel-specific device selection and runtime properties, but it also introduces a conversion step and a dependency on the supporting tokenizer and configuration files. The model must still load and generate valid output while the available device evidence is recorded.

The project kept OpenVINO as its Intel-focused route so that its device and runtime properties could be examined. Later chapters document the model revisions, OpenVINO versions and drivers used in each experiment, along with the evidence collected about device placement.

2.7 KV-Cache Compression and the TurboQuant Route

As tokens are generated, the KV cache holds information from earlier tokens so that the model does not have to calculate it again at every step. With increasing context length, this cache can consume a large amount of memory, which makes compression useful for local inference [10]. Different methods reduce that memory in different ways. Quantisation represents the key and value vectors with fewer bits, token-selection methods retain fewer positions from the sequence, and low-rank methods replace the original vectors with smaller representations. Since each approach changes a different part of the cache, the effects on memory use, generation speed and output quality must be measured separately.

2.7.1 Standard KV-Cache Quantisation and Baselines

For KV-cache comparisons, F16 and BF16 provide higher-precision reference points, while supported lower-precision formats may include Q8_0, Q4_0, U8 and U4. Comparing these formats shows how much memory is saved and whether reducing cache precision changes the generated output.

Bit width alone does not determine how much memory a quantisation method uses. This is because different methods organise and store the data in different ways. KIVI, for example, quantises keys per channel and values per token, while retaining the most recent tokens in a small higher-precision residual cache [10]. Its memory use depends on both the selected precision and how much residual data is retained. As a result, methods with the same nominal bit width can still require different amounts of memory and produce different generation behaviour.

2.7.2 Other KV-Cache Compression Approaches

Lowering numerical precision is one way to shrink the KV cache, but some methods reduce the amount of information stored instead. H₂O keeps recent tokens together with attention “heavy hitters”, which are earlier positions that continue to receive high attention. SnapKV also removes positions, using an observation window near the end of the prompt to decide which ones should remain [45, 46]. Palu works at the vector level rather than the token level by storing keys and values in a smaller low-rank form and reconstructing the information needed for attention [47]. These methods require changes to different parts of the inference process, so they are included as related work rather than used as direct Granite baselines.

2.7.3 PolarQuant, QJL and TurboQuant

PolarQuant starts by applying a random rotation to the KV vector, spreading its information across the rotated coordinates. It then converts those coordinates into a recursive sequence of radii and angles, with the angles stored through low-bit codebooks. When the vector is needed, this stored representation is used to reconstruct an approximation of the original [48]. The paper reports KV-cache compression of more than $4.2\times$ in its long-context experiments.

Quantized Johnson–Lindenstrauss (QJL) approaches the problem through the query–key inner product used in attention. A shared random projection is applied to each key, after which the method stores the sign of every projected value together with the key norm. The query passes through the same projection at higher precision, allowing the stored sketch to estimate the original inner product without reconstructing the full key [49]. Under the assumptions made in the paper, this estimate is unbiased, although it remains an approximation. The paper’s “zero overhead” description refers to the absence of the usual per-block scales and zero points. The projection, key norm and additional computation are still part of the method.

TurboQuant combines low-bit vector quantisation with a correction based on QJL. It first normalises and randomly rotates a vector, then maps each rotated coordinate to a nearby codebook value chosen to reduce mean-squared error (MSE). This approximation leaves a difference from the original vector, known as the *residual*. TurboQuant applies a 1-bit QJL sketch to that residual and uses the resulting estimate to account for part of the remaining error during inner-product calculations [11].

Although PolarQuant and TurboQuant both use random rotations and low-bit codebooks, their internal representations are different. PolarQuant converts the rotated vector into a polar form and quantises its angles. TurboQuant quantises the rotated coordinates directly before applying QJL to the residual. PolarQuant is consequently a separate compression method rather than an earlier stage of the TurboQuant process.

In the experiments reported by its authors, TurboQuant maintained quality at 3.5 bits per channel and showed limited degradation at 2.5 bits per channel [11]. These results apply to the authors’ test conditions and need independent evaluation for Granite on the selected Intel Windows system. Later work comparing TurboQuant with RaBitQ found that it did not perform better in every tested setting, and some published results could not be reproduced [50]. The project therefore evaluates TurboQuant against matched baselines where the configurations are directly comparable, instead of carrying its reported benefits across to the Intel system unchanged.

The presence of TurboQuant code or recognised command-line flags does not, by itself, show that TurboQuant was used during a run. That claim requires evidence from the running system identifying the cache path that was active. The checks used to establish this are described in Chapter 4.

2.7.4 TurboVec as a Separate Vector-Compression Route

Despite the similar name, TurboVec addresses embedding storage rather than model inference. It compresses and indexes embedding vectors, which represent items such as pieces of text and allow similar items to be found [51]. A query vector can search that index and retrieve nearby entries, but this process does not compress Granite weights or reduce its KV cache.

Using TurboVec with Granite would require a separate process to generate and index document embeddings, retrieve relevant passages and add them to the model context. Its storage, retrieval and evaluation are therefore treated as separate concerns.

2.8 Existing Local AI Applications, Inspection and Hardware-Fit Tools

Five tools were selected to compare Granite support, offline operation, inspection, hardware and memory checks, device reporting, KV-cache support and reproducibility across the intended workflow.

Table 2.2: Comparison of selected existing applications and tools, reviewed 14 August 2026

Tool	What it provides	Gap relative to this project
LM Studio [52–54]	Runs compatible GGUF models through <code>llama.cpp</code> . Once the required files are available, it can run offline, and its memory estimates take context length and GPU offload into account.	It does not provide the controlled Granite-specific evaluation or verification of a custom TurboQuant route required by this project.
Hugging Face GGUF viewer [55]	Displays GGUF metadata and tensor information, including names, shapes and precisions.	It does not assess host hardware, full-configuration memory or runtime execution.
llmfit v1.1.9 [56]	Reads CPU, GPU and RAM information, then ranks models by expected fit, speed, quality and context support. Its catalogue includes Granite models, and it can benchmark supported local providers.	Its results help with planning, but they do not validate the exact combination of model artefact, runtime build, cache method and execution device used by this project.
OpenVINO developer tools [57, 58]	Support local IR and GenAI execution, Intel device discovery, runtime properties and benchmarking.	Granite conversion and device support still require direct checking, while a custom TurboQuant route requires separate integration and verification.
GPUStack v2.2.3 [59]	Manages model deployment across GPU systems, including engine selection, scheduling, monitoring and failure recovery.	The worker versions reviewed require Linux, so they do not match the project’s Windows 11 target environment.

The reviewed tools address different parts of the workflow, which means that assessing a single configuration may require users to move between several applications. Across the five versions examined, none combined guided Granite import, model inspection, hardware and memory assessment, configuration selection, runtime device reporting and controlled TurboQuant evaluation within one Windows desktop application. This conclusion applies only to the versions reviewed on 14 August 2026 and does not rule out other tools with similar capabilities.

2.9 Technology Alternatives and Selection Rationale

The technology choices were guided by the project’s Windows 11 and offline scope, its need to support Granite model formats and Intel hardware, and the practical limits of testing, reproducing and maintaining the system within an individual MSc project [13, 14]. Upstream `llama.cpp` served as the GGUF baseline for comparison with the TurboQuant routes [60], while OpenVINO GenAI supplied a separate Intel-focused route with access to device and compilation information [37]. Maintaining both routes increased the conversion, integration and testing effort, but they supported different parts of the evaluation. The alternatives considered and the resulting architectural trade-offs are discussed in Section 5.2.

2.10 Chapter Summary

Local Granite inference depends on the complete configuration, including the model, KV cache, runtime and device. The review of existing tools also identified gaps in the guided workflow intended by this project. Together, these findings shape the requirements, experimental method and architecture presented in Chapters 3–5.

Chapter 3

Requirements, Users, and Project Method

This chapter sets out the intended users, operating conditions, requirements and development method. The requirements are based on the project definition, advice from the supervisor, controlled workflows, feasibility and test evidence, together with the risk, assumption, constraint and licence registers. MoSCoW v1.2 is the current working baseline, while Appendix A contains the full requirements catalogue, revision history and Requirements Traceability Matrix (RTM). The following sections explain the priorities, user tasks, verification approach and change-control process.

3.1 Stakeholders and Intended Users

The primary user is a non-specialist who needs guidance when running models locally and recovering from errors without resorting to terminal commands. A secondary, more technical user may examine configuration details, diagnostics and saved results when investigating a problem.

Later guidance from Dr Dean Mohamedally clarified the focus on Windows and limited-memory Intel laptops, while also identifying schools as an intended setting. A separate project caption mentioned health-care. Neither setting was validated through intended-user research because no interviews, observations or surveys were conducted. The two user roles are therefore treated as proto-personas based on project guidance, requirements and assumptions rather than observed user needs [13, 21]. They describe different kinds of activity rather than fixed user groups, so one person could carry out both roles. Appendix B contains the supporting sources, detailed profiles and linked requirements. It also records their intended tasks and assumed knowledge.

Other stakeholders are:

- **IBM and Intel:** IBM was the named project partner, while Intel technology was part of the technical scope. Neither directly contributed to the development or evaluation.
- **UCL supervisor and examiners:** the supervisor provided academic advice, while the examiners assess the project.
- **Future maintainers:** need recorded dependency versions, stable interfaces, tests, build instructions and saved evidence.

3.2 Operational Context and Assumptions

The application is intended for Windows 11 x64 systems and the Intel hardware available to the project. Its GGUF route must handle large model files stored locally, whereas the OpenVINO route expects a complete IR model folder. Before an OpenVINO model can be loaded, that folder must pass the relevant integration checks. Any claim of support is therefore limited to the configurations tested on the project's hardware.

The memory-fit assessment uses the RAM that is available at the time, rather than the total RAM installed. Shared memory is also taken into account, as explained in Section 2.2. The 4 GB, 8 GB and 16 GB values are used as evaluation budgets and do not mean that every configuration will fit within those memory budgets. Section 4.3 describes the experimental environment.

Once the required models and tools have been downloaded, the tested main workflow can run locally without depending on a cloud service or requiring an open local HTTP port. This result applies only to the tested route. Internet access may still be needed for optional, approved downloads or additional dependencies.

Deployment may be limited by permissions on managed computers, read-only locations, available storage, third-party licence conditions and the MSc project timetable. The project does not include testing with sensitive personal data, such as real patient or pupil records. External files, paths and process arguments must be checked before they are used [20]. The application may report that an NPU is detected, but it claims NPU inference only when suitable hardware and direct evidence from an actual run are available.

Final testing used Windows 11 Home build 26200 on a computer with a 12th Gen Intel Core i5-12450H processor, Intel UHD Graphics driver 32.0.101.7076, about 16 GB of RAM and 264 GiB of free storage. Selected GGUF, TurboQuant and OpenVINO setups worked on this computer, although some OpenVINO tests did not fully confirm which device performed the work. No managed school or healthcare computer was tested, and no sensitive personal data was used.

TurboQuant reduced KV-cache memory in several comparable tests, but it did not always improve speed or output quality. TurboVec remained a separate demonstrator. The memory estimator was not calibrated, the application was not tested with intended users, and the final package licences still need review. Appendix K summarises these outcomes and remaining limitations.

3.3 Functional Requirements

Appendix A contains the 20 Active functional Must Have requirements from the MoSCoW v1.2 catalogue. Table 3.1 brings them together by workflow area, giving a shorter version of the required scope while preserving each individual requirement in the appendix. Their full wording remains in the appendix, along with an indication of which draft acceptance criteria still need to be checked against the final controlled RTM.

Table 3.1: Grouped summary of Active functional Must Have requirements

Area and requirement IDs	Required capability
Application launch F-M01	Open successfully on the target Windows 11 x64 Intel computer.

Continued on the next page

Table 3.1 continued

Area and requirement IDs	Required capability
Model selection and inspection F-M02–F-M05, F-M14	Select, validate and inspect a local model, show its state and relevant metadata, and keep the original file unchanged.
Hardware and memory fit F-M07–F-M09	Read the relevant hardware information, estimate peak memory and explain whether the complete configuration is likely to fit, needs optimisation or has no verified safe option.
Configuration selection F-M10–F-M12	Offer complete valid configurations, provide Automatic, Quality, Balanced and Efficiency modes when valid choices are available, and show the selected configuration before it is launched.
Explanations and recovery F-M06, F-M15	Explain warnings and failures in plain English and provide a suitable recovery action.
Local chat and runtime control F-M13, F-M18, F-M19, F-M28	Generate valid local chat output from at least one supported Granite GGUF model through WinUI without terminal commands. Show progress or output while it is running, support safe cancellation and allow the response to be copied.
TurboQuant and fallback F-M21, F-M22	Run at least one verified TurboQuant-enabled Granite configuration through the application and keep a dependable upstream <code>llama.cpp</code> alternative available if the TurboQuant route is unavailable or fails.

These requirements separate three questions that can be confused: whether the model artefact is valid (F-M03), whether the complete setup is likely to fit in memory (F-M09), and whether it can produce valid output (F-M13). A positive result for one question does not answer the other two. F-M10 therefore requires the model, weight format, cache, runtime, backend and device settings to be assessed together as a single configuration, with the inspection, memory and execution records tied to that same configuration. This describes how the system should be designed, not evidence that the checks have already passed. The component and state models are discussed in Sections 5.4 and 5.7.

3.4 Non-Functional Requirements

Where the functional requirements describe what the application must do, the non-functional requirements define the conditions and qualities under which the wider workflow must operate. These conditions apply across several features, so Table 3.2 groups them by quality area and summarises how they are intended to be assessed. Each requirement states a condition that can be checked in practice, following the approach to measurable architectural characteristics described in *Fundamentals of Software Architecture* [14]. These are planned acceptance conditions and should not be read as completed test results.

Table 3.2: Active non-functional Must Have requirements and planned checks

Quality area and IDs	Planned acceptance focus
Local operation N-M01, N-M02, N-M12	The core workflow must work offline once the required models and tools are available locally. Models, prompts, knowledge files and generated answers must not be uploaded to a cloud AI service. The main inference route must also work without a required local server or open network port.
Responsiveness and cleanup N-M03, N-M04, N-M06	The window must stay responsive and show the current task state. After completion, cancellation, timeout or failure, no unintended child processes or unmanaged temporary files should remain, and an incomplete artefact must not be shown as complete.
Input and repository handling N-M05, N-M09	Paths and process arguments, including spaces, Unicode characters and quotation marks, must be handled safely without unsafe shell commands. The repository must not contain secrets, personal test data or large proprietary model files.
Execution reporting N-M07, N-M10, N-M11	The requested runtime, backend, device and optimisation settings must be kept separate from what the available execution evidence shows actually happened. Experimental options must stay clearly labelled, and any claim that an optimisation was active must be supported by runtime evidence.
Build and release N-M08, N-M14	A clean checkout must build and run the required automated tests. The Windows x64 release must launch on the target computer, and its source revision, checksum and dependencies must be recorded.
Accessibility N-M13	The core workflow must work with keyboard-only use, keep the current focus visible and remain usable at 200% Windows text scaling without critical clipping.

This table only uses a number when it is already defined by the requirements. N-M13 requires the main workflow to remain usable at 200% Windows text scaling without important content being cut off. Cancellation and clean-up are checked using clear pass conditions instead of a time limit. No acceptable error limit is given because the memory estimator was not calibrated.

3.5 MoSCoW Prioritisation

Must Have requirements are necessary for the project aim, core product or academic evaluation. Should Have requirements are important extensions once the Must Have work is stable, while Could Have work depends on remaining time and Won't Have work is excluded from the release. Table 3.3 summarises the v1.2 scope recorded in Appendix A; its counts describe priority and lifecycle, not completion.

Table 3.3: MoSCoW v1.2 scope by priority and lifecycle

Class	Count
Active Must	56
Active Should	14
Active Could	2
Deferred	11
Superseded	10

Continued on the next page

Table 3.3 continued

Class	Count
Excluded	18
Total	111

The application must include at least one verified TurboQuant integration. TurboQuant is therefore part of the product scope as well as a separate research subject. F-M21 sets the minimum expectation: at least one verified TurboQuant-enabled Granite configuration must run through the application. F-M22 provides the corresponding safeguard by retaining an upstream `llama.cpp` route if the TurboQuant route is unavailable or fails (Appendix A). CHG-013 originally kept this limited integration as an experimental Must requirement and moved model downloading, multi-turn chat and persistent weight processing into the Should Have scope. Later evidence allowed only the verified GGUF TurboQuant 3-bit configuration, `GGUF-CURRENT-08EF-CPU-TURBO3-COMPAT-01`, to use the normal supported path. Other TurboQuant configurations remain experimental or unavailable until they have enough supporting evidence. This leaves model assessment and local generation in the first-release workflow while reducing the mandatory scope.

R-M02 requires the project to identify the TurboVec implementation and decide its release role. The feasibility evidence can support R-M02 and RQ-TV, but it does not show that TurboVec was integrated into the application or change the Deferred status of R-M13.

MoSCoW v1.2 was approved as the working development baseline on 14 July 2026 through CHG-013 and CR-013. Supervisor approval was not recorded and is not claimed. CHG-019 later updated the controlled workbook to RTM v1.3.1 without changing the first-release scope. No final release tag had been created by the report cut-off.

3.6 User Journeys and Use Cases

The journey maps P1’s main task to the project requirements, taking the user from model checks and hardware assessment through configuration selection to inference. It represents the intended workflow as a scenario and task-flow model, rather than evidence that the application already enforces every step [13,21].

1. The user selects a model stored on the computer. If an approved download option is available, the download must finish and pass its integrity checks before the model can be used.
2. The application checks and inspects the model. If conversion is needed, it must use a supported route, and the converted model must be checked and inspected again.
3. The application checks the hardware and available memory, then assesses whether the complete configuration is likely to fit. The user reviews and confirms the configuration before it is launched.
4. The user enters a prompt and starts inference. The application shows progress or streamed output, supports safe cancellation, and lets the user review and copy the response. Requested settings are recorded separately from what the available execution evidence shows actually happened.

Appendix B lists the alternative paths for invalid input, not enough memory, conversion, cancellation, runtime failure and fallback. If a fallback is used, the application must explain the configuration change and the user must confirm it before launch. The fallback must not be reported as a successful run of the original configuration.

Appendix B contains two user profiles and six detailed use cases. They cover model selection, inspection, hardware checks, configuration, local chat, cancellation and error recovery. They describe how

the application is intended to work, not how real users behaved. No user walkthrough or usability study was completed, so this stays as a project limitation.

3.7 Requirements Traceability and Verification Plan

The RTM links each requirement to the planned work, relevant experiments or controlled documents, together with the evidence needed to check it. These links can also be followed in reverse, connecting components, tests and report claims to the requirement or project need that supports them. Appendix A contains the detailed mappings and explains how planning objectives O1–O11 map to report objectives CO1–CO5.

A traceability link, populated evidence path or high completeness value does not by itself show that a requirement has been fulfilled or that the referenced evidence exists and satisfies its acceptance criterion. Verification determines whether the stated acceptance criterion has been met, whereas validation considers whether the requirement and resulting workflow address the intended need [12].

Table 3.4 shows the planned trace for F-M21 in the application. This is separate from the matched experimental comparison required by R-M01.

Table 3.4: F-M21: planned trace from requirement to application evidence

Trace element	Recorded or required link
Source and allocation	Appendix A links F-M21 to Section 5.2.11 of the Project Definition and to work packages QX-03 and QX-04.
Objectives and research questions	F-M21 is planned to support objectives O5, O6 and O8 and research questions RQ1, RQ2 and RQ4. These are planning-objective IDs and are different from the condensed report objectives CO1–CO5, as explained in Appendix A.
Required outcome	A pinned TurboQuant-enabled Granite configuration must be selected through WinUI, load successfully and generate valid chat output through the application. Evidence is also required for TurboQuant activation, the runtime, backend, device and cache state supported by the available execution evidence, and cancellation. The upstream fallback is covered separately by F-M22.
Planned verification	The planned checks are an end-to-end application test and a TurboQuant activation check. AC-F-M21 identifies the planned acceptance test and evidence reference.
Planned evidence location	The evidence is expected to be stored in docs/evidence/requirements/F-M21/ . This path shows where the evidence is intended to be stored; it does not show that the evidence has been produced.
Evidence status	F-M21 can only be considered fulfilled after the final evidence has been checked against its acceptance criterion and the assessed application build. Results from a standalone runtime do not show that this application requirement has been met.

R-M01–R-M12 and R-M14 define the research requirements addressed by the methods in Chapter 4. For R-M03, the outcome may be either an official OpenVINO baseline or a reproducible blocker, although a blocker does not demonstrate successful execution on the device. R-M12 assesses whether configurations fit within the defined memory budgets, whereas R-M14 measures memory-estimator error. Since these are separate checks, each requires its own evidence and final status.

Appendix A contains all 111 requirements from MoSCoW v1.2 and the available RTM links. CHG-019 later updated RTM v1.3.1 without changing the requirements or their priorities and scope. The appendix was not created specifically from the RTM v1.3.1 workbook, so the controlled RTM remains the main

source. Incomplete, failed, deferred and excluded requirements are still shown. An evidence path by itself does not prove that a requirement was completed.

3.7.1 Experimental Evidence Coverage

The release `unified-final-results-2026-09-01-v2` contains five inference campaigns for RQ1–RQ3, while `EXP-TV-COMP-001` contains the separate TurboVec study. Blocked and unavailable attempts were kept, but no performance or quality values were invented for them. The memory-budget findings are still incomplete. R-M14 remains open because the memory estimator was not validated, while RQ4 remains open because the application evaluation was not completed. Chapter 8 explains these limits.

3.8 Project Management and Change Control

Development followed the incremental approach described in Section 1.6 [13]. Stable work-package IDs and the Active Task Catalogue were used to organise and track the work. Where later work depended on earlier results, those results were intended to be checked before being used. Partial implementation alone was not enough to justify integration. Planned feasibility checks for OpenVINO, TurboQuant, TurboVec and device support were to use pinned software in a recorded test environment. This approach was informed by risk-reduction prototyping and architectural trade-off analysis [12, 14]. Upstream `llama.cpp` was kept as the GGUF control baseline and intended fallback if the TurboQuant route failed.

The supporting TurboVec run completed the index lifecycle, but none of the compressed formats passed all Gate A criteria. The `DEMONSTRATOR_ONLY` decision therefore supported R-M02 without showing that TurboVec was integrated into the application or changing R-M13’s Deferred status (Section 8.8).

G-M01–G-M09 cover the project definition, requirements and traceability, architecture records, risks, assumptions, constraints, licences, change records, evidence and release documentation. Any change to the requirements or scope must record why it was made and which requirements, work packages and checks it affects. CHG-013 records the scope changes discussed in Section 3.5, while Appendix A contains the revision history. The final release must bring these records together in a tagged evidence pack.

The planned Definition of Done requires reviewed evidence that the relevant acceptance criteria have been met, including any needed success, failure, cancellation and cleanup checks. Build and test records support development checkpoints, while experimental conclusions need their own measured results. Closing a task or passing CI is not enough to confirm that a requirement has been met. Chapter 7 presents the test strategy and outcomes.

CHG-013 and CR-013 set the first-release requirements on 14 July 2026. PR #14 recorded the MoSCoW v1.2 and RTM v1.3 planning baseline, but it did not show that every product requirement was complete. PR #26 reviewed the risk, assumption, constraint and licence records. CHG-019 later updated their status in RTM v1.3.1. PR #108 improved the experiment evidence and documentation and restored the Workbook 05 checks without repeating the experiments. Supervisor advice helped focus the project on Windows and Intel hardware, but no formal approval is claimed. The application checkpoint is `a4e35b7c1d2cce0a3890d25e89fb63adf1b7cc02`. There was no final release tag, and the final build checks had not passed by the report cut-off. The remaining risks and assumptions can be found in Appendix K.

Chapter 4

Experimental Methodology

This chapter explains how the project evaluated local IBM Granite inference, KV-cache quantisation and the separate TurboVec study. It separates model weights from the KV cache, distinguishes successful execution from valid output and compares results only when the main experimental settings match.

4.1 Evaluation Design and Research-Question Mapping

Table 4.1 shows how each research question was evaluated.

Table 4.1: Operational mapping of research questions to controlled evidence

RQ	Evaluation	Observed variables	Interpretive boundary
RQ1	Five inference routes spanning upstream and modified <code>llama.cpp</code> and OpenVINO repositories	Load, generation and output health; requested and recorded backend; activation evidence; failure and fallback records	Compatibility is limited to the captured commits, models, host and runtime conditions
RQ2	Matched standard-cache and TurboQuant cases within a route, plus 15 matched experimental/official OpenVINO configurations	KV allocation where available, peak working set, TTFT, decode throughput, quality and stability	The planned standalone EXP-TQ-COMP-001 KV-cache result package was not completed; retained matched observations remain usable
RQ3	Measured memory and safety-frontier observations on the approximately 16-GB host	Peak working set, KV allocation, context, minimum available RAM and safety-floor outcome	The planned calibrated 4/8/16-GB estimator validation was not completed

RQ	Evaluation	Observed variables	Interpretive boundary
RQ4	Frozen application tasks APP-EVAL-01-APP-EVAL-12	Task completion, recommendation correctness, recovery, accessibility and usability	No completed result set exists under the frozen protocol; benchmark workbooks cannot substitute for user evaluation
RQ-TV	EXP-TV-COMP-001: Exact embeddings compared with TQ2, TQ3 and TQ4	Retrieval quality, query latency, index storage and lifecycle reliability	Separate feasibility study; no application or RAG integration

Each configuration had a route-specific test-case ID. The controlled settings included the repository revision, model, weight format, cache format, backend and context where applicable. Outcomes included execution status, output health, performance, memory and quality. Repetitions and individual scoring records were lower-level observations, not separate configurations.

4.2 Evidence Authority, Provenance and Data Grain

The final authority is `unified-final-results-2026-09-01-v2` under [docs/testing/final-results](#). It contains normalised data, provenance indexes, hashes, validation records and reproduction material. Historical evidence was included only when its identity and source could be checked. Missing values were not changed to zero, and recorded precedence rules resolved conflicts with older summaries. Appendix F gives the evidence grains, counts and main anomalies.

4.3 Hardware, Software and Repository Identity

The study used one upstream `llama.cpp` baseline, two independent community forks, one experimental OpenVINO fork and one official OpenVINO retest. TurboVec was evaluated separately as a vector-index library. Table D.1 in Appendix D records their pinned identities and coverage.

Where recorded, experiments ran on a Windows 11 Lenovo laptop with an Intel Core i5-12450H, Intel UHD integrated graphics and approximately 16 GB of physical RAM. Missing environment or backend fields were reported as unknown; they were not treated as proof of CPU execution or fallback.

Weight and cache formats were recorded separately. Repository-specific labels such as `turbo3`, `tq3_0` and `tbq3` were not treated as equivalent. The final TurboVec run used the same embedding weights but a different Python version and OpenVINO export from the blocked formal environment, so it provides supporting rather than replacement evidence.

4.4 Controlled Execution Protocols

Each campaign followed its preserved route-specific matrix and sequence rather than a reconstructed universal command. Common controls were: validate required artefacts; record the intended model, weights, cache and backend; perform a pilot or load check where specified; exclude declared warm-ups; execute formal repetitions; capture timing and memory; run the applicable quality prompts; and terminate

and clean the process tree after a safety or runtime failure. Route-specific commands, context settings and dependencies are retained under each package’s **reproduction** directory.

Requested settings were kept separate from what actually happened during execution. For the **llama.cpp** routes, the protocol recorded the requested context length, cache format, device and GPU-offload settings, together with the available evidence for device placement, cache state and fallback behaviour.

The principal repetition and aggregation rules were:

- **Upstream llama.cpp:** prompt and decode throughput are medians of three **llama-bench** samples, except UL-13, which has five. KV allocation is the median of three deduplicated allocations. TTFT is the median of three server samples, and peak working set is the median of repetition-level process-tree maxima. Benchmark and server observations are not represented as one synchronised run.
- **AtomicBot:** a pilot and one warm-up were excluded before three validated repetitions. Throughput and TTFT use the campaign’s preserved three-repetition summaries; peak working set is the maximum of the three measurements. CPU/GPU summaries are arithmetic means of the three repetition means.
- **animehacker:** performance values are medians of exactly three included formal samples, while peak working set is the maximum of those samples. Safety-classified rows were not forced past the memory reserve.
- **OpenVINO:** each passed configuration has three benchmark repetitions. Decode throughput is their median and peak working set is their maximum. TTFT is taken from the repetition selected by median decode throughput; it is therefore not necessarily median TTFT.
- **TurboVec:** Exact, TQ2, TQ3 and TQ4 were tested at 1,000 and 10,000 chunks. Each scale used 128 fixed queries. Every configuration used five warm-up batches followed by 30 measured full-query batches. The full comparison was repeated five times, with the configuration order changed between repetitions to reduce ordering bias.

Before each TurboVec scale, the computer was left idle for five minutes and then monitored for 60 seconds. Testing only began if CPU use, available memory, power mode and background activity met the fixed readiness rules. Both scales passed these checks. After testing, an independent validator checked that each result package contained five valid repetitions.

The OpenVINO throughput workload used approximately 23–24 input tokens and 32 generated tokens. It measures short-request behaviour, not long-context cache growth. The quality harness allowed up to 128 generated tokens. These bounds are material when interpreting peak memory, TTFT and completeness.

4.5 Metrics and Calculations

4.5.1 Performance and Memory

Decode throughput was measured in generated tokens per second; TTFT, defined in Section 2.1, was measured in milliseconds; and peak process working set in bytes, later displayed in GiB. Prompt throughput, CPU/GPU utilisation, minimum available memory and explicit KV allocation were retained only for routes that captured them. Peak working set and KV allocation are not interchangeable: the former includes much of the mapped model and runtime process, whereas the latter describes the cache allocation exposed by the route. TurboVec serving bytes describe serialised index storage, not process memory.

For a matched baseline value (B) and candidate value (C), percentage change was interpreted as

$$\Delta\% = 100 \times \frac{C - B}{B}. \quad (4.1)$$

The sign was retained rather than converting every difference to an “improvement”. Memory reduction was reported as $100 \times (B - C)/B$. No value was calculated when the baseline was zero, missing or methodologically unmatched.

4.5.2 Generative Output Quality

The final OpenVINO suite, `OPENVINO-SECTOR-EXPERIENCE-QUALITY-v3`, contains 48 prompts covering healthcare, education, numerical/statistical reasoning and general tasks. It contains 16 short, 16 medium and 16 long prompts, including supplied facts, format constraints and safety-sensitive requests. Every prompt contributes three deliberately unequal criteria: a 5-point primary criterion for correctness, numerical accuracy or safety; a 3-point fact-retention criterion; and a 2-point instruction-following criterion. Thus each configuration has 144 criterion records and 480 available points:

$$Q_{10} = 10 \times \frac{\sum_i w_i p_i}{480}, \quad (4.2)$$

where p_i is the binary pass result for criterion i and w_i is 5, 3 or 2. Deterministic output-health gates reject empty, degenerate or repeated- character responses. The remaining criteria use specified lexical checks such as `contains_all`; they are reproducible but do not amount to human semantic adjudication. Correct synonyms can produce false negatives, keyword inclusion can produce false positives, and 128-token truncation can penalise an otherwise valid answer. One generated answer per prompt means that quality scores do not estimate generation-to-generation variability. No human/automated judge agreement or inter-rater calibration statistic was collected.

The three `llama.cpp` routes used `GTQ-PROMPTS-v1` and `GTQ-QUALITY-RUBRIC-v1`. Its dimensions weighted correctness at 30%, instruction/format adherence at 25%, completeness at 20%, relevance and coherence at 15%, and stability/integrity at 10%, with conservative caps following deterministic checks. The upstream route used temperature 0, top-p 1, seed 42 and up to 256 generated tokens where recorded; four prompts were scored for most rows and six for UL-05. AtomicBot and animehacker used six prompts per eligible row, but their adjudication versions differ and AtomicBot is explicitly limited/provisional. These legacy scores are valid within their own route but are not ranked against OpenVINO v3 scores.

4.5.3 TurboVec Retrieval Quality

The earlier TurboVec feasibility run used a small 30-vector corpus. The final supporting rerun used fixed collections of 1,000 and 10,000 different chunks. Each scale used 128 fixed queries: 32 development queries and 96 evaluation queries. The test material covered education, non-sensitive healthcare, technical and general topics. It also included distractors, similar passages, tables and questions whose answers were not present.

The same frozen embeddings were used for Exact, TQ2, TQ3 and TQ4 at each scale. Exact floating-point search was the reference. Recall@10 measured how many relevant chunks appeared in the first ten results. Recall@10 against Exact measured how much of the Exact top ten was kept by a compressed index. Relative nDCG@10 measured how well the compressed index kept the ranking quality of Exact. The experiment also recorded source accuracy, page accuracy, p95 query latency and the saved index size.

Lifecycle testing checked whether each index could be built, saved, loaded and queried correctly. It also checked that damaged index data was rejected and that temporary resources were cleaned up.

At 10,000 chunks, a compressed format passed Gate A only if it completed all lifecycle checks and met every one of these limits:

- Recall@10 of at least 0.90;
- relative nDCG@10 of at least 0.95;
- p95 latency no more than 1.10 times the Exact latency; and
- a storage ratio of at least 2.00.

The storage ratio was calculated by dividing the Exact index size by the compressed index size. A value of 2.00 therefore means that the compressed index used half as much storage as Exact. The limits were fixed before the final decision and were not changed after the results were known. The 100,000-chunk scale was optional and was only intended as a later extension if the 10,000-chunk results supported further testing.

4.6 Status, Safety and Comparison Rules

Terminal status was assigned as follows: **passed** means that the required route stages completed; **failed** records an attempted but invalid or rejected outcome; **blocked** means a prerequisite or safety gate prevented the intended execution; and **artifact_unavailable** means that no validated model artefact was supplied. A passed runtime row can still have invalid output and a zero quality score. Conversely, blocked and unavailable rows have no fabricated performance or quality value.

Direct comparison required matching model identity, weight format, cache format, backend class, prompt or workload, metric definition, denominator and aggregation. The guarded comparison matrix classifies performance as direct only for the 15 configurations shared by the two OpenVINO routes. Other cross-route performance values are descriptive; OpenVINO and legacy quality scores are not combined. Even a direct protocol match does not isolate one causal factor when repository implementation and runtime version also differ.

Runs stopped when conversion or loading failed, the available-RAM floor was crossed, the required codec or artefact was absent, optimisation activation could not be verified, output was unusable, or process cleanup could not be confirmed. The 2-GiB emergency reserve used by the later OpenVINO and guarded animehacker work was not lowered simply to obtain more rows.

4.7 Analysis Boundaries, Ethics and Reproducibility

Analysis proceeded from availability to execution validity, then performance, memory, quality and practical trade-off. Route-local matched cases were given priority; the 15 OpenVINO matches were considered next; unmatched results were reported only as bounded observations. The analysis reports numerical ranges and percentage changes where justified, but does not claim population-level confidence because no power analysis or inferential sampling design was used.

The healthcare and education prompts were synthetic evaluation material; the benchmark evidence contains no claim of testing with real patient or student records. These prompts assess bounded response properties, not clinical safety, diagnostic competence, educational effectiveness or suitability for autonomous decision-making. Product recommendations would require domain-expert review, privacy assessment and user evaluation beyond this experiment.

Reproduction depends on the route package, fixed code version, model files and route manifests. Power settings, background activity, temperature, test order and device tracing were not always recorded and remain limitations. The appendices and evidence package contain the prompts, scores, outputs, commands and hashes.

Chapter 5

System Architecture and Design

This chapter explains how the system is organised, how its main parts work together and how it handles failures. The main diagrams show the overall design, while the appendix provides more detail.

5.1 Architectural Drivers

The architecture supports local model execution on Windows PCs with limited resources, guided by five main concerns:

Local operation and privacy.

Models, prompts and responses stay on the device during the core workflow (N-M01–N-M02), while model downloading is handled as a separate network operation.

Responsiveness and recovery.

Asynchronous tasks and separate worker processes help keep the interface responsive, while progress updates, cancellation and cleanup support long-running operations (N-M03–N-M04, N-M06).

Safe and clear decisions.

Imported files and paths are checked before use, and process arguments are passed safely (N-M05). Configuration choices depend on evidence, memory and storage checks, with estimates and requested settings kept separate from confirmed runtime behaviour (N-M07, N-M10–N-M11).

Maintainability and reproducibility.

Shared workflow logic is kept separate from code for each runtime, while defined interfaces and fixed package versions support testing and repeatable builds (N-M08, N-M14). This also adds work to package the application and coordinate its processes.

Accessible guidance.

Keyboard support and Windows text scaling (N-M13), alongside clear warnings and recovery actions, help users follow the workflow and understand its limits.

5.2 Architecture Alternatives and Decisions

Table 5.1 summarises the main design choices and their trade-offs. Not all alternatives were built or tested. Appendix C provides the supporting architecture diagrams.

Table 5.1: Main architecture choices and trade-offs

Design area	Choice and reason	Alternative and trade-off
Windows interface	WinUI 3 provides a native interface for the Windows 11 target.	WPF is another Windows option, while .NET MAUI supports other platforms. The chosen interface remains Windows-specific.
Separate processes	Native model operations run in child processes, helping keep runtime failures separate from the UI process.	One process would reduce startup and communication work. Child processes need careful control and cleanup; they are not security sandboxes.
Runtime integration	Adapters control workers and command-line tools, keeping runtime details outside the interface code.	Direct native calls from the UI process would reduce communication work but increase dependence on native libraries. Workers may still use native bindings internally.
Shared workflow	Common services and one chat interface reduce repeated code. Adapters handle runtime differences.	Separate interfaces could be easier to customise but would duplicate common features. An overly general interface could hide important differences.
Model inspection	Quick metadata checks precede deeper inspection, avoiding immediate full-model loading.	Full parsing or loading checks more of the model but takes more time and memory. Quick checks cannot prove valid generation.
Configuration choice	Automatic selection uses eligible options and shows the chosen configuration for user confirmation.	Unrestricted settings offer more control but allow untested combinations. Memory estimates cannot guarantee successful execution.
Output handling	New model files are staged and checked before use. Runtime-only changes are recorded separately.	Direct writes reduce staging overhead but risk incomplete files. Staging needs storage and cleanup; export is a separate action.

5.3 System Context

Figure 5.1 shows how Granite Edge AI interacts with the user, Windows services, local model files and the approved download source. The system boundary includes the desktop interface, application services, local workers and runtimes, as well as managed models, outputs and chat history. It covers the whole application rather than a single process.

The user selects a model, confirms a configuration and enters prompts, while the application shows progress, results and chat responses. Windows provides file and folder selection, while hardware APIs supply processor, memory, graphics and storage information for compatibility checks. Local model files are not automatically trusted and must pass the required checks before use.

The approved download source sits outside the local system boundary, with selected packages downloaded over HTTPS and checked against their fixed file size and SHA-256 hash before model inspection. Once the required models and tools are available, the core workflow runs locally without sending models, prompts or responses to a cloud AI service. The GGUF and OpenVINO runtimes sit inside the system boundary because they run locally as part of the application.

““latex

5.4 Component Architecture

Figure 5.2 groups the application into presentation, application services and runtime integration. These groups show how responsibilities are divided, instead of representing separate programs or matching individual source folders.

The presentation components handle navigation, model selection, configuration review and chat. The application shell manages page changes and passes model and inspection data between features, while

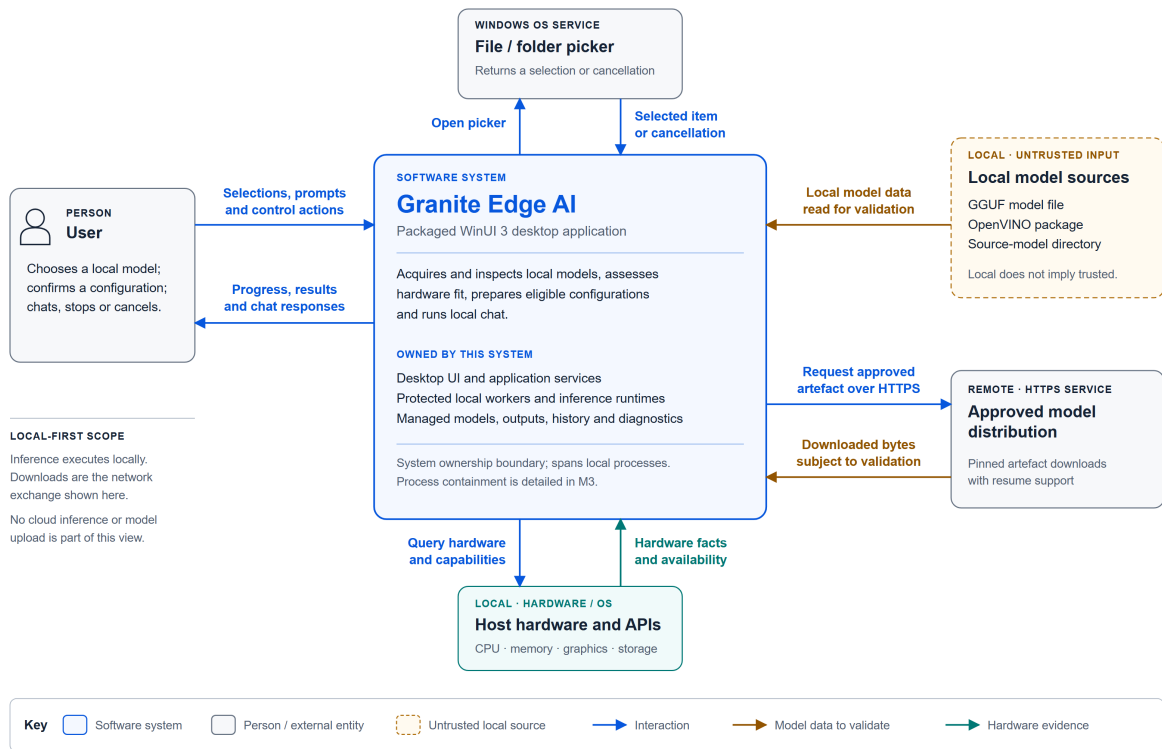


Figure 5.1: System context and main interactions of Granite Edge AI.

views and view models show progress, results and available actions.

Application services manage inspection, compatibility checks, configuration selection and optimisation. They check evidence and available resources before preparing an execution plan, keeping the main validation and planning rules separate from the display code. Optimisation may prepare model files or select runtime settings, instead of always rewriting model weights.

Runtime clients and adapters pass requests to the appropriate runtime and return progress updates, results and generated text. Process control manages arguments, communication, cancellation and cleanup, while separate workers carry out deeper model inspection and runtime execution. GGUF and OpenVINO keep their own runtime handling but share parts of the workflow and chat logic, avoiding repeated code while preserving their differences.

Services and adapters use local models, approved downloads, Windows hardware information and managed storage. The arrows show requests, while results and progress updates return to the interface. Defined interfaces allow components to be tested with replacement dependencies, although managing communication between components and processes makes the design more complex.

Figures C.4 and C.5 show the detailed compatibility and optimisation flows.

5.5 User Interface and Navigation Design

The interface¹ uses XAML views, view models, coordinators and code-behind instead of a strict Model–View–ViewModel (MVVM) structure.

The application shell displays the current page inside a **Frame**, with a separate five-step indicator that stays visible during setup and disappears when chat opens. This gives GGUF and OpenVINO the same navigation structure while allowing different inspection logic. The shell checks that requests come from

¹Screen layouts: [Figma designs](#).

Logical architecture — design overview

Responsibilities shown; verified delivery is assessed separately.

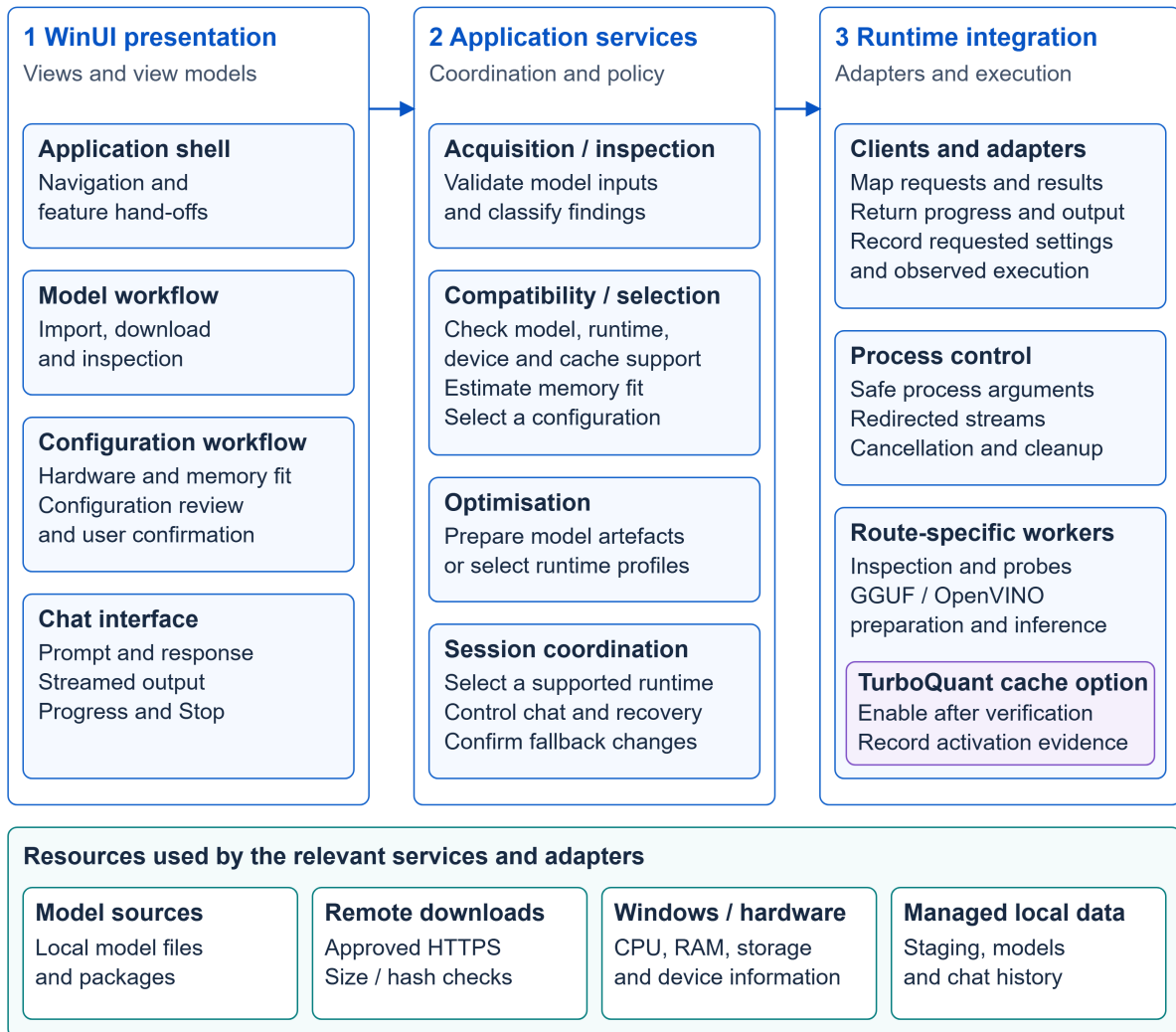


Figure 5.2: Logical components and their main responsibilities.

the current page and removes its event subscriptions when it is detached, helping prevent results from an earlier operation from moving the current workflow forward.

Long-running tasks run asynchronously, with pages using `DispatcherQueue` when background updates need to reach the UI thread. Shared theme resources keep colours consistent, while progress and available actions show the operation's current state. Page setup, removal and code-behind logic still need testing alongside the services.

Figures C.1 and C.2 expand on the user journey and import sequence.

5.6 Runtime Processes and Deployment

Figure 5.3 shows how the application runs on Windows 11 x64, with the WinUI parent process managing the interface and worker clients, while child processes carry out deeper inspection, hardware checks, model preparation and inference. Each group may contain several programs, which do not necessarily run at the same time.

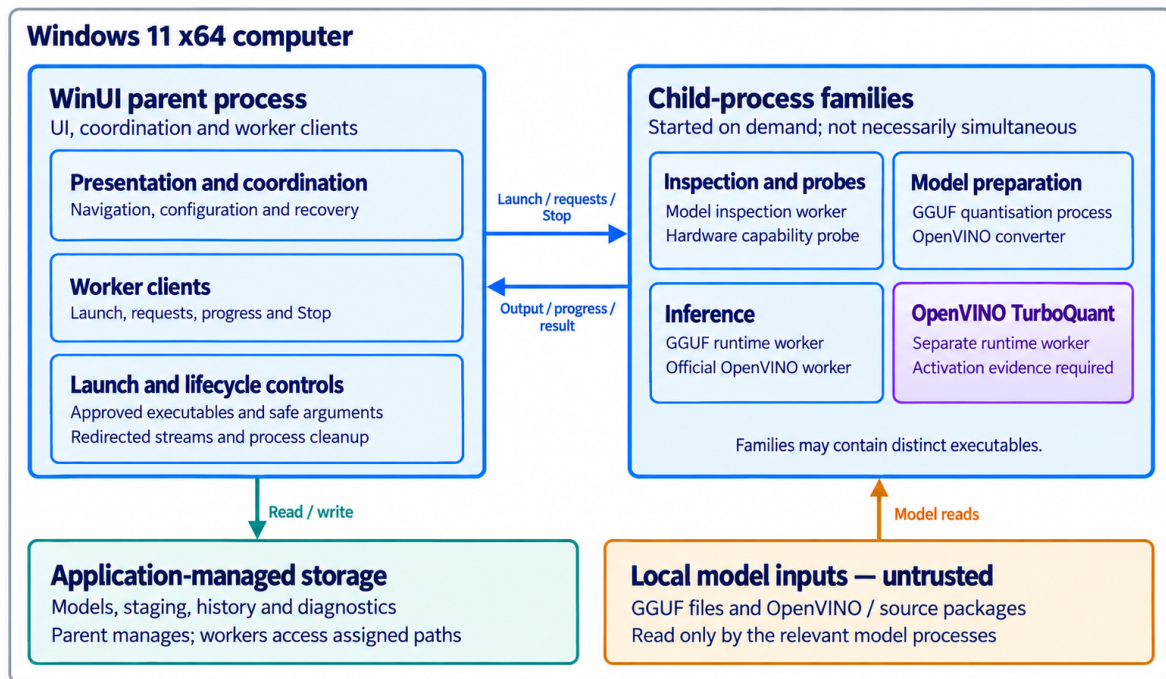


Figure 5.3: Local process groups and storage responsibilities.

Each route uses a specific worker package and runtime dependencies, with required components checked against manifests before launch. Clients send requests and receive progress and results through protocols for each route. Arguments are passed without joining them into shell commands, while redirected streams carry messages and diagnostic information, with time and output limits where needed.

The TurboQuant GGUF adapter uses a server at `127.0.0.1`, so this route opens a local listening port. OpenVINO has separate official and TurboQuant worker packages, with the selected route subject to its own evidence checks.

The parent manages local storage and gives workers the paths they need, while temporary outputs must pass checks before being treated as completed results. Imported models are not automatically trusted. Cancellation and shutdown manage cleanup, with Windows Job Objects helping stop child processes where these launch controls are used. These controls help limit the impact of runtime failures but do not

provide a complete security sandbox.

Figure C.3 details the runtime interactions.

5.7 Lifecycle, Cancellation and Recovery

Figure 5.4 divides the intended workflow into choosing and checking a model, confirming and preparing a configuration, and running chat with recovery. These stages are not individual screens, and recovery options depend on what happened during the operation.

Inspection and compatibility checks decide whether the workflow can continue, while missing evidence is treated differently from a confirmed unsupported result. Memory fit is still an estimate, and the selected plan is checked again before execution. New model files must be checked, but this does not necessarily reopen the inspection screen; runtime-only cache settings do not require a new weight file.

Cancellation is treated separately from failure and success. During hardware inspection, an accepted cancellation belongs to the current attempt and enters a stopping state, preventing a later success from moving the workflow forward. Checks on the current page and operation help ignore old results when the user changes models or starts again.

During chat, Stop requests that generation end while keeping the text already received visible, although the session may need to recover before another prompt can run. Failures show the recovery actions available for that state instead of automatically retrying or silently changing the configuration. The return paths show which stage needs attention after a problem, but do not mean that every screen offers retry or fallback.

5.8 Security, Privacy and Ethical Design

The core workflow runs locally once models and tools are available, with approved downloads handled separately, although chat history still saves prompts and responses on the device. The shared fault reporter stores limited error codes and categories in memory instead of chat content or full exception details; other diagnostic records may work differently.

File checks, worker-manifest checks, safe process arguments and controlled output paths help protect against untrusted inputs, while temporary outputs must pass checks before being treated as complete. These safeguards do not prove complete process isolation, minimum access permissions or licence compliance, which need separate review. Memory estimates and successful text generation also do not prove accuracy or suitability for healthcare or education, which require separate safety and ethical assessment.

Figure C.6 shows the data flows and boundaries.

5.9 Design Evolution

The controlled workflow update of 14 July 2026 records two main clarifications. CHG-WF-004 replaced silent route selection and automatic fallback with user confirmation, while separate execution-plan checks in the application reject differences between the selected settings and those prepared for execution.

CHG-WF-005 and CHG-WF-008 separated new model files from runtime settings, and weight compression from KV-cache compression, making clear that not every optimisation creates new weights.

Reported cancellation and old-result problems led to checks that cancellation belongs to the current attempt and results belong to the current page. These prevent accepted hardware cancellation from later advancing the workflow as a success and help ignore old results, but do not replace wider release testing.

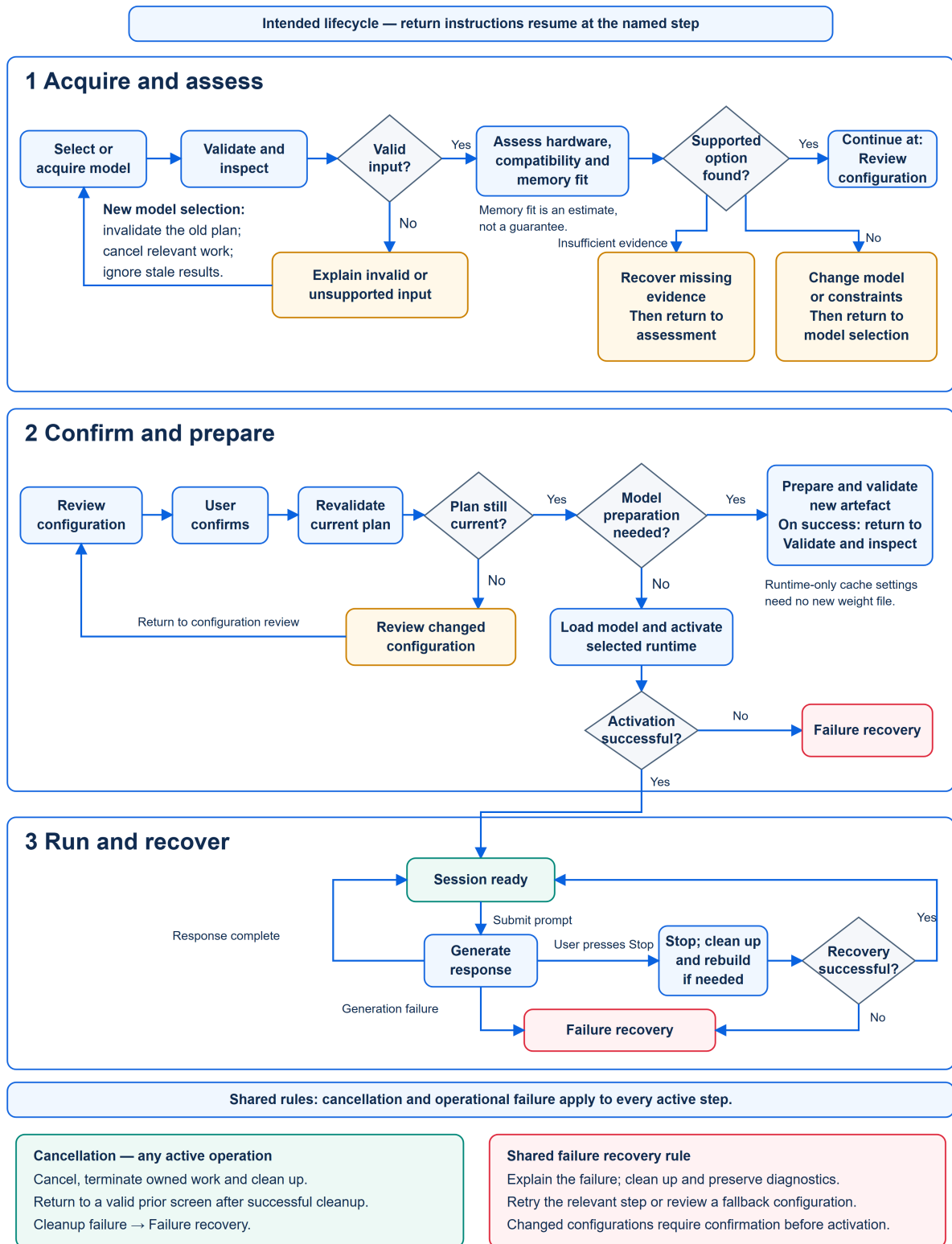


Figure 5.4: Intended lifecycle, validation decisions and recovery paths.

Chapter 6

Implementation

This chapter explains how the Windows application was built, including model inspection, hardware checks, optimisation and local chat. It also explains how the application handles progress, cancellation and errors, using short code examples that shows key decisions.

6.1 Development Environment and Repository Structure

The application was built in C# and XAML using WinUI 3 for Windows 11 x64. It uses Windows App SDK 2.2.0 and Windows SDK Build Tools 10.0.28000.2270. The [global.json](#) file selects .NET SDK 10.0.301 and allows later patch versions, while the application targets `net8.0-windows10.0.19041.0`. Debug and Release configurations are used for development and deployment builds.

The application code is grouped into feature folders for model import, inspection, hardware checks, optimisation and chat. Shared contracts and compatibility logic are in `shared/`. Code for communicating with worker processes and checking hardware is in `infrastructure/`. Runtime adapters and worker programs are in `runtime/` and `workers/`. The `tests/` folder contains unit, contract, integration and end-to-end tests.

Git commits, branches and pull requests record changes brought into `main`. GitHub Actions is set up to build the x64 Release application and run unit and WinUI tests on Windows. Chapter 7 discusses the results. Supporting scripts are in `scripts/`, and project and testing records are in `docs/`. The `experiments/` folder holds experiment inputs, manifests, raw outputs and processed results. The collected final evidence is in [docs/testing/final-results/](#).

6.2 WinUI Application Shell and Navigation

The main window contains the title bar and a root `Frame` that opens `OnboardingShellPage`. A second frame inside the shell shows the current page. A five-step indicator guides the user through model selection, inspection, hardware checks, configuration and chat. It stays visible during setup and disappears when chat opens. Shared theme resources keep colours consistent across the window and pages.

The shell controls navigation and passes model and inspection data between pages. When a page is no longer active, the shell disconnects its event handlers. Before opening hardware inspection, it checks that the request comes from the current model-inspection page and that the inspection data is valid. This helps prevent old results from moving the workflow forward. When the application closes, the main window waits for the shell's cleanup routine to finish.

6.3 Model Import Workflow

Users can select a local model with a Windows picker or drag and drop one file or folder onto the import page. The GGUF picker shows only `.gguf` files. For OpenVINO, users select the model folder rather than choosing XML and BIN files separately. Both methods use the same process to check the location and identify the model type. GGUF files receive a quick scan, while OpenVINO folders are checked for matching XML/BIN files in a recognised layout before detailed inspection.

The checks run asynchronously, allowing the interface to remain responsive. Each selection has an identifier and a cancellation token, preventing results from an earlier selection from replacing the current one. Continue is enabled only when the initial checks pass. If an input is invalid, the page explains the problem and lets the user choose another model. For accepted GGUF files, the model card shows available details, including the name, file size, architecture, quantisation and declared context length. Passing these checks does not yet confirm runtime support or hardware fit.

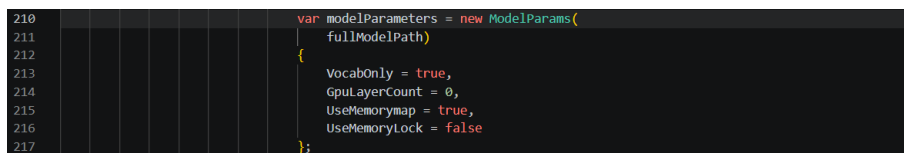
Users can also download one of five Granite 4.0 H Micro GGUF packages from a fixed catalogue using the preference slider. Downloads show progress and can be cancelled or resumed. Before making a download available, the application checks its size and SHA-256 hash against the recorded values. The downloaded model then goes through the same selection checks and quick scan. If these checks pass and the user has not changed the selection, model inspection opens automatically.

6.4 GGUF Validation and Metadata Inspection

GGUF validation has two parts: a quick scan during import and a deeper inspection in a separate worker process. The quick scanner opens the file without changing it, checks the four-byte **GGUF** signature and accepts GGUF versions 2 and 3. It reads details such as the architecture, model name, quantisation and declared context length. The architecture is required. If the model name is missing, the filename is used instead.

The scanner limits the number of metadata entries and the size of strings and arrays to avoid excessive processing. Invalid encoding, incomplete metadata and unsupported versions produce clear error messages. This first scan checks only the header and metadata; it does not read tensor descriptors or load model weights.

The deeper inspection uses LLamaSharp with the CPU runtime in `VocabOnly` mode. Figure 6.1 shows these settings. This mode opens the model's metadata and vocabulary without loading the full weights for inference.

A screenshot of a code editor showing C# code for configuring a model. The code is as follows:

```
210 var modelParameters = new ModelParams(  
211     fullModelPath  
212 )  
213 {  
214     VocabOnly = true,  
215     GpuLayerCount = 0,  
216     UseMemorymap = true,  
217     UseMemoryLock = false  
218 };
```

Figure 6.1: GGUF vocabulary-only inspection settings.

The worker reads available model details, including layer and attention-head counts, vocabulary and special tokens. It also runs a short tokenisation test and checks whether the model contains a chat template. Finding a template does not confirm that it works correctly. To check that the file remained unchanged, the worker compares its path identity, size, modification time and SHA-256 hash before and after inspection.

The model is classified only after inspection finishes successfully. If its chat template is missing, a warning explains that a suitable template must be set up and checked before chat. Cancelled or failed

checks do not count as a pass. Passing inspection does not guarantee that the model fits in memory or can successfully generate text.

6.5 Hardware Analysis

Both model routes use the same hardware inspection service, which collects processor, memory, graphics, storage and runtime information locally and combines it into a hardware report. The checks run asynchronously and support cancellation. Inspection tools are checked against their package manifests before use, while external probes have limits on running time and output size.

Windows APIs provide processor and memory details, including the processor name, architecture, physical cores and logical processors. `GetPhysicallyInstalledSystemMemory` reports installed RAM, while `GlobalMemoryStatusEx` reports RAM usable by Windows and RAM currently available. These values are kept separate because not all installed RAM is available for use. Figure 6.2 shows their collection, conversion to bytes and consistency checks. Storage checks use `GetDiskFreeSpaceExW` to read the capacity and available space of the Windows system volume.

```
48 | cancellationToken.ThrowIfCancellationRequested();
49 | if (!_memoryApi.TryGetPhysicallyInstalledKilobytes(out ulong installedKilobytes) ||
50 |     !_memoryApi.TryGetMemoryStatus(out ulong usableBytes, out ulong availableBytes))
51 | {
52 |     throw new WindowsSystemSnapshotException(MemoryUnavailableCode);
53 | }
54 |
55 | ulong installedBytes;
56 | try
57 | {
58 |     installedBytes = checked(installedKilobytes * 1024UL);
59 | }
60 | catch (OverflowException)
61 | {
62 |     throw new WindowsSystemSnapshotException(MemoryOverflowCode);
63 | }
64 |
65 | if (installedBytes < usableBytes || usableBytes < availableBytes)
66 | {
67 |     throw new WindowsSystemSnapshotException(MemoryInconsistentCode);
68 | }
```

Figure 6.2: Collecting and checking installed, OS-usable and available RAM.

DXGI identifies graphics adapters and records dedicated video memory, dedicated system memory and shared system memory separately. Shared memory is not counted as extra dedicated VRAM or assumed to be free. NPU detection is currently unavailable, which does not mean that the device has no NPU. A separate, verified CPU probe reports the capabilities of its packaged `llama.cpp` runtime. The shared hardware inspector does not list OpenVINO devices, so hardware detection alone does not confirm that OpenVINO can use them.

The application records the source of each value and checks its age, units and consistency. Optional LLM Fit results support these checks but cannot replace missing Windows memory data. If usable LLM Fit and Windows readings differ beyond the allowed limits, a conflict is recorded. Figure 6.3 shows how conflicting readings are prevented from becoming accepted memory facts.

```
122 | bool criticalFailure = totalConflict || availableConflict;
123 | MemoryFacts? memory = criticalFailure
124 |     ? null
125 |     : new MemoryFacts(
126 |         snapshot.PhysicallyInstalledBytes,
127 |         snapshot.OSUsablePhysicalBytes,
128 |         snapshot.AvailablePhysicalBytes,
129 |         snapshot.CapturedAtUtc);
```

Figure 6.3: Preventing conflicting readings from becoming accepted memory facts.

Missing optional details can produce warnings, while missing essential data or unresolved conflicts in required values cause inspection to fail. Compatibility checks read available RAM and free space on the Windows system volume again, checking that the readings are recent and consistent. The report supports memory estimation but does not measure performance or confirm that a model will run.

6.6 Memory Estimation

The application estimates memory for each candidate configuration using the inspected model details, context length, weight format, KV-cache format and runtime route. GGUF and OpenVINO use separate estimators, but both include weights, KV cache, compute buffers, backend memory and application overhead. Temporary staging memory is added where needed.

Each component is assigned to loading, compilation or generation. The largest total at any stage becomes the estimated peak for that memory pool, avoiding the addition of allocations that never exist together. The system-memory and shared-device peaks are then added to estimate RAM use. This can overestimate demand when the two peaks occur at different stages. Dedicated device memory and storage are kept separate.

Weight estimates start from the recorded file size. For GGUF format changes, the size is scaled using the ratio between the source and target weight formats. OpenVINO weight scaling assumes a 16-bit source. The provisional policy adds 3% weight overhead and rounds weight allocations up to multiples of 4,096 bytes. It also allows 512 MiB for the application, 64 MiB for the CPU backend or 256 MiB for a GPU backend. Compute buffers use the greater of 128 MiB and 32 KiB per context token. OpenVINO also multiplies this allowance by its stream count.

KV-cache estimates use the layer count, KV-head count, head dimension, context length and cache format, assuming one sequence. Figure 6.4 shows how the GGUF calculation pads TurboQuant head dimensions to multiples of 128 and rounds allocations up to complete blocks before adding the key and value caches. OpenVINO uses its own cache layouts and includes padding, keeping higher-precision cache estimates at least as large as lower-precision options. No estimate is produced if required dimensions remain unavailable, the layout is unsupported or a calculation exceeds the supported numeric range.

```

81         ulong effectiveHeadDimension = turboQuant
82         ? CeilingDivide((ulong)headDimension, 128) * 128
83         : (ulong)headDimension;
84         ulong valuesPerTensorPerLayer =
85         (ulong)context.Tokens
86         * (ulong)keyValueHeads
87         * effectiveHeadDimension
88         * ParallelSequences;
89
90         ulong blocks = CeilingDivide(valuesPerTensorPerLayer, (ulong)spec.ValuesPerBlock);
91         ulong bytesPerTensorPerLayer = blocks * (ulong)spec.BytesPerBlock;
92
93         // Key and value tensors are sized separately so an asymmetric
94         // format can be introduced without reshaping this calculation.
95         ulong keyBytes = bytesPerTensorPerLayer;
96         ulong valueBytes = bytesPerTensorPerLayer;
97
98         bytes = ByteCount.FromBytes((keyBytes + valueBytes) * (ulong)layers);

```

Figure 6.4: GGUF KV-cache estimation with head padding and block rounding.

The current safety policy reserves some available RAM and adds a separate margin to the predicted RAM use. For available RAM A and predicted RAM use P , both measured in bytes:

$$\begin{aligned}
 R &= \min(A, \max(\lceil 0.10A \rceil, 512 \text{ MiB})), \\
 B &= A - R, \\
 M_{\text{required}} &= P + \max(\lceil 0.10P \rceil, 512 \text{ MiB}).
 \end{aligned}$$

Here, R is the reserve and B is the remaining memory budget for the model. The memory-fit check compares M_{required} with B . No further fixed Windows allowance is removed because the available RAM reading already accounts for memory used by Windows and running programs.

The estimate records its limitations, including file-size-based weight estimates and the assumption of one sequence. Conversion estimates also state when source-model storage is not included. Passing the memory check does not prove that execution will succeed; runtime support, optimisation evidence and storage checks remain separate requirements.

6.6.1 Experimental Validation Status

The estimator was implemented, but its error and false-safe or false-unsafe recommendations were not measured because the planned R-M14 calibration study was not completed. Its constants therefore remain provisional, and its outputs remain predictions rather than validated memory guarantees.

6.7 Configuration and Optimisation Selection

The application creates possible configurations from the inspected model, supported runtime features and recorded optimisation evidence. Each configuration has its own identity and settings for the runtime route, weights, KV cache and context length. It must pass checks for runtime support, minimum context, policy-based quality, memory and storage before it can be offered to the user. A format being included in the code does not mean that every model can use it.

The configuration slider selects from these checked configurations rather than allowing any combination of settings. Table 6.1 shows the five intended choices for the released OpenVINO Raw 3B configurations. Balanced is the default. If it is unavailable, the nearest available preference is selected, provided another valid choice exists. Each position must match exactly one released, non-experimental configuration with the stated weight format, cache format and 4,096-token context. The mapping is rejected if several candidates match one position or the same candidate is used more than once.

Table 6.1: OpenVINO Raw 3B preference mappings, subject to current checks.

Position	Preference	Weights	KV cache
10	Maximum efficiency	INT4	TurboQuant TBQ3
30	Efficient	INT4	TurboQuant TBQ4
50	Balanced	INT4	U4
70	High capability	INT4	U8
90	Maximum capability	INT8	Automatic OpenVINO default

The GGUF slider orders its recognised cache choices as TurboQuant 3-bit, TurboQuant 4-bit and Q8_0, where available. These choices change the KV-cache setting, not necessarily the model’s weight format. Preference and expected-quality labels follow application rules; they do not guarantee measured response quality or speed. Users can choose another allowed configuration, but cannot bypass memory, storage or runtime-evidence checks through the slider.

When Start optimisation is selected, the application checks the configuration again using current memory and storage readings. The same candidate must still be available before an execution plan is created; it is not silently replaced with another configuration. Figure 6.5 shows the checks that confirm the candidate’s approval still matches the model journey, workload, runtime capabilities and current hardware evidence. The code also checks that the hardware evidence is recent and that the plan agrees on whether a persistent conversion is needed. Separate checks confirm that the execution settings match the chosen candidate and approved runtime.

```

174 OptimizationAdmissionProof? proof = candidate.AdmissionProof;
175 Require(
176     proof is not null
177     && proof.MatchesCandidate(candidate)
178     && proof.MatchesAuthority(snapshot, workload, binding)
179     && string.Equals(
180         proof.HardwareAuthoritySha256,
181         currentHardwareAuthority.AuthoritySha256,
182         StringComparison.Ordinal)
183     && currentHardwareAuthority.IsFreshAt(createdAtUtc)
184     && proof.RequiresPersistentChange == payload.RequiresPersistentConversion,
185     "frontier admission authority",
186     "candidate quantities, snapshot, workload, journey, hardware, freshness, or persistence differs");

```

Figure 6.5: Checking candidate admission before issuing an optimisation plan.

The result shows whether the model is estimated to fit, needs optimisation or cannot yet be assessed. Experimental candidates still need their own evidence and consent; approving one configuration does not approve every configuration using the same format. If the selected candidate no longer passes the checks, Start is blocked rather than relying on earlier approval. The execution plan keeps the exact selected settings, but does not guarantee that the model will run successfully.

6.8 Runtime Integration

The application runs inference in worker processes, separate from the WinUI interface. It sends checked settings to the selected runtime route and receives structured progress, text and completion messages. Runtime packages are verified before use, and failures are not treated as successful results.

6.8.1 llama.cpp Integration

The GGUF route uses a worker and a runtime adapter. The adapter chooses either the upstream CPU implementation through LLamaSharp or the packaged AtomicBot runtime. LLamaSharp loads the weights, creates a context and starts a chat session. Unlike the earlier vocabulary-only inspection, this loads the model for text generation.

For the AtomicBot route, the adapter starts the packaged server with settings for the model, context, key and value cache formats, GPU layers, threads, batch size and flash attention. Figure 6.6 shows these arguments. They are passed separately to the process rather than joined into a shell command. Automatic fitting and context shifting are disabled so these features cannot change the selected configuration.

```

17 return
18 [
19     "--host", "127.0.0.1",
20     "--port", Format(port),
21     "--no-ui",
22     "--no-context-shift",
23     "--model", options.ModelPath,
24     "--ctx-size", Format(options.ContextSize),
25     "--cache-type-k", Format(options.KeyCacheType),
26     "--cache-type-v", Format(options.ValueCacheType),
27     "--n-gpu-layers", Format(options.GpuLayerCount),
28     "--threads", Format(options.ThreadCount),
29     "--batch-size", Format(options.BatchSize),
30     "--flash-attn", options.FlashAttention ? "on" : "off",
31     "--parallel", "1",
32     "--cache-ram", "0",
33     "--fit", "off",
34     "--offline",
35 ];

```

Figure 6.6: Settings passed to the packaged GGUF runtime.

The server listens on 127.0.0.1, and the adapter sends local HTTP requests to it. Inference stays on the device, but uses a listening loopback port. Streamed responses are turned into structured text

updates, and cancellation is passed to the active operation. Invalid responses and model-loading failures produce controlled errors rather than successful results.

6.8.2 OpenVINO Integration

The OpenVINO route uses a native C++ worker with OpenVINO Runtime and GenAI. It requires a checked package containing the model IR, tokenizer and configuration files. The worker reads `openvino_model.xml`, compiles it for the requested device and creates an `ov::genai::LLMPipeline`.

The worker checks the execution devices reported by a compiled model. It accepts explicit CPU, GPU or indexed GPU.*n* requests and requires one matching execution device. A mismatch causes an error rather than allowing an unnoticed fallback. These worker options do not mean that every GPU configuration is available in the application.

For explicit U4 or U8 cache settings, the worker reads the compiled model's key and value cache properties and rejects any mismatch. Figure 6.7 shows this check. The default-cache route is kept separate and does not claim a verified explicit cache precision.

```
194         const ov::element::Type requested = kv_cache_precision_ == "u8"  
195         ? ov::element::u8  
196         : ov::element::u4;  
197         const ov::element::Type actual_key =  
198             compiled.get_property(ov::key_cache_precision);  
199         const ov::element::Type actual_value =  
200             compiled.get_property(ov::value_cache_precision);  
201         if (actual_key != requested || actual_value != requested) {  
202             throw std::runtime_error(  
203                 "compiled model KV-cache precision did not match request");  
204         }  
205         actual_kv_cache_precision_ = actual_key.get_type_name();
```

Figure 6.7: Checking the applied OpenVINO key and value cache precision.

Generation uses a streaming callback that checks for stop and cancellation requests. The worker also checks package and runtime integrity before returning results. Only approved package, device and configuration combinations are supported; having an OpenVINO plugin installed does not by itself confirm application support.

6.8.3 TurboQuant Integration or Feasibility Investigation

TurboQuant uses specific runtime integrations rather than a general on/off setting. The GGUF route uses a fixed AtomicBot runtime version, whose staging check requires source commit `519f0c594a8e31467d2e2f2cf17054c9e7e11536`. The OpenVINO route uses a separate Windows x64 TurboQuant worker built from fixed OpenVINO and GenAI source versions with matching runtime libraries. Its runtime evidence records the source and implementation revisions, together with patch and runtime manifest hashes.

The OpenVINO TurboQuant worker checks the requested cache precision and runs a separate synthetic attention test to collect evidence of activation. This test does not directly measure every attention operation in the loaded Granite model. These integrations change how the KV cache is stored, which is separate from weight quantisation.

Experimental status. TurboQuant was tested through fixed runtime versions. Evidence that a cache format was active was kept separate from whether a command-line option was accepted. The experiments showed which tested configurations ran successfully, but standalone runtime results do not prove that the same route worked through the WinUI application. The integration code alone also does not prove successful application execution.

6.8.4 TurboVec Feasibility Harness

A separate command-line harness built, saved, loaded and searched Exact, TQ2, TQ3 and TQ4 indexes using fixed Granite embeddings. It recorded retrieval quality, latency and storage at 1,000 and 10,000 chunks and checked the planned release conditions. This does not mean that all conditions were met. TurboVec was not integrated into the WinUI application or a RAG workflow.

6.9 Inference User Interface

GGUF and OpenVINO use the same chat interface, with route adapters turning runtime events into a common format for updating messages. The page shows the selected model and runtime description alongside the conversation, and users submit prompts with Send or Enter, while Shift+Enter adds a new line. Empty prompts are rejected, and submission is blocked while a response is being generated, a model is switching or a session is being prepared. The chat coordinator also checks that the conversation is ready before starting a response, preventing overlapping user requests.

Each response starts as an empty assistant message marked “Thinking”, with incoming text added to the same message as generation continues. When a screen update is already waiting, further render requests are combined with it and processed on the UI thread, avoiding a separate queued render for every event. The interface shows whether a response is still generating, was stopped, is incomplete, failed or reached its limit, so these outcomes are not mistaken for a completed answer.

Figure 6.8 shows how the coordinator keeps the received text while updating its status. Responses that reach the token limit are marked separately, while stopped responses are marked incomplete if the session needs to be reloaded. Users can also continue the latest response that reached its token limit.

```
580 | | | | | GgufChatCompleted
581 | | | | | {
582 | | | | |     Kind: GgufChatCompletionKind.Length
583 | | | | | } => assistant.WithContent(
584 | | | | |     assistant.Content,
585 | | | | |     ChatCompletionStatus.LimitReached),
586 | | | | | GgufChatStopped stopped => assistant.WithContent(
587 | | | | |     assistant.Content,
588 | | | | |     stopped.NeedsReload
589 | | | | |     ? ChatCompletionStatus.Incomplete
590 | | | | |     : ChatCompletionStatus.Stopped),
591 | | | | | GgufChatFailed => assistant.WithContent(
592 | | | | |     assistant.Content,
593 | | | | |     ChatCompletionStatus.Failed),
```

Figure 6.8: Preserving response text while recording its completion state.

During generation, Stop requests an end to the active response and shows a stopping state. If this fails, the page explains the problem while keeping the visible response. Failure messages appear separately from the assistant’s text, while application errors are sent to a separate fault reporter.

Before generation, the coordinator saves the user prompt and pending assistant message to local history, then saves again when the response completes, stops or fails. It also tries to save any remaining unsaved text when generation ends, helping preserve partial responses during normal use without guaranteeing recovery of every update after a sudden application or system failure.

6.10 Reliable Programming and Observability

The application checks inputs and uses defined result types and cancellation tokens to keep successful, cancelled and failed operations separate. Request identifiers and generation checks help prevent old results from changing a newer operation. The inspection worker also uses `Interlocked.CompareExchange` to

allow only one final-message claim, as shown in Figure 6.9. This prevents duplicate final messages but does not decide the operation’s outcome.

```

6 internal sealed class WorkerTerminalCoordinator
7 {
8     1 reference
9     private int _terminalStarted;
10
11     2 references
12     internal bool TryBeginTerminal() =>
13         Interlocked.CompareExchange(
14             ref _terminalStarted,
15             value: 1,
16             comparand: 0) == 0;
17 }

```

Figure 6.9: Preventing duplicate final messages.

The application closes resources when they are no longer needed. The hardware probe runner uses a Windows Job Object that terminates its processes when closed, alongside limits on running time and output size. Cleanup failures are recorded, runtime arguments are passed separately rather than through a shell command, and chat history is written to a temporary file before replacing the saved record.

The shared fault reporter keeps up to 32 recent fault codes and error categories in memory, without storing exception messages, stack traces, prompts or model paths. Worker request identifiers help track operations, but do not provide a complete saved log of interface actions. Error records are kept separate from chat history, where conversation text is saved.

6.11 Implementation Status

Table 6.2 summarises the implemented features and their main limits, where “Implemented” means that the functionality is present in the code rather than that every model or device combination has been shown to work. Application testing is covered in Chapter 7.

Table 6.2: Implementation status and main limitations.

Feature	Status	Scope and limitation
GGUF import and inspection	Implemented	Header and metadata checks are followed by vocabulary-only inspection, although passing does not prove that the model can generate text.
OpenVINO import and inspection	Implemented	Model package and runtime checks cover accepted package layouts and supported runtime combinations.
Model downloading	Implemented	Downloads are limited to five fixed GGUF packages, each checked by file size and SHA-256 hash.
Hardware analysis	Implemented	Local checks collect processor, memory, graphics, storage and runtime information, while NPU detection remains unavailable.
Memory estimation	Implemented	Estimates are provided for supported configurations, but the R-M14 study to measure estimator accuracy was not completed.
Configuration selection	Implemented	Approved choices and plans for the exact selected configuration remain subject to evidence, memory and storage checks.
Local inference and chat	Implemented	GGUF and OpenVINO runtime connections support streamed responses and local chat history, with support depending on the configuration.
TurboQuant routes	Implemented	Specific runtime integrations use evidence checks rather than providing support for every TurboQuant configuration.
TurboVec	Test harness only	A separate retrieval test program was implemented, but was not added to the application or a RAG workflow.

The earlier sections explain how these features were implemented, while Chapter 8 and Appendix F present the experimental results. A runtime test that passes outside the application does not prove that the same configuration worked within it.

Chapter 7

Software Verification and User-Experience Evaluation

7.1 Verification Approach

Testing covered invalid models, configuration choices, cancellation, recovery and incomplete outputs, following the requirements in Chapter 3. Unit and integration tests checked individual rules and how components worked together, using controlled inputs to reproduce errors and timing problems. Automated end-to-end (E2E) tests followed inspection journeys in the installed application, while manual checks covered local and offline use.

Appendix G lists the test groups, results and limits. Supporting files are under [docs/testing/application-verification](#). Passing these tests does not mean that every requirement or part of the code was fully checked.

7.2 Automated Testing and Results

Automated checks covered model validation, downloads, inspection results, memory limits, configuration choices, optimisation outputs, export and chat sessions. They used valid and invalid inputs, boundary values and changes in application state. Integration tests checked how components passed requests and results between stages.

One important check confirmed that accepted cancellation blocked a later success result, even when the service finished successfully. Other checks rejected results from earlier attempts. These help stop cancelled or outdated work from moving the workflow forward.

The GitHub Actions run, E1 in the appendix, passed both builds and all 1,877 selected cases, with no failures, skips or aborted cases. Three separate local GGUF checks, E2, also passed. They checked that Q8 and both TurboQuant choices were available, that a missing or invalid quantizer blocked saved-model conversion, and that runtime-only choices remained available without it.

E2 checked which choices were available, not whether TurboQuant ran during inference. The totals are not added together because some checks may overlap. Excluded and deferred checks are not counted as passes.

7.3 Application Journeys, Packaging and Recovery

Local checks found problems that the passing CI run did not rule out. OpenVINO inspection could not start its verified runtime. The GGUF route passed model inspection but could not verify the hardware-inspection tool. Later checks of the restored package passed these stages.

Three automated E2E inspection journeys passed without failures or skips: GGUF reached compatibility, OpenVINO reached configuration with an enabled slider and Start action, and an invalid GGUF blocked Continue before a valid replacement completed inspection and compatibility checks.

These results are recorded as E6 in the appendix. The tests used Windows UI Automation and real local models. The application was not changed or rebuilt. The selected files were checked against recorded hashes, and the running app matched the recorded executable path.

These tests covered inspection and recovery, not optimisation, export or chat. They also did not check every slider choice or installation on a clean computer.

Other recovery gaps remain. A source-conversion test was removed after an invalid worker manifest caused an unhandled error and stopped the test host. Removing the test did not fix that issue. The revised export-cleanup test also does not cover the earlier overlapping-timeout case.

7.4 Offline Use and Safety Checks

The developer confirmed completing inspection, optimisation, export and chat with Wi-Fi disconnected and the required files stored locally. This supports offline use for that manual journey. Its exact model, route and build were not recorded, and network traffic was not captured. It does not therefore confirm both routes, rule out attempted network requests or fully verify N-M12's no-server requirement.

Selected download and output checks passed. They checked when files could be accepted or made available, helping prevent incomplete or unsupported outputs from being shown as ready. These were specific safety checks, not a full security assessment. No penetration test or complete network audit was confirmed.

7.5 Interface and Accessibility Checks

Selected automated checks of displayed states, controls, layouts and accessibility features passed. The developer carried out the evaluation; no formal user study was recorded.

Three chat auto-scroll checks remain deferred. Four inspection checks also remain deferred: progress announcements, late announcements, focus after failure and title wrapping in a retained result control. Their assertions remain in the code and are not counted as passes.

These gaps may make progress and recovery harder to follow. Full keyboard and 200% text-scaling checks were not confirmed, so N-M13 is not fully verified. Passing selected interface checks does not prove that the whole application is easy to use.

7.6 Overall Findings and Remaining Limitations

The checks support some application functions, but only partly support N-M08 and N-M14, while N-M13 is not fully verified. Further work includes automated testing from optimisation to chat, installation on a clean computer, testing on more hardware and checking memory estimates against measured use, as well as the recovery and accessibility issues above. Configuration checks do not confirm runtime activation or fallback under F-M21–F-M22; Chapter 8 presents the separate runtime experiments.

Chapter 8

Experimental Results and Discussion

This chapter reports which tested configurations worked and the trade-offs in memory, speed and output quality. It also reports negative results and the separate TurboVec decision. The findings apply only to the recorded models, runtimes, settings and test computer. Appendix F provides the supporting audit tables and evidence locations.

8.1 Experimental Completeness and Evidence Coverage

The unified inference release contains 169 terminal attempt records across 168 distinct route/test-case identifiers. AH-09 appears twice because both its accepted record and its rejected environment-only attempt are retained. Table 8.1 is a terminal ledger, not a count of benchmark repetitions or quality prompts.

Table 8.1: Terminal status of the five inference-route campaigns

Route	Passed	Failed	Blocked	Unavailable	Total
Upstream llama.cpp	13	0	0	0	13
AtomicBot TurboQuant	19	0	0	0	19
animehacker TQ3_0	7	1	3	0	11
Experimental OpenVINO fork	27	0	0	54	81
Official OpenVINO upstream	15	5	25	0	45
Total	81	6	28	54	169

The statuses are not interchangeable. The animehacker failure was a rejected historical attempt, while three other cases were stopped by safety checks. The 54 experimental OpenVINO cases had no validated model artefact. Five official OpenVINO conversions reached the emergency RAM floor, and 25 further cases were blocked before execution. These outcomes are not treated as runtime crashes. TurboVec is excluded from this ledger because it tested retrieval rather than inference. Appendix F.2 provides the supporting campaign summary.

8.2 Runtime Feasibility by Route

The upstream campaign completed all 13 intended workloads on CPU, Vulkan and SYCL. It remains a conditional rather than unconditional pass because the broader SYCL repository suite retained three edge-case failures, although the UL-13 project inference workload passed. Within this campaign only,

UL-08 is the approved best-observed CPU label, UL-10 the approved best-observed Intel GPU label, and UL-05 the recorded fallback label; none is a universal ranking.

All 19 AtomicBot runtime configurations completed, spanning F16, Q8_0, Turbo2, Turbo3 and Turbo4 cache variants on CPU and Vulkan. A generated UI asset problem and an early memory guard were resolved. One repository executable remained blocked by Windows Device Guard, which is an external setup-policy limitation rather than a failed model run. Activation evidence supports the tested TurboQuant paths, but the result does not prove equivalence to similarly named implementations in other repositories.

The animehacker campaign completed seven runnable configurations. CPU TQ3_0 activation was established, while the recovered SYCL result used bounded partial placement: only one layer was placed through the selected path and the KV cache was not claimed to be GPU resident. Three 8B configurations were safety-classified without lowering the reserve. The route is therefore useful as a controlled comparator, not as evidence of production-ready full-GPU TQ3_0 execution.

The experimental OpenVINO fork completed 27 configurations from the three validated artefact groups. The official route completed 15 configurations and demonstrated that the upstream TurboQuant merge could be exercised for the available 3B INT4/INT8 and 8B INT4 models. OpenVINO properties recorded the requested SDPA/cache configuration, but the absence of an independent device trace prevents a stronger claim about physical device use or fallback for every row.

8.3 Inference Performance

Table 8.2 presents route-local ranges. They show the scale of the observations, not a league table: model identity, prompt length, backend, context and aggregation differ between routes.

Table 8.2: Observed performance ranges within each route

Route	Decode tok/s	TTFT (ms)	Peak working set (GiB)
Upstream llama.cpp	4.39–37.19	602.99–7246.13	0.91–10.07
AtomicBot TurboQuant	4.19–37.50	72.31–687.21	0.89–10.21
animehacker TQ3_0	11.54–46.32	137.75–491.00	0.89–3.59
Experimental OpenVINO	7.36–17.79	953.45–15094.58	3.38–7.95
Official OpenVINO	7.10–17.75	961.84–14040.07	3.39–7.95

The only cross-repository performance comparison classified as direct by the final comparability matrix is the 15-row OpenVINO overlap. The identical case IDs match model scale, weight format, cache format and metric protocol, although runtime/repository differences still prevent attributing the entire delta to a single code change.

Official decode throughput was lower in all 15 matched rows, but the relative difference ranged from approximately 0.23% to 11.34%; it was not a consistent large regression. The largest difference occurred for 3B INT8 TBQ3. TTFT was mixed rather than uniformly better or worse. Because OpenVINO TTFT belongs to the repetition selected by median decode throughput, it can move sharply when a different repetition becomes the selected row and should not be read as a median-TTFT estimate.

The workload was also short: approximately 23–24 input tokens and 32 generated tokens. It is suitable for repeated short-request throughput measurement but does not substantially fill the KV cache. The near-identical working sets—approximately 3.63 GB for 3B INT4, 6.64 GB for 3B INT8 and 8.54 GB for 8B INT4 when expressed in decimal source units—are therefore unsurprising. They are dominated by weights and runtime allocation and do not disprove cache-memory savings at long context.

Table 8.3: Matched experimental and official OpenVINO results

Model/weight	Cache	Decode tok/s		Quality /10	
		Exp.	Official	Exp.	Official
3B INT4	F16	16.97	16.45	0.00	5.77
3B INT4	U8	17.13	16.44	0.00	5.77
3B INT4	U4	17.12	16.50	5.67	5.52
3B INT4	TBQ3	17.79	17.75	5.21	5.42
3B INT4	TBQ4	16.63	16.40	5.00	5.85
3B INT8	F16	10.54	10.06	0.00	5.77
3B INT8	U8	10.46	10.11	0.00	5.77
3B INT8	U4	10.34	10.21	6.08	5.79
3B INT8	TBQ3	9.80	8.69	5.75	5.48
3B INT8	TBQ4	10.38	10.18	5.94	5.52
8B INT4	F16	7.69	7.61	0.00	5.60
8B INT4	U8	7.66	7.58	0.00	5.60
8B INT4	U4	7.89	7.56	5.08	5.38
8B INT4	TBQ3	7.37	7.10	5.38	5.21
8B INT4	TBQ4	7.77	7.59	5.98	5.60

8.4 Memory and TurboQuant Trade-Offs

The `llama.cpp` routes exposed explicit KV allocation, making their within-route comparisons more informative about cache storage than OpenVINO peak working set. Table 8.4 gives selected AtomicBot matches. The result is consistent: Turbo3 normally used less recorded KV memory, but throughput and quality did not move monotonically with nominal precision.

Table 8.4: Selected within-route AtomicBot cache trade-offs

Model/backend	ID	Cache	KV MiB	Decode tok/s	Quality /10
Gemma 1B CPU	AB-01	F16	26.0	37.50	6.22
	AB-02	Turbo3	5.1	30.80	5.39
Granite 3B CPU	AB-03	F16	160.0	15.90	6.39
	AB-04	Q8_0	170.0	15.90	6.72
	AB-05	Turbo4	170.0	11.80	6.84
	AB-06	Turbo3	125.0	11.80	5.95
	AB-07	Turbo2	89.2	13.10	0.70
Granite 8B CPU	AB-08F	F16	320.0	6.80	5.69
	AB-08Q	Q8_0	340.0	6.80	6.69
	AB-09	Turbo4	170.0	5.90	5.97
	AB-10	Turbo3	125.0	5.90	6.67
Granite 3B Vulkan	AB-11	F16	320.0	14.00	6.72
	AB-12	Turbo3	125.0	9.60	6.22
Granite 8B Vulkan	AB-14	Q8_0	340.0	6.00	8.36
	AB-15	Turbo3	125.0	5.40	7.51

For the Gemma pair, Turbo3 reduced the recorded KV allocation from 26.0 to 5.1 MiB (80.4%) while decode throughput fell by 17.9% and the limited legacy quality score fell by 0.83 points. For the 8B CPU cache group, Turbo3 used 125 MiB compared with 340 MiB for Q8_0 (63.2% less), but both decoded at 5.9–6.8 tokens/s and quality was response dependent. The Vulkan pairs similarly showed lower KV allocation and modestly lower peak working set with Turbo3, but also lower decode throughput. These observations answer whether compression was active and useful in specific cases; they do not establish that TurboQuant is always faster or better than U4, U8 or F16.

The upstream cache comparisons reinforce the distinction between cache and whole-process memory. For Granite 3B Q4_K_M CPU, moving from F16–F16 (UL-04) to Q8_0–Q8_0 (UL-05) reduced recorded

KV allocation from 320 to 170 MiB and peak working set from approximately 3.74 to 3.59 GiB, while decode throughput changed only from 15.98 to 15.77 tokens/s. For Granite 8B, UL-07 to UL-08 reduced KV allocation from 640 to 340 MiB and peak working set from 9.03 to 8.74 GiB, while decode throughput remained approximately 7.09 tokens/s. Cache compression was therefore measurable even when its effect was muted in total working set.

The tested host supplied useful 16-GB feasibility observations, including safe execution of selected 3B and 8B cases and controlled rejection of higher-risk configurations. It does not supply calibrated predictions for hypothetical 4-GB or 8-GB machines. The planned estimator and controlled-budget experiments were not completed, so no false-safe rate, false-unsafe rate or validated 4/8/16-GB classification accuracy is claimed.

8.5 Generative Output Quality

The quality catalogue contains 81 configuration summaries. Table 8.5 reports route-level descriptive distributions, but the means must not be used as a cross-route ranking because the prompt sets, rubrics, denominators and generation settings differ.

Table 8.5: Descriptive quality-score distributions by route

Route	Configurations	Minimum	Maximum	Mean
Upstream llama.cpp	13	4.00	8.88	6.88
AtomicBot TurboQuant	19	0.70	8.36	6.21
animehacker TQ3_0	7	3.27	6.72	5.52
Experimental OpenVINO	27	0.00	6.08	3.82
Official OpenVINO	15	5.21	5.85	5.60

The low experimental OpenVINO mean is not evidence that its Granite models were universally worse. Six executed configurations—3B INT4 F16, 3B INT4 U8, 3B INT8 F16, 3B INT8 U8, 8B INT4 F16 and 8B INT4 U8—produced degenerate repeated-character or punctuation output and scored zero after the output-health gate. They passed the execution campaign but failed the usefulness/quality stage. Polar3 on 3B INT4 also scored only 0.90. By contrast, viable experimental rows ranged up to 6.08, and every official matched row lay between 5.21 and 5.85.

The matched table also explains why a simple “more bits means better quality” interpretation is invalid. Experimental 3B INT4 TBQ3 scored 5.21 while TBQ4 scored 5.00; for 3B INT8, TBQ4 exceeded TBQ3; and for 8B INT4, TBQ4 again exceeded TBQ3. In the official route TBQ4 exceeded TBQ3 for 3B INT4 and 8B INT4, but not by a constant amount. One generated response can cross a lexical or health threshold while another narrowly misses it, and bit width is only one factor alongside model weights, cache implementation, runtime, prompt and generation behaviour. The available design does not estimate the variance of quality over repeated generations.

The official route’s large positive quality differences over the six corrupted experimental rows should therefore be interpreted as removal of an implementation/output-health failure, not as evidence that the corresponding F16 or U8 cache inherently adds approximately 5.6 quality points. Likewise, the slightly lower mean for the official 8B rows than the official 3B rows does not prove that the larger model is worse: only one weight group was available for 8B, the outputs were capped at 128 tokens, and the rubric measures compliance with this fixed suite rather than general intelligence.

Legacy quality results provide route-local evidence only. The upstream route’s higher descriptive mean used four or six manually adjudicated prompts and a different 100-point weighted rubric. AtomicBot used six prompts and remains labelled limited/provisional; animehacker used a historical harsh content screen.

No OpenVINO-versus-llama.cpp quality winner is consequently reported.

8.6 Context, Stability and Failure Analysis

The tests covered selected 4K cases and one earlier 16K quality case, but did not systematically test different context lengths. AtomicBot’s 4K F16 baseline cases used 320 MiB of cache for Granite 3B and 640 MiB for Granite 8B. An animehacker 16K Q8_0 quality test reached its 900-second time limit, then completed when only that prompt was rerun with a 1,800-second limit. This shows that the case could finish, but does not guarantee response times for other cases.

OpenVINO’s performance prompt was too short to measure cache savings at longer contexts. Its quality tests used short, medium and long prompts, but answers were limited to 128 tokens and did not show how memory use changed with context length. The results therefore do not establish the largest supported context, how cache memory grows or whether long conversations remain stable. Table F.3 in Appendix F summarises the main failures, blocked cases and recovery results.

These cases show why forcing every intended row to execute would have reduced, not increased, the validity of the campaign. Safety blocks establish a measured host boundary; unavailable artefacts establish an evidence gap; failed conversion and corrupted output establish different defects requiring different responses.

8.7 Application Evaluation Status

The planned application checks, labelled APP-EVAL-01–APP-EVAL-12, were not completed. They were intended to compare predictions with actual results and assess task completion, recovery, accessibility and ease of use. Runtime experiments and implementation tests do not replace this evaluation, so they do not answer RQ4 or confirm the accuracy of the recommendations. Chapter 7 describes the application checks that were completed and their limits.

8.8 TurboVec Feasibility

The supporting experiment compared Exact, TQ2, TQ3 and TQ4 at 1,000 and 10,000 chunks. Each scale used 128 fixed queries and five repetitions. All four formats could be created, saved, loaded and searched. Table 8.6 gives the median 10,000-chunk results.

Table 8.6: TurboVec results at 10,000 chunks

Index	Recall@10	Recall vs Exact	Rel. nDCG@10	p95 (ms)	Storage ratio
Exact	0.2024	1.0000	1.0000	110.0517	1.0000
TQ2	0.1012	0.2708	0.4125	3.0801	4.7686
TQ3	0.1607	0.4729	0.7561	4.0255	3.6352
TQ4	0.1310	0.6396	0.7382	4.9119	3.6351

The compressed indexes were much faster and smaller than Exact, but all lost retrieval quality. TQ2 was fastest and smallest, TQ3 had the best Recall@10 and relative nDCG@10 among the compressed formats, and TQ4 kept the largest share of Exact’s top ten. No format was best on every measure.

All compressed formats passed the lifecycle, latency and storage rules but failed both Gate A retrieval-quality rules. The Exact baseline was also weak, which suggests that the embeddings, export or test collection affected the absolute scores. Even so, the comparison used the same frozen embeddings and

shows further loss after compression. Gate B also failed, and the experiment did not assess generated Granite answers or citations.

The original formal campaign remains **BLOCKED**. The completed run is supporting evidence because its Python environment and OpenVINO export were different. The 100,000-chunk extension was not required after every compressed format failed Gate A. TurboVec was therefore kept as **DEMONSTRATOR_ONLY** and was not added to the application. Detailed results and evidence locations are in Appendix F.4.

8.9 Relationship to Prior Claims

TurboQuant’s published results do not guarantee the same benefits for Granite on this Intel computer [11]. Memory use, speed and quality depend on more than bit width: model weights, cache metadata, runtime code, how work is shared between devices, context length and software maturity also matter. This highlights the need for independent testing [50].

OpenVINO’s documentation describes what it supports, but does not prove which device carried out every operation in these tests [37]. Different results do not necessarily mean that published findings are wrong, since the models, hardware, software and test methods may differ.

8.10 Threats to Validity

Internal validity. Background activity, temperature and shared CPU/GPU memory could affect timings, while power settings, cooldown and test order were not always recorded. Most inference tests used three repetitions; TurboVec used five, changed the test order and included idle checks. These steps reduce variation but cannot remove it. OpenVINO settings do not independently confirm which device was used, and 77 retained summaries cannot be fully recalculated from the normalised data alone.

Construct validity. Peak process working set does not measure total system memory and may hide small cache savings when model weights use most of the memory. OpenVINO’s short performance prompt did not test longer contexts. Its quality scoring could miss correct answers or reward matching keywords, while responses were limited to 128 tokens and were not independently scored by experts. TurboVec’s Exact baseline also scored poorly, suggesting problems with the embeddings or test data. Using the same embeddings allows comparison of the extra loss from compression, but retrieval scores do not measure the quality of generated answers or citations.

External validity. The results cover selected models and fixed runtime versions on one Intel Windows computer, with missing model files and memory limits restricting what could be tested. Results may differ on other hardware, software or longer contexts. Synthetic education and healthcare material does not prove real-world safety or usefulness. TurboVec was tested at 1,000 and 10,000 chunks using one embedding model and CPU route.

Conclusion validity. The study did not calculate the sample size needed for reliable comparisons, confidence intervals or adjustments for making multiple comparisons. Most quality prompts had only one generated answer, and scores from different scoring systems cannot be directly ranked. The 15 matched OpenVINO cases also used different runtime versions, so their differences cannot be attributed to cache compression alone. TurboVec’s supporting run used a different Python environment and OpenVINO export from the formal campaign. The findings therefore describe the tested configurations, not formats that are always better or confirmed causes of the differences.

Chapter 9

Critical Evaluation and Conclusion

9.1 Summary of Achievements

The project produced a guided WinUI 3 application for selecting, inspecting, configuring and running selected IBM Granite models locally. It supports GGUF and OpenVINO, hardware checks, memory estimates, optimisation, export and streamed chat. Chapters 6–8 describe the application, its tests and the experimental results.

The achievements and remaining limits for each objective are summarised below:

- CO1 — Guided application workflow.** The guided workflow was built without requiring users to enter runtime commands. Its ease of use for non-specialists was not formally tested.
- CO2 — Runtime integration and execution control.** Local runtimes, streamed output and execution controls were added. Some recovery problems remain, and device and fallback behaviour were not fully checked.
- CO3 — TurboQuant and TurboVec investigation.** Selected TurboQuant routes were investigated, keeping weight and KV-cache quantisation separate. TurboVec was tested but not added to the application. Matched experiments supported the cache comparison, although the planned standalone study was not completed.
- CO4 — Experimental evaluation.** Selected configurations were measured under controlled conditions. A full context-length study and checks of memory estimates on lower-memory systems were not completed.
- CO5 — Testing and project documentation.** Tests, CI results, an installed package and supporting documentation were produced. Installation on a clean computer and full release checks were not confirmed.

These assessments separate what was built from what was fully tested. Passing selected tests does not prove that every requirement or error case was covered.

9.2 Final Answers to the Research Questions

These answers draw on the experimental results in Chapter 8 and the application tests in Chapter 7.

RQ1: Hardware and runtime compatibility. Selected configurations ran through CPU, Vulkan, bounded SYCL and OpenVINO routes on the tested Intel Windows computer. Success depended on the model, runtime version and available files. Some routes used only partial GPU placement, and device tracing was incomplete. The results therefore do not fully confirm device use or fallback.

RQ2: TurboQuant trade-offs. TurboQuant reduced recorded KV-cache allocation in selected matched cases, but did not consistently improve speed or quality. In the AtomicBot Granite 3B Vulkan F16/Turbo3 pair, allocation fell from 320 to 125 MiB, while decode speed and the recorded quality score also fell. This was a cache saving, not the same reduction in total application memory.

RQ3: Lower-memory feasibility. Selected 3B and 8B configurations completed on the approximately 16-GB test computer. Physical 4-GB and 8-GB systems were not tested, and memory estimates were not checked against measurements on those systems. The question is therefore only partly answered.

RQ4: Application usefulness and reliability. The three passing automated inspection journeys and the manual offline check support selected application functions. They do not prove ease of use for non-specialists, full accessibility or accurate recommendations. The planned user evaluation was not completed, so RQ4 remains unanswered under that evaluation.

RQ-TV: TurboVec feasibility. The tested compressed indexes were faster and smaller than Exact, but none met the retrieval-quality rules. Exact quality was also low, although compression reduced it further in the tested comparisons. The results in Appendix F.4 support keeping TurboVec as a demonstration rather than adding it to the application.

9.3 Evaluation of the Software Product

The application’s main value is bringing local-inference tasks into one guided workflow. Users can inspect models and review available configurations without writing runtime commands. The manual offline check also supports use without Wi-Fi once the required files are stored locally.

However, a working workflow is not a fully tested product. Memory estimates are not guarantees, and available choices depend on current resources. Without a formal user study, the benefit for non-specialists remains unproven. The application is a working project outcome with tested paths, rather than a release checked across many users and devices.

9.4 Evaluation of Architecture and Technology Choices

The architecture in Chapter 5 separates the interface, inspection, compatibility checks and runtime execution. WinUI 3 provides a native Windows interface. Runtime adapters let GGUF and OpenVINO share the guided workflow while keeping their different input and execution rules. This supports focused testing, although long-term maintenance benefits were not measured.

Worker processes run native runtime operations separately from the interface process. This makes process management, communication and packaging more complex. The failures in Chapter 7 showed that passing builds and component tests did not guarantee a working installation.

Strict verification rejected missing or mismatched files. The lesson is to improve package checks and error messages, not weaken verification. If restarting, I would test package consistency and the installed app earlier, before adding more runtime choices and interface states.

9.5 Evaluation of the Research Contribution

The project studied selected Granite configurations across three `llama.cpp` routes and two OpenVINO routes on one Intel Windows computer. The supporting records separate weight precision from KV-cache precision and allow suitably matched cases to be compared. Failures, blocked cases and unavailable configurations are recorded alongside successful results.

These records show where the tested methods helped and where they were limited. The TurboVec decision also shows that speed and storage savings are not enough when quality is too low. The project did not create a new quantisation algorithm or find one configuration that is always best.

9.6 Limitations

Testing used one Intel computer with about 16 GB of RAM and selected models. The results may differ on other devices. Memory estimates were not compared with actual use across lower-memory systems.

Three automated inspection journeys passed. Optimisation, export and chat were checked manually, not through automated tests of the whole workflow. Some recovery and accessibility issues remain, and installation on a clean computer was not verified.

The experiments used small samples, fixed runtime versions and limited quality measures. Long-context testing was incomplete. TurboVec used synthetic data, and its Exact baseline also had low retrieval quality.

No formal user study or full security assessment was completed. Chapter 7 and Section 8.10 give further details.

9.7 Prioritised Future Work

1. **Improve release reliability.** Strengthen package checks, test installation on a clean computer, fix known recovery problems and extend automated journeys through optimisation, export and chat.
2. **Check recommendations and ease of use.** Compare memory estimates with measured use on physical 4-, 8- and 16-GB systems. Complete the user evaluation and address keyboard, text-scaling, announcement and scrolling problems.
3. **Extend the experiments.** Test longer contexts and more Intel hardware using matched configurations and human-reviewed outputs. Improve and test the Exact retrieval baseline before reconsidering TurboVec with real, non-sensitive documents.

9.8 Final Conclusion

The project met the build part of its aim and evaluated selected Granite configurations on the tested system. The main finding is that lower KV-cache allocation does not automatically mean faster execution or better output. Useful local inference depends on the complete configuration, available resources and verified runtime package, not the quantisation label alone. The application and recorded evidence provide a basis for further work, but testing on more hardware and with intended users is still needed.

Bibliography

- [1] M. Armbrust, A. Fox, R. Griffith, A. D. Joseph, R. H. Katz, A. Konwinski, G. Lee, D. A. Patterson, A. Rabkin, I. Stoica, and M. Zaharia, “A view of cloud computing,” *Communications of the ACM*, vol. 53, no. 4, pp. 50–58, 2010.
- [2] L. Yuan, D.-J. Han, S. Wang, and C. G. Brinton, “Local-cloud inference offloading for LLMs in multi-modal, multi-task, multi-dialogue settings,” in *Proceedings of the 2025 International Symposium on Theory, Algorithmic Foundations, and Protocol Design for Mobile Networks and Mobile Computing*. Association for Computing Machinery, 2025, pp. 201–210.
- [3] C. Huyen, *AI Engineering: Building Applications with Foundation Models*. O’Reilly Media, 2025.
- [4] K. M. L. Jones, “Learning analytics and higher education: A proposed model for establishing informed consent mechanisms to promote student privacy and autonomy,” *International Journal of Educational Technology in Higher Education*, vol. 16, no. 24, 2019.
- [5] S. H. Houser, C. A. Flite, and S. L. Foster, “Privacy and security risk factors related to telehealth services: A systematic review,” *Perspectives in Health Information Management*, vol. 20, no. 1, p. 1f, 2023. [Online]. Available: <https://pmc.ncbi.nlm.nih.gov/articles/PMC9860467/>
- [6] P. Ndibalema, “Constraints of transition to online distance learning in higher education institutions during COVID-19 in developing countries: A systematic review,” *E-Learning and Digital Media*, vol. 19, no. 6, pp. 595–618, 2022.
- [7] X. Wang, Z. Tang, J. Guo, T. Meng, C. Wang, T. Wang, and W. Jia, “Empowering edge intelligence: A comprehensive survey on on-device AI models,” *ACM Computing Surveys*, vol. 57, no. 9, pp. 1–39, 2025.
- [8] Y. Sheng, L. Zheng, B. Yuan, Z. Li, M. Ryabinin, B. Chen, P. Liang, C. Ré, I. Stoica, and C. Zhang, “FlexGen: High-throughput generative inference of large language models with a single GPU,” in *Proceedings of the 40th International Conference on Machine Learning*, ser. Proceedings of Machine Learning Research, vol. 202. PMLR, 2023, pp. 31 094–31 116. [Online]. Available: <https://proceedings.mlr.press/v202/sheng23a.html>
- [9] Microsoft, “Globalmemorystatusex function,” <https://learn.microsoft.com/en-us/windows/win32/api/sysinfoapi/nf-sysinfoapi-globalmemorystatusex>, 2024, accessed 2026-08-13.
- [10] Z. Liu, J. Yuan, H. Jin, S. Zhong, Z. Xu, V. Braverman, B. Chen, and X. Hu, “KIVI: A tuning-free asymmetric 2bit quantization for KV cache,” in *Proceedings of the 41st International Conference on Machine Learning*, ser. Proceedings of Machine Learning Research, vol. 235. PMLR, 2024, pp. 32 332–32 344. [Online]. Available: <https://proceedings.mlr.press/v235/liu24bz.html>

- [11] A. Zandieh, M. Daliri, M. Hadian, and V. Mirrokni, “TurboQuant: Online vector quantization with near-optimal distortion rate,” 2025, arXiv:2504.19874. [Online]. Available: <https://arxiv.org/abs/2504.19874>
- [12] A. Kossiakoff, S. J. Seymour, D. A. Flanigan, and S. M. Biemer, *Systems Engineering Principles and Practice*, 3rd ed. John Wiley & Sons, 2020.
- [13] I. Sommerville, *Engineering Software Products: An Introduction to Modern Software Engineering*. Pearson, 2021.
- [14] M. Richards and N. Ford, *Fundamentals of Software Architecture: A Modern Engineering Approach*, 2nd ed. O’Reilly Media, 2025.
- [15] J. Ousterhout, *A Philosophy of Software Design*, 2nd ed. Yaknyam Press, 2021.
- [16] S. McConnell, *Code Complete: A Practical Handbook of Software Construction*, 2nd ed. Microsoft Press, 2004.
- [17] M. Fowler, *Refactoring: Improving the Design of Existing Code*, 2nd ed. Addison-Wesley, 2019.
- [18] R. Osherove and V. Khorikov, *The Art of Unit Testing: With Examples in JavaScript*, 3rd ed. Manning Publications, 2024.
- [19] A. Zeller, *Why Programs Fail: A Guide to Systematic Debugging*, 2nd ed. Morgan Kaufmann, 2009.
- [20] L. Kohnfelder, *Designing Secure Software: A Guide for Developers*. No Starch Press, 2022.
- [21] R. Hartson and P. Pyla, *The UX Book: Agile UX Design for a Quality User Experience*, 2nd ed. Morgan Kaufmann, 2019.
- [22] M. Kleppmann and C. Riccomini, *Designing Data-Intensive Applications: The Big Ideas Behind Reliable, Scalable, and Maintainable Systems*, 2nd ed. O’Reilly Media, 2026.
- [23] A. Vaswani, N. Shazeer, N. Parmar, J. Uszkoreit, L. Jones, A. N. Gomez, L. Kaiser, and I. Polosukhin, “Attention is all you need,” in *Advances in Neural Information Processing Systems*, vol. 30, 2017, pp. 5998–6008. [Online]. Available: <https://proceedings.neurips.cc/paper/2017/hash/3f5ee243547dee91fbd053c1c4a845aa-Abstract.html>
- [24] Hugging Face, “Caching,” https://huggingface.co/docs/transformers/main/cache_explanation, 2026, accessed 2026-08-13.
- [25] Intel, “Frequently asked questions for intel graphics memory on windows 10 and windows 11,” <https://www.intel.com/content/www/us/en/support/articles/000020962/graphics.html>, 2026, accessed 2026-08-13.
- [26] Microsoft, “Process_memory_counters structure,” https://learn.microsoft.com/en-us/windows/win32/api/psapi/ns-psapi-process_memory_counters, 2024, accessed 2026-08-13.
- [27] Intel, “Post-training quantization,” <https://docs.openvino.ai/2026/openvino-workflow/model-optimization-guide/quantizing-models-post-training.html>, 2026, openVINO documentation, accessed 13 August 2026.
- [28] ggml-org, “llama.cpp quantize tool documentation,” <https://github.com/ggml-org/llama.cpp/blob/master/tools/quantize/README.md>, 2026, accessed 13 August 2026.

- [29] IBM, “Granite 4.1,” IBM Granite Documentation, 2026, accessed 14 August 2026. [Online]. Available: <https://www.ibm.com/granite/docs/models/granite4-1>
- [30] IBM Granite Team, “Granite-4.1-3b model card,” Hugging Face, 2026, accessed 14 August 2026. [Online]. Available: <https://huggingface.co/ibm-granite/granite-4.1-3b>
- [31] —, “Granite-4.1-8b model card,” Hugging Face, 2026, accessed 14 August 2026. [Online]. Available: <https://huggingface.co/ibm-granite/granite-4.1-8b>
- [32] —, “Granite-4.1-30b model card,” Hugging Face, 2026, accessed 14 August 2026. [Online]. Available: <https://huggingface.co/ibm-granite/granite-4.1-30b>
- [33] Intel OpenVINO Team, “openvino_genai.tokenizer,” OpenVINO GenAI API Documentation, 2026, accessed 14 August 2026. [Online]. Available: https://docs.openvino.ai/2026/api/genai_api/_autosummary/openvino_genai.Tokenizer.html
- [34] IBM Granite Team, “Granite-4.1-3b-gguf model repository,” Hugging Face, 2026, accessed 14 August 2026. [Online]. Available: <https://huggingface.co/ibm-granite/granite-4.1-3b-GGUF>
- [35] —, “Granite-4.1-8b-gguf model repository,” Hugging Face, 2026, accessed 14 August 2026. [Online]. Available: <https://huggingface.co/ibm-granite/granite-4.1-8b-GGUF>
- [36] Intel OpenVINO Team, “Ai models verified for openvino,” OpenVINO Documentation, 2026, accessed 14 August 2026. [Online]. Available: <https://docs.openvino.ai/2026/documentation/compatibility-and-support/supported-models.html>
- [37] —, “Inference with openvino genai,” OpenVINO Documentation, 2026, accessed 14 August 2026. [Online]. Available: <https://docs.openvino.ai/2026/openvino-workflow-generative/inference-with-genai.html>
- [38] ggml-org, “GGUF file format specification,” GitHub documentation, 2026, specification revision at commit 6af560d55df03ad92116e3c0a697779584477e85; accessed 14 August 2026. [Online]. Available: <https://github.com/ggml-org/ggml/blob/6af560d55df03ad92116e3c0a697779584477e85/docs/gguf.md>
- [39] —, “llama.cpp: LLM inference in C/C++,” GitHub repository, 2026, tag b9870, commit 2d973636e292ee6f75fadcf08d29cb33511f509f; accessed 14 August 2026. [Online]. Available: <https://github.com/ggml-org/llama.cpp/tree/2d973636e292ee6f75fadcf08d29cb33511f509f>
- [40] —, “Build llama.cpp locally,” GitHub documentation, 2026, tag b9870, commit 2d973636e292ee6f75fadcf08d29cb33511f509f; accessed 14 August 2026. [Online]. Available: <https://github.com/ggml-org/llama.cpp/blob/2d973636e292ee6f75fadcf08d29cb33511f509f/docs/build.md>
- [41] —, “llama.cpp command-line interface documentation,” GitHub documentation, 2026, tag b9870, commit 2d973636e292ee6f75fadcf08d29cb33511f509f; accessed 14 August 2026. [Online]. Available: <https://github.com/ggml-org/llama.cpp/blob/2d973636e292ee6f75fadcf08d29cb33511f509f/tools/cli/README.md>
- [42] Intel OpenVINO Team, “OpenVINO IR format,” OpenVINO Documentation, 2026, accessed 14 August 2026. [Online]. Available: <https://docs.openvino.ai/2026/documentation/openvino-ir-format.html>
- [43] Intel, “Supported devices,” <https://docs.openvino.ai/2026/documentation/compatibility-and-support/supported-devices.html>, 2026, openVINO documentation, accessed 2026-08-13.

- [44] Intel OpenVINO Team, “Heterogeneous execution,” OpenVINO Documentation, 2026, accessed 14 August 2026. [Online]. Available: <https://docs.openvino.ai/2026/openvino-workflow/running-inference/inference-devices-and-modes/hetero-execution.html>
- [45] Z. Zhang, Y. Sheng, T. Zhou, T. Chen, L. Zheng, R. Cai, Z. Song, Y. Tian, C. Ré, C. Barrett, Z. Wang, and B. Chen, “H2O: Heavy-hitter oracle for efficient generative inference of large language models,” 2023. [Online]. Available: <https://arxiv.org/abs/2306.14048>
- [46] Y. Li, Y. Huang, B. Yang, B. Venkitesh, A. Locatelli, H. Ye, T. Cai, P. Lewis, and D. Chen, “SnapKV: LLM knows what you are looking for before generation,” 2024. [Online]. Available: <https://arxiv.org/abs/2404.14469>
- [47] C.-C. Chang, W.-C. Lin, C.-Y. Lin, C.-Y. Chen, Y.-F. Hu, P.-S. Wang, N.-C. Huang, L. Ceze, M. S. Abdelfattah, and K.-C. Wu, “Palu: Compressing KV-cache with low-rank projection,” 2024. [Online]. Available: <https://arxiv.org/abs/2407.21118>
- [48] I. Han, P. Kacham, A. Karbasi, V. Mirrokni, and A. Zandieh, “PolarQuant: Quantizing KV caches with polar transformation,” 2025. [Online]. Available: <https://arxiv.org/abs/2502.02617>
- [49] A. Zandieh, M. Daliri, and I. Han, “QJL: 1-bit quantized JL transform for KV cache quantization with zero overhead,” 2024. [Online]. Available: <https://arxiv.org/abs/2406.03482>
- [50] J. Gao, Y. Gou, Y. Xu, J. Shi, Y. Yang, S. Li, R. C.-W. Wong, and C. Long, “Revisiting RaBitQ and TurboQuant: A symmetric comparison of methods, theory, and experiments,” 2026. [Online]. Available: <https://arxiv.org/abs/2604.19528>
- [51] R. Codrai, “turbovec: A TurboQuant-based vector index with python bindings,” GitHub repository, 2026, tag v0.9.0, commit 1e7200cfd8f26c92ce2855652db64bc7f85bc039, MIT licence; accessed 14 August 2026. [Online]. Available: <https://github.com/RyanCodrai/turbovec/tree/1e7200cfd8f26c92ce2855652db64bc7f85bc039>
- [52] LM Studio, “LM Studio application documentation,” Official documentation, 2026, accessed 14 August 2026. [Online]. Available: <https://lmstudio.ai/docs/app>
- [53] —, “Load models and estimate memory with lms load,” Official documentation, 2026, accessed 14 August 2026. [Online]. Available: <https://lmstudio.ai/docs/cli/local-models/load>
- [54] —, “Offline operation,” Official documentation, 2026, accessed 14 August 2026. [Online]. Available: <https://lmstudio.ai/docs/app/offline>
- [55] Hugging Face, “GGUF on the hugging face hub,” Hugging Face Hub Documentation, 2026, accessed 14 August 2026. [Online]. Available: <https://huggingface.co/docs/hub/gguf>
- [56] AlexsJones, “llmfit,” GitHub repository, 2026, version 1.1.9, commit a02e13f1013ed69889ff44426a651bf7c68c292e, MIT licence; accessed 14 August 2026. [Online]. Available: <https://github.com/AlexsJones/llmfit/tree/a02e13f1013ed69889ff44426a651bf7c68c292e>
- [57] Intel OpenVINO Team, “Class ov::Core,” OpenVINO C++ API Documentation, 2026, accessed 14 August 2026. [Online]. Available: https://docs.openvino.ai/2026/api/c_cpp_api/classov_1_1_core.html
- [58] —, “OpenVINO Benchmark Tool,” OpenVINO Documentation, 2026, accessed 14 August 2026. [Online]. Available: <https://docs.openvino.ai/2026/get-started/learn-openvino/openvino-samples/benchmark-tool.html>

- [59] GPUStack Authors, “GPUStack: Open-source gpu cluster manager,” GitHub repository, 2026, version 2.2.3, Apache License 2.0; accessed 14 August 2026. [Online]. Available: <https://github.com/gpustack/gpustack/tree/v2.2.3>
- [60] ggml-org, “llama.cpp,” GitHub repository, 2026, accessed 14 August 2026; the exact experimental tag and commit are recorded in Chapter 4. [Online]. Available: <https://github.com/ggml-org/llama.cpp>
- [61] M. Murphy, “Introducing the ibm granite 4.1 family of models,” IBM Research, Apr. 2026, published 29 April 2026; accessed 14 August 2026. [Online]. Available: <https://research.ibm.com/blog/granite-4-1-ai-foundation-models>

Appendix A

Requirements Catalogue and Traceability Matrix

This appendix contains all 111 MoSCoW v1.2 requirement records and the available mappings for its 72 Active requirements: 56 Must, 14 Should and 2 Could. The remaining records are 11 Deferred, 10 Superseded and 18 Excluded. These are scope counts, not completion counts. The checked repository records supervisor review as pending.

Table A.1: Baseline identity and selected revisions

Date	Record	Effect
11 July 2026	MoSCoW v1.1; CHG-001	Imported the proposed catalogue; retained as history.
14 July 2026	MoSCoW v1.2; CHG-013	Adopted the working first-release scope and resolved priority changes.
14 July 2026	RTM v1.3	Recorded active requirement mappings and planned evidence references.
14 July 2026	CHG-014	Reorganised the catalogue by type without changing content or scope.
14 July 2026	CHG-018	Established separate risk, assumption, constraint and licence registers.
15 July 2026	RTM v1.3.1; CHG-019	Updated register-management statuses, evidence links and scheduling; requirement meaning, priority, release role and first-release scope were unchanged.

The [catalogue](#), [RTM mappings](#) and [change log](#) are pinned to repository commit [44f339cca92aabb0e9fbb724393fa49b83bb795f](#), checked on 6 September 2026. This is a documentation snapshot, not the assessed application-release identity. Criteria, sources, rationales and checking methods are retained from the earlier RTM v1.3 Draft. F-M19 retains the scoped report-draft criterion; F-M12, F-S11 and R-M02 contain labelled report clarifications. These are verification-planning records, not amendments to the controlled baseline. Their agreement with RTM v1.3.1 remains to be confirmed before they are used for final acceptance.

[ART-RTM-XLSX-001](#) identifies the externally held editable workbook as `IXN_IBM_Granite_Requirements_Traceability_Matrix_v1.3.1_RACL_Validated.xlsx` (120,290 bytes), with recorded SHA-256 `2414c6790c2815a75edfcab0c9f35462dd334fe14d8c16e9810e80261f098e96`. That workbook has not

been checksum-verified for this appendix. CHG-019 records G-M05 (requirement), PD-05 (work package) and EP-007 (engineering practice) as *Implemented* (working status), *Validated* (validation state) and *Verified* (effective status) for register management only. As stated in [CR-019](#), this does not confirm every technical assumption, risk treatment, constraint check or final licence decision.

Complete Requirements Catalogue

Completion assessment. The review notes below record implementation or research completion, not formal acceptance or a change to the controlled RTM. Substantially completed means the main work exists but final reconciliation or publication remains. Developer confirmations are identified where used; unconfirmed checks are not treated as passes. Research assessments use the retained evidence reviewed at commit `ddea6c06e6b20541f67093aea038b590ae363c36`; the external RTM workbook was not independently checked. The measured research set contains 81 passed configurations; all 169 terminal attempt records, including failed, blocked and unavailable outcomes, remain in the evidence package. Original criteria and lifecycle classifications are preserved.

In Table A.2, group headings give category, priority and lifecycle. Release role is *Core* for Active Must records, except F-M21 (*App-integrated Experimental*) and R-M01–R-M07, R-M11, R-M12 and R-M14 (*Research*). Otherwise, release role matches *Should*, *Could*, *Deferred* or *Superseded*; Excluded records have role *Out of Scope*. IDs retain their original prefixes when priorities change. Each Active requirement is followed by a recorded draft criterion or a labelled report clarification; neither is a test result.

Table A.2: Complete catalogue with draft Active acceptance criteria

Requirement ID and statement; draft Active acceptance criterion

Functional — Must; Active

F-M01 The application must open on the target Windows 11 x64 Intel computer. *Acceptance:* Clean checkout builds; release build launches twice on the target laptop without fatal error. **Review:**

Implemented.

F-M02 The application must let the user select a supported local model. *Acceptance:* A user selects a valid GGUF file with the Windows picker; cancel returns safely; path is retained for inspection. **Review:**

Implemented.

F-M03 The application must validate a selected input before using it. *Acceptance:* Missing, empty, wrong-type, truncated and corrupt fixtures are rejected with a classified reason; valid fixture proceeds. **Review:**

Implemented.

F-M04 The application must show the correct inspection result state. *Acceptance:* Fixtures produce Ready, Ready with warnings, Conversion required, Unsupported, and Invalid or incomplete. **Review:**

Implemented.

F-M05 The application must display the model details needed for compatibility and memory checks. *Acceptance:* For a verified Granite GGUF, the app displays format, architecture/name where available, file size, weight type, context information, tokenizer/chat-template information or explicit missing-data warnings. **Review:**

Implemented.

F-M06 The application must show plain-English reasons for warnings or failures. *Acceptance:* Every tested failure code maps to a non-technical explanation and a valid recovery action; no raw exception is the only user message. **Review:**

Implemented.

Continued on the next page

Table A.2 continued

Requirement ID and statement; draft Active acceptance criterion
F-M07 The application must read the hardware information needed for fit analysis. <i>Acceptance:</i> The app records CPU details, installed and available RAM, available GPU/device information, disk space and runtime availability with clear units; values are cross-checked against trusted Windows tools. Review: Implemented.
F-M08 The application must estimate peak memory for a supported configuration. <i>Acceptance:</i> The estimate shows weights, KV cache, runtime/app overhead, OS allowance and safety reserve; matched measured error is recorded. Review: Implemented.
F-M09 The application must show whether a configuration is likely to fit, needs optimisation, or has no verified safe option. <i>Acceptance:</i> Boundary tests change the result at documented thresholds and show the limiting reason and uncertainty. Review: Implemented.
F-M10 The application must generate only complete configurations valid for the current model, runtime and device. <i>Acceptance:</i> All approved combinations are generated; all known invalid model/weight/cache/runtime/backend/device combinations are rejected. Review: Implemented.
F-M11 The application must provide Automatic, Quality, Balanced and Efficiency modes when valid alternatives exist. <i>Acceptance:</i> Each visible mode resolves to a complete valid configuration with a documented ranking rationale; unavailable modes are hidden or disabled; slider label updates correctly. Review: Partially implemented; available choices depend on the model and admitted configurations.
F-M12 The application must show the chosen complete configuration before starting an operation. <i>Draft clarification:</i> Before launch the UI shows model, runtime, backend, requested device target, weight format, KV cache, context, estimated memory, status and selection reason. Actual execution is checked separately under N-M07. Review: Implemented; confirmed during developer review. Full field coverage is not independently verified.
F-M13 The application must run local chat with at least one supported Granite GGUF model. <i>Acceptance:</i> One real Granite GGUF completes import, inspection, fit, selection, loading and valid local generation through WinUI without manual terminal commands. Review: Implemented.
F-M14 The application must keep the original model file unchanged. <i>Acceptance:</i> SHA-256 before and after inspection/processing is identical; generated artefacts use a different path. Review: Implemented.
F-M15 The application must give the user a clear next step after a failure. <i>Acceptance:</i> Each formal failure case offers a safe action such as Retry, Choose another model/configuration, Open details, or Return to standard route. Review: Implemented.
F-M18 The WinUI application must launch and control supported local command-line runtimes without requiring the user to enter terminal commands. <i>Acceptance:</i> Structured arguments, redirected stdout/stderr, hidden console, startup/error result, timeout and process-tree cleanup all pass tests. Review: Implemented.
F-M19 Long-running operations included in the active first-release workflow must stream progress or output and support safe cancellation. <i>Scoped draft criterion:</i> Progress or output remains visible for long-running operations in the active first-release workflow; cancellation returns a controlled state and leaves no owned orphan process or partial artefact presented as valid. Review: Implemented.

Continued on the next page

Table A.2 continued

Requirement ID and statement; draft Active acceptance criterion
F-M21 The application must run at least one verified TurboQuant-enabled Granite configuration end to end. <i>Acceptance:</i> A pinned supported model/runtime/cache/device combination is selected in WinUI, loads, generates through normal chat, proves TQ activation, records actual state, and supports cancellation. Review: Implemented; a TurboQuant chat was manually confirmed, without a recorded exact build and configuration for that check.
F-M22 A dependable upstream llama.cpp configuration must remain available when the TurboQuant route is unavailable or fails. <i>Acceptance:</i> A forced TQ unavailability/failure returns the user to a verified upstream configuration without claiming TQ was active. Review: Partially implemented; the upstream route is available, with full fallback verification still outstanding.
F-M28 The user must be able to copy generated chat output. <i>Acceptance:</i> Copy action places the exact selected/full generated response on the Windows clipboard and handles empty output safely. Review: Implemented.
Functional — Should; Active
F-M16 The application should download at least one approved recommended Granite model from a fixed trusted source. <i>Acceptance:</i> A listed model shows source/licence/size/location, downloads with progress, and enters inspection after completion. Review: Implemented for the five approved download packages.
F-M17 If model downloading is implemented, the application should prevent partial, corrupt or unverified downloads from being used as valid models. <i>Acceptance:</i> Disk space is checked; cancellation/failure leaves no valid-state artefact; final size and SHA-256 match the approved manifest. Review: Implemented; downloaded packages undergo integrity checks.
F-M20 The local chat should support at least two user turns in the same session. <i>Acceptance:</i> Two sequential prompts complete; second prompt uses the intended session context; reset starts a new session. Review: Implemented; local multi-turn chat is available.
F-M23 The application should create one new validated GGUF model artefact through a supported weight-quantisation workflow. <i>Acceptance:</i> A suitable source model is processed into a new GGUF, original hash is unchanged, output loads/inspects and can be selected for chat. Review: Implemented; creation of a new weight-quantised GGUF file through the application was manually confirmed.
F-M24 If the application generates a new model artefact, it should create a processing manifest containing source/output hashes, tool/version, settings and result. <i>Acceptance:</i> Manifest is saved for success and classified failure and validates against the agreed schema. Review: Implemented, based on developer confirmation; complete record-field coverage remains unverified.
F-S01 The application should support drag-and-drop model import. <i>Acceptance:</i> Supported file drop follows the same validation path; invalid/multiple drops are handled. Review: Implemented.
F-S02 The application should recognise complete OpenVINO IR model folders. <i>Acceptance:</i> Valid XML/BIN/tokenizer/config folder is recognised and incomplete folder receives a classified result. Review: Implemented.
F-S03 The application should recognise selected Hugging Face/Safetensors model folders. <i>Acceptance:</i> Selected known folder structures are classified without being falsely treated as directly runnable GGUF. Review: Implemented for supported source recognition; conversion is assessed separately.

Continued on the next page

Table A.2 continued

Requirement ID and statement; draft Active acceptance criterion	
F-S04	The user should be able to change the requested context length before configuration selection. <i>Acceptance:</i> Only supported range/steps are allowed and estimator/configuration update immediately. Review: Not established; a user-selectable context setting was not found in the reviewed interface.
F-S08	The user should be able to copy or save the technical inspection report. <i>Acceptance:</i> Report includes model/hardware/result/assumptions without secrets or sensitive prompts. Review: Not implemented; non-functional technical-report actions were removed.
F-S11	The application should provide one official OpenVINO GenAI inference route after its integration gate passes. <i>Draft clarification:</i> Pinned supported official-source configuration launches from WinUI, records actual device and generates. Deferral after gate review is recorded separately as a scope decision, not verified delivery. Review: Implemented; official OpenVINO inference is supported, with verification assessed separately.
F-S12	The application should export benchmark results as CSV or JSON. <i>Acceptance:</i> Export validates against schema and contains units, evidence classification and requested/actual state. Review: Not established in the application; experiment exports do not fulfil this application requirement.
F-S15	The project should provide one verified source-to-OpenVINO model-preparation route. <i>Acceptance:</i> Pinned IBM source revision converts in a clean recorded environment and the output passes structure/runtime checks, or a reproducible blocker is preserved. Review: Partially implemented; conversion support is in place, with complete success and recovery checks still outstanding.
Functional — Could; Active	
C-02	The application could keep a local benchmark history. <i>Acceptance:</i> When approved, completed benchmark summaries can be stored, listed and removed locally without storing sensitive prompt or document content. Review: Not established in the application; experiment records and chat history are not benchmark history.
C-03	The application could show charts for memory and speed trade-offs. <i>Acceptance:</i> When approved, charts label units and distinguish measured, estimated and externally reported values. Review: Partially implemented; memory information is visualised, but report trade-off tables do not establish application charts.
Functional — Could; Deferred	
C-04	The application could support selected non-Granite GGUF models.
C-05	The project could explore DirectML as another Windows route.
C-06	The project could test an Intel NPU when suitable hardware is available.
C-07	The project could explore macOS and Metal after the Windows release.
C-08	The project could explore a cross-platform CMake backend.
C-09	The project could try a model near 32 billion parameters on stronger hardware.
C-10	The application could support more than one model session at a time.
F-M25	A future release could import and preserve at least one supported text-based knowledge file for a bounded retrieval workflow.
F-M26	A future release could create an uncompressed embedding baseline and use a pinned TurboVec implementation to compress or optimise vectors.
F-M27	A future release could retrieve relevant sections from a local index and provide them to the Granite chat workflow.

Continued on the next page

Table A.2 continued

Requirement ID and statement; draft Active acceptance criterion	
Functional — Should; Superseded	
F-S05	The user should be able to cancel a long task.
F-S06	Chat answers should appear while they are being generated.
F-S07	The chat should support more than one turn in the same session.
F-S09	The application should create a new GGUF file through a supported weight-quantisation step.
F-S10	The application should save a processing record for each new model file.
F-S13	The main local inference route should not need a local web server or network port.
F-S14	The application should expose an AtomicBot TurboQuant route only after its gate passes.
Functional — Could; Superseded	
C-01	The application could help the user find or download suitable models.
Non-functional — Must; Active	
N-M01	The core workflow must work locally after required models and tools are installed. <i>Acceptance:</i> With network disconnected, model inspection, configuration selection and upstream local chat complete using installed assets. Review: Implemented; offline use was manually confirmed.
N-M02	The application must not upload models, prompts, knowledge files or answers to a cloud AI service in the core workflow. <i>Acceptance:</i> Network inspection shows no cloud AI upload during core model/document/chat workflow; any initial approved download is separately documented. Review: Implemented by design; no independent network-traffic audit was completed.
N-M03	The application window must remain responsive during long tasks. <i>Acceptance:</i> UI input/paint remains responsive during each long task; no synchronous runtime call blocks the UI thread. Review: Implemented; asynchronous operation is present, with limited responsiveness verification.
N-M04	The application must show the current state of a long task. <i>Acceptance:</i> Each long operation reports at least Starting, Running/Loading/Generating/Processing, Completed, Cancelled or Failed; progress is determinate where available. Review: Implemented.
N-M05	The application must handle model/document paths and process arguments safely. <i>Acceptance:</i> Tests cover spaces, Unicode, quotes and supported special characters; no shell string concatenation/injection path is used. Review: Implemented; selected checks cover path and argument handling.
N-M06	The application must clean temporary files and stopped child processes. <i>Acceptance:</i> Success, cancellation, timeout and crash tests leave no unintended child process and no partial artefact marked complete. Review: Implemented; cleanup routines are present, while complete verification remains outstanding.
N-M07	Each final run must record requested and actual backend, device and optimisation state. <i>Acceptance:</i> Manifest separates requested from actual runtime/backend/device/cache and cites the evidence used to determine actual state. Review: Implemented, based on developer confirmation; this does not establish actual device placement for every experimental run.
N-M08	A clean copy of the repository must build and run its automated tests. <i>Acceptance:</i> Documented x64 restore/build/test commands pass from a clean checkout in the pinned environment. Review: Implemented; selected hosted build and test checks passed, not every test group.

Continued on the next page

Table A.2 continued

Requirement ID and statement; draft Active acceptance criterion	
N-M09	The repository must not contain secrets, personal test data or large proprietary model files. <i>Acceptance:</i> Release scan finds no credentials, real patient/pupil data, unapproved model weights or prohibited third-party files. Review: Implemented; a limited tracked-file scan found no confirmed credentials or full production model weights. Git history, binaries and personal-data absence were not verified.
N-M10	Experimental options must be clearly labelled in the interface. <i>Acceptance:</i> Every Experimental registry entry displays a persistent label and limitations before selection and launch. Review: Implemented; experimental labelling is retained where required.
N-M11	An experimental option must not be reported as active unless activation is proved. <i>Acceptance:</i> When proof is absent the run is labelled Unverified/Fallback rather than active; supported routes have direct activation evidence. Review: Implemented for configurations admitted by their supporting evidence.
N-M12	The main local inference route must not require a local web server or open network port. <i>Acceptance:</i> Main upstream route completes with no listening port created by the application/runtime; evidence is recorded. Review: Implemented for the upstream route; full runtime verification remains separate.
N-M13	The main workflow must support keyboard operation and Windows text scaling. <i>Acceptance:</i> Core tasks complete keyboard-only; focus is visible; content remains usable at 200% Windows text scaling with no critical clipping. Review: Implemented, based on developer review; complete keyboard and 200% text-scaling verification is not established.
N-M14	The project must produce a basic distributable Windows x64 release build with documented dependencies. <i>Acceptance:</i> Release artefact launches on the target machine; checksum, commit/tag, runtime dependencies and install/run steps are recorded. Review: Partially completed; the app runs on the test computer, but the complete release package and setup instructions are not yet confirmed.
Non-functional — Should; Active	
N-S02	The project should provide a simple packaged installation route such as MSIX. <i>Acceptance:</i> Fresh install/uninstall works on the target Windows machine and dependencies/limitations are documented. Review: Partially completed; a complete installer and successful installation and removal on a clean computer are not yet confirmed.
Non-functional — Should; Superseded	
N-S01	The main workflow should support keyboard use and Windows text scaling.
Research — Must; Active	
R-M01	The project must compare at least one verified TurboQuant run with a matched standard KV-cache baseline. <i>Acceptance:</i> Same model, weights, prompt/template, context, settings, hardware and measurement method; TQ activation proved; differences reported. Review: Completed through matched standard-cache and TurboQuant runs with retained activation evidence.
R-M02	The project must identify, pin and document the exact TurboVec implementation and decide its release role. <i>Draft clarification:</i> Repository, commit/version, licence, platform, input/output, vector representation and retrieval contract are recorded before integration. A dated decision records the release role. Review: Completed; the pinned TurboVec implementation was documented and assigned a demonstrator-only release role.

Continued on the next page

Table A.2 continued

Requirement ID and statement; draft Active acceptance criterion
R-M03 The project must complete and preserve an official OpenVINO GenAI Granite baseline or a reproducible blocker. <i>Acceptance:</i> Exact official-source model/revision, conversion/runtime versions, requested/actual device and outcome are recorded; pass or blocker is reproducible. Review: Completed; the official OpenVINO baseline is retained, but independent actual-device and fallback tracing is incomplete.
R-M04 The project must measure memory use for every final test configuration. <i>Acceptance:</i> Each final row records available RAM before, peak process-tree working set/private where used, relevant GPU shared memory and measurement definition. Review: Completed for peak working-set measurement across the 81 passed configurations; other required memory fields were not recorded consistently.
R-M05 The project must evaluate runtime performance for every final test configuration. <i>Acceptance:</i> Cold/warm load, TTFT, prompt speed, generation speed and total response duration are reported where available using fixed definitions and repetitions. Review: Completed; all passed configurations have key timing and throughput results, with route-specific metric limitations.
R-M06 The project must compare output quality with a fixed prompt set and scoring guide. <i>Acceptance:</i> Prompt/rubric versions are frozen before final comparison; raw answers, instruction/format/fact scores and limitations are retained. Review: Completed using fixed route-specific prompts and scoring guides; incompatible quality scores are not ranked across routes.
R-M07 The project must identify the largest stable tested context for selected final configurations. <i>Acceptance:</i> A versioned step test records maximum attempted and maximum stable context, success/retrieval result, memory and failure reason; it is not called the model maximum. Review: Partially completed; context observations exist, but the planned systematic context-step study was not completed.
R-M08 The project must preserve a complete record of every final experiment. <i>Acceptance:</i> Each formal experiment has an ID, hardware/model/runtime/config hashes, requested/actual state, command, raw stdout/stderr, measurements, outputs and failure status. Review: Completed; final records and raw evidence are retained, with documented metadata and derivation gaps.
R-M09 The project must record failed attempts and known limitations. <i>Acceptance:</i> Every formal failure has ID, stage, code, evidence, likely cause, next action and resolution status; limitations appear in final documentation. Review: Completed; negative outcomes and limitations are retained, although some required structured failure fields are missing.
R-M10 Another developer must be able to reproduce the clean build and core result from written instructions. <i>Acceptance:</i> Independent clean-checkout reproduction completes the documented x64 build, tests and main upstream route; deviations are recorded. Review: Partially completed; instructions and automated checks exist, but independent clean-checkout reproduction is not recorded.
R-M11 The project must determine which selected Granite/runtime/backend combinations run reliably on the tested Intel CPU and integrated GPU. <i>Acceptance:</i> Matrix includes upstream llama.cpp, selected TQ forks, official OpenVINO and OpenVINO TQ with model, commit/version, build, requested/actual device, success/failure and role. Review: Completed for the selected tested combinations, subject to incomplete OpenVINO device tracing and partial-placement limitations.

Continued on the next page

Table A.2 continued

Requirement ID and statement; draft Active acceptance criterion	
R-M12	The project must assess selected complete configurations against 4 GB, 8 GB and 16 GB total system-memory budgets. <i>Acceptance:</i> Each result is labelled physical, controlled-limit, calculated or predicted and includes OS/app/model/KV/runtime/shared-memory allowance and uncertainty. Review: Partially completed; the approximately 16 GB host was assessed, but complete 4 GB and 8 GB assessments were not completed.
R-M14	The project must quantify memory-estimator error and false-safe/false-unsafe recommendations. <i>Acceptance:</i> Matched rows report predicted, measured, absolute error, percentage error, fit decision, false-safe/false-unsafe and confidence. Review: Partially completed; the estimator and measured data provide the groundwork for validation, but error and false-safe or false-unsafe rates were not measured.
Research — Should; Deferred	
R-M13	If TurboVec integration is reactivated, the project should compare compressed or optimised vectors with an uncompressed-vector baseline. Review: Deferred; later supporting comparisons did not formally reactivate this conditional requirement.
Research — Should; Superseded	
R-S01	The project should run a small TurboVec feasibility test if the scope decision supports it. Review: Superseded; later feasibility work does not change its recorded lifecycle.
Governance — Must; Active	
G-M01	The project problem, aim, research questions, objectives, first-release scope, exclusions and satisfactory outcome must be version-controlled. <i>Acceptance:</i> Dated document contains all agreed sections, source links and status; linked from repository README. Review: Completed; the version-controlled baseline records the aims, objectives, scope and exclusions.
G-M02	The project must maintain one versioned MoSCoW requirements baseline with stable IDs and measurable acceptance criteria. <i>Acceptance:</i> Every active Must has a stable ID, source, rationale, acceptance and verification; changes are dated and approved. Review: Completed; stable IDs and acceptance criteria exist. Older summaries still need reconciliation.
G-M03	The project must maintain bidirectional traceability from requirements to objectives/RQs, work packages, implementation, tests and evidence. <i>Acceptance:</i> No active Must row lacks source, rationale, acceptance, WP, component, verification, evidence path, owner and status; release audit confirms reverse links. Review: Substantially completed; mappings and evidence links exist, with final agreement checks remaining.
G-M04	The project must record proportionate architecture views, key ADRs, stable contracts, states and diagnostic codes. <i>Acceptance:</i> Context/component/process/deployment views and ADRs for WinUI, adapters/no port, upstream/TQ/OpenVINO/TurboVec/evidence agree with code and requirements. Review: Completed, pending review-reference update; the diagram accuracy review is owner-confirmed, but its reference and the outdated note need updating.
G-M05	The project must maintain a consolidated risk, assumption, constraint and licence register. <i>Acceptance:</i> Each high risk has probability, impact, owner, trigger, validation, mitigation, contingency and status; assumptions are confirmed/rejected with evidence; licences are reviewed. <i>Status boundary:</i> CHG-019 records register-management completion, not satisfaction of this full criterion. Review: Completed; the consolidated registers were validated. Open risks and licence decisions remain separate.

Continued on the next page

Table A.2 continued

Requirement ID and statement; draft Active acceptance criterion	
G-M06	Requirement and scope changes must be linked to dated decisions, GitHub issues, pull requests/commits and affected tests. <i>Acceptance:</i> Every approved change records reason, date, affected IDs/WPs/tests/report sections and superseded/replacement links. Review: Partially completed; dated changes exist, but full links to affected tests and commits remain unconfirmed.
G-M07	Irreplaceable raw evidence from completed feasibility campaigns must be recovered, hashed, indexed and backed up. <i>Acceptance:</i> Upstream, AtomicBot, animehacker and OpenVINO campaigns each have model/runtime/hash manifest, raw-output location, checksum and independent backup or explicit gap. Review: Completed for the tested snapshot; owner-supplied recovery confirmation covers 2,708 final and raw evidence files with no differences. Later additions need a refreshed backup.
G-M08	The release must include a README, user manual, developer/build guide, known limitations and final feature-status table. <i>Acceptance:</i> Documents cover install/run/use/build/test/evidence/limitations; feature statuses match the RTM. Review: Substantially completed; guides and limitations are written. The latest guides still need publishing, feature statuses need checking against the requirements, and runtime setup instructions need completing.
G-M09	The final release must be tied to a Git tag, checksums, evidence pack, independent backup and evidence-based answers to every RQ. <i>Acceptance:</i> Tag/commit/checksums/backups exist; all Must rows have final status/evidence; report answers each RQ and states limitations/negative results. Review: Partially completed; final release identification, backup coverage and research-question links remain to be confirmed.
Exclusion — Won't; Excluded	
W-01	The project will not train or fine-tune a Granite model.
W-02	The first release will not support every GGUF, OpenVINO or Hugging Face model.
W-03	The project will not guarantee that a 32B model runs on the target laptop.
W-04	The project will not guarantee that every model runs on a 4 GB, 8 GB or 16 GB computer.
W-05	The project will not claim six-times compression, near-zero quality loss or sub-second latency without its own evidence.
W-06	The core workflow will not depend on cloud inference.
W-07	The project will not use real patient data or provide medical advice.
W-08	The interface will not show an unverified experimental option as a normal working option.
W-09	The project will not build a full replacement for llama.cpp, OpenVINO, LM Studio or Ollama.
W-10	Full cross-platform delivery is not required for this release.
W-11	The application will not automatically re-quantise an already quantised GGUF through an unsafe route.
W-12	A runtime-only TurboQuant setting will not be presented as a new exportable model file.
W-13	TurboVec will not be described as directly compressing the original knowledge file.
W-14	The application will not provide unrestricted model downloads from arbitrary or untrusted sources.
W-15	The first release will not be a full enterprise RAG, document-management or vector-database platform.
W-16	The prototype will not claim operational clinical, NHS, school or classroom approval.
W-17	The first release will not provide multi-user server operation or multiple simultaneous model sessions.

Continued on the next page

Table A.2 continued

Requirement ID and statement; draft Active acceptance criterion

W-18 The first release will not automatically update runtimes/models or collect prompts/documents for analytics.

Active Requirements Traceability

The available draft workbook uses the label *Validation* for a more specific purpose: it records the review of acceptance evidence and contributes to the effective status. It must not be interpreted as evidence that the workflow has been validated with its intended users.

Table A.3 supplies the source, rationale, checking method and mappings for the Active requirements in Table A.2. Neither table reports completed implementation or final verification. The retained source rationales describe project intentions, not demonstrated user needs or completed capabilities. Objective/RQ, work-package and evidence mappings follow the pinned RTM v1.3 export.

The O1–O11 identifiers in the mappings and source references below refer to the planning objectives in *Project Definition and Scope Baseline v1.1, Section 4*, not Chapter 1’s report-summary objectives CO1–CO5. This report-level crosswalk interprets their principal coverage; it does not change controlled IDs, optional scope or deferrals.

Report objective	Report scope, condensed	Planning objectives
CO1	Guided application and model workflow	O1–O4; O6
CO2	Runtime integration and execution control	O5; O6; O8
CO3	Optimisation and experimental routes	O7–O9
CO4	Controlled configuration evaluation	O8; O10
CO5	Verification, packaging and documentation	O1; O11

For each record, the bracketed mapping lists *objectives; research questions; work packages*, separated by vertical bars. “All RQs” and an absent RQ mapping are retained as recorded. In source references, M means MoSCoW v1.1 at the same requirement ID; PD means Project Definition. Unless an exception is shown, the test/evidence ID is *AC-requirement-ID* and the planned path is [docs/evidence/requirements/requirement-ID/](#). Codes E01–E25 resolve to the exact exceptional paths in Table A.4. These references identify where evidence is expected, not proof that it exists or meets the criterion.

Table A.3: Active requirements: source and traceability

Requirement ID; source and rationale; check; mappings

F-M01 *Check:* Clean-build test and release demo. *Source:* M; PD §5. *Rationale:* A usable prototype must launch on the target platform. [O1; O11 | RQ4 | IM-01; FR-04]

F-M02 *Check:* Picker integration test and UI test. *Source:* M; PD §5.2.2. *Rationale:* Model import is the first user task. [O2 | RQ4 | IM-02]

F-M03 *Check:* Automated validation matrix. *Source:* M; PD §5.2.2–5.2.4. *Rationale:* Untrusted or corrupt files must not reach a runtime. [O2; O8 | RQ4 | IM-03; IM-04]

F-M04 *Check:* State-decision table test and UI state test. *Source:* M; acceptance rule p.6. *Rationale:* Users need a consistent decision before continuing. [O2; O8 | RQ4 | IM-06; IM-07]

Continued on the next page

Table A.3 continued

Requirement ID; source and rationale; check; mappings	
F-M05	<i>Check:</i> Real-model inspection test and fixture tests. <i>Source:</i> M; PD §5.2.4. <i>Rationale:</i> Later decisions depend on trustworthy metadata. [O2; O3; O4 RQ3; RQ4 IM-05]
F-M06	<i>Check:</i> Failure-message tests and task-based usability test. <i>Source:</i> M; UX requirements. <i>Rationale:</i> Non-specialist users must understand what happened. [O1; O2; O8 RQ4 IM-07; RT-05; FR-02]
F-M07	<i>Check:</i> Hardware service unit/integration test. <i>Source:</i> M; PD §5.2.5. <i>Rationale:</i> Memory and backend recommendations require current hardware data. [O3; O4 RQ1; RQ3; RQ4 HE-01; HE-02]
F-M08	<i>Check:</i> Formula/unit tests and predicted-versus-measured calibration. <i>Source:</i> M; PD §5.2.6. <i>Rationale:</i> Users need a transparent fit prediction before loading a model. [O3; O10 RQ3; RQ4 HE-03; HE-04]
F-M09	<i>Check:</i> Boundary and UI decision tests. <i>Source:</i> M; PD §5.2.6. <i>Rationale:</i> The raw estimate must become an understandable decision. [O3; O4; O8 RQ3; RQ4 HE-07]
F-M10	<i>Check:</i> Decision-table and registry tests. <i>Source:</i> M; PD §5.2.7. <i>Rationale:</i> Individually valid settings can still form an invalid combination. [O4; O8 RQ1; RQ3; RQ4 HE-02; HE-05]
F-M11	<i>Check:</i> Mode algorithm tests and UI interaction tests. <i>Source:</i> M; PD §5.2.8. <i>Rationale:</i> Users need goal-based choices rather than low-level flags. [O4; O8 RQ3; RQ4 HE-06]
F-M12	<i>Check:</i> UI acceptance test. <i>Source:</i> M; PD §5.2.8. <i>Rationale:</i> Users must understand what will run and whether it is experimental. [O4; O8 RQ4 HE-08]
F-M13	<i>Check:</i> End-to-end application test. <i>Source:</i> M; PD §5.2.10–5.2.12. <i>Rationale:</i> A complete inference result proves the selected configuration is usable. [O5; O6; O8 RQ1; RQ4 RT-01; RT-02; RT-03; RT-04]
F-M14	<i>Check:</i> Hash integrity test. <i>Source:</i> M. <i>Rationale:</i> Inspection and optimisation must not damage the user’s source asset. [O2; O7 RQ4 IM-04; QX-01; QX-02]
F-M15	<i>Check:</i> Failure/recovery test and usability task. <i>Source:</i> M. <i>Rationale:</i> Failure handling must support recovery, not only reporting. [O1; O8 RQ4 IM-07; RT-05; FR-02]
F-M18	<i>Check:</i> Contract, path-safety and integration tests. <i>Source:</i> Agreed Objective O5; promoted from F-S13. <i>Rationale:</i> The app’s main contribution is to hide specialist CLI operation behind a safe adapter. [O5; O6; O8 RQ1; RQ4 RT-01; RT-02; RT-05]
F-M19	<i>Check:</i> Streaming, responsiveness, cancellation and cleanup tests. <i>Source:</i> Agreed PD §5.2.9–5.2.12; promoted from F-S05/F-S06. <i>Rationale:</i> Users need responsiveness and control during model operations. [O5; O6; O8 RQ4 IM-03; RT-03; QX-03]
F-M21	<i>Check:</i> App-integrated TurboQuant E2E and activation audit. <i>Source:</i> Agreed PD §5.2.11. <i>Rationale:</i> TurboQuant is a central contribution and must be more than an external benchmark. [O5; O6; O8 RQ1; RQ2; RQ4 QX-03; QX-04]
F-M22	<i>Check:</i> Forced-failure and recovery test. <i>Source:</i> Agreed PD §5.2.11. <i>Rationale:</i> An experimental backend must not make the core product unusable. [O5; O6; O8 RQ1; RQ4 QX-04; RT-05]
F-M28	<i>Check:</i> UI/clipboard test. <i>Source:</i> Agreed PD §5.2.12. <i>Rationale:</i> Users need to reuse useful local output. [O6; O8 RQ4 RT-03]
F-M16	<i>Check:</i> Download integration and end-to-end test. <i>Source:</i> Agreed PD §5.2.3; promoted from C-01. <i>Rationale:</i> The agreed product aim includes controlled model download for non-specialists. [O2 RQ4 DL-01; DL-02; DL-03]

Continued on the next page

Table A.3 continued

Requirement ID; source and rationale; check; mappings	
F-M17	<i>Check:</i> Failure-injection, cancellation and hash tests. <i>Source:</i> Agreed PD §5.2.3. <i>Rationale:</i> Large external assets require integrity and failure controls. [O2; O11 RQ4 DL-02; DL-03]
F-M20	<i>Check:</i> Multi-turn end-to-end test. <i>Source:</i> Agreed PD §5.2.12; promoted from F-S07. <i>Rationale:</i> A real chat workflow needs maintained conversation context. [O6 RQ4 RT-04]
F-M23	<i>Check:</i> Processing integration, hash, reinspection and generation tests. <i>Source:</i> Agreed Objective O7; promoted from F-S09. <i>Rationale:</i> The project includes a real model-file optimisation/export capability separate from KV-cache optimisation. [O7 RQ3; RQ4 QX-01; QX-02]
F-M24	<i>Check:</i> Manifest schema/contract test and evidence audit. <i>Source:</i> Agreed Objective O7; promoted from F-S10. <i>Rationale:</i> Optimised artefacts need provenance and reproducibility. [O7; O11 RQ3; RQ4 QX-02]
F-S01	<i>Check:</i> UI drag/drop tests. <i>Source:</i> M. <i>Rationale:</i> Drag-and-drop improves convenience after the picker route is stable. [O2 RQ4 IM-03]
F-S02	<i>Check:</i> Folder fixture matrix. <i>Source:</i> M. <i>Rationale:</i> The second runtime route needs correct multi-file input recognition. [O2; O8 RQ1; RQ4 IM-06; OV-02]
F-S03	<i>Check:</i> Folder fixture tests. <i>Source:</i> M. <i>Rationale:</i> Recognition enables honest Conversion required/Unsupported decisions. [O2 RQ4 IM-06]
F-S04	<i>Check:</i> Boundary and UI tests. <i>Source:</i> M. <i>Rationale:</i> Context is a major memory/quality/performance trade-off. [O3; O4 RQ2; RQ3; RQ4 HE-06; HE-07]
F-S08	<i>Check:</i> Export content test. <i>Source:</i> M. <i>Rationale:</i> Users and developers need a shareable explanation of compatibility. [O2; O8 RQ4 IM-07]
F-S11	<i>Check:</i> App integration E2E test. <i>Source:</i> M. <i>Rationale:</i> A second Intel-optimised route strengthens the product but must not block the dependable core. [O8 RQ1; RQ4 OV-01; OV-02]
F-S12	<i>Check:</i> Schema/export test. <i>Source:</i> M. <i>Rationale:</i> Structured results support analysis and reproducibility. [O10; O11 RQ1; RQ2; RQ3 FR-03]
F-S15	<i>Check:</i> Conversion and runtime test. <i>Source:</i> M. <i>Rationale:</i> Official provenance and reproducibility improve the OpenVINO claim. [O8 RQ1 OV-01; OV-03 <i>Path:</i> E24]
C-02	<i>Check:</i> Repository and privacy tests. <i>Source:</i> MoSCoW v1.1. <i>Rationale:</i> Lower priority or future work. [O10; O11 — FR-03; FR-05 <i>Test/evidence:</i> Scope review]
C-03	<i>Check:</i> Chart and data-classification test. <i>Source:</i> MoSCoW v1.1. <i>Rationale:</i> Lower priority or future work. [O8; O10 — FR-03 <i>Test/evidence:</i> Scope review]
N-M01	<i>Check:</i> Offline end-to-end test. <i>Source:</i> M. <i>Rationale:</i> Local inference is the product’s defining operational property. [O1; O5; O6 RQ4 RT-05; FR-02]
N-M02	<i>Check:</i> Network/offline check and data-flow review. <i>Source:</i> M; expanded scope. <i>Rationale:</i> Target sectors require careful data handling. [O1; O8; O9 RQ4; RQ-TV FR-02]
N-M03	<i>Check:</i> UI responsiveness test. <i>Source:</i> M. <i>Rationale:</i> Blocking the UI makes download, loading, quantisation and generation unusable. [O1; O8 RQ4 IM-03; RT-03; QX-01; DL-02; TV-03]
N-M04	<i>Check:</i> State-transition and UI tests. <i>Source:</i> M. <i>Rationale:</i> Users need progress and confidence that work is continuing. [O1; O8 RQ4 IM-03; RT-03; QX-01; DL-02; TV-03]
N-M05	<i>Check:</i> Path and argument security tests. <i>Source:</i> M. <i>Rationale:</i> Imported paths and CLI arguments are untrusted boundaries. [O5; O11 RQ4 RT-01; RT-02; QX-01; DL-02; TV-02]

Continued on the next page

Table A.3 continued

Requirement ID; source and rationale; check; mappings	
N-M06	<i>Check:</i> Cleanup/failure-injection tests. <i>Source:</i> M. <i>Rationale:</i> Cancellation and failures must not leave unsafe resource leaks. [O5; O7; O11 RQ4 RT-03; RT-05; QX-01; DL-02; TV-03]
N-M07	<i>Check:</i> Manifest audit. <i>Source:</i> M. <i>Rationale:</i> Silent fallback would invalidate compatibility and performance claims. [O8; O10; O11 RQ1; RQ2 PD-09; FR-03 <i>Path:</i> E07]
N-M08	<i>Check:</i> Clean-build and CI test. <i>Source:</i> M. <i>Rationale:</i> The prototype must not depend on an unrecorded local machine state. [O11 RQ4 PD-08; FR-04 <i>Path:</i> E13]
N-M09	<i>Check:</i> Repository scan. <i>Source:</i> M. <i>Rationale:</i> The public/private repo must remain safe and legally manageable. [O11 RQ4 PD-08; FR-05 <i>Path:</i> E14]
N-M10	<i>Check:</i> Registry/UI inspection test and UX task. <i>Source:</i> M. <i>Rationale:</i> Users must not confuse limited-evidence routes with dependable ones. [O8 RQ4 HE-02; QX-03; OV-02; TV-04]
N-M11	<i>Check:</i> Log/manifest audit and forced-unverified test. <i>Source:</i> M. <i>Rationale:</i> Accepted flags and successful output are insufficient proof. [O8; O11 RQ1; RQ2 QX-03; QX-04; OV-03; TV-03]
N-M12	<i>Check:</i> Port/network inspection test. <i>Source:</i> Promoted/strengthened from F-S13; architecture decision. <i>Rationale:</i> Direct child-process communication reduces attack surface and deployment complexity. [O5; O11 RQ4 PD-06; RT-01; RT-02]
N-M13	<i>Check:</i> Accessibility checklist and task test. <i>Source:</i> Promoted from N-S01; satisfactory outcome. <i>Rationale:</i> The intended users require a basic accessible desktop experience. [O1; O8; O11 RQ4 IM-01; FR-02 <i>Path:</i> E15]
N-M14	<i>Check:</i> Release smoke and checksum test. <i>Source:</i> Satisfactory outcome; PD §5.2.20. <i>Rationale:</i> A working project needs a repeatable handover artefact, even if full MSIX is deferred. [O11 RQ4 FR-04; FR-05 <i>Path:</i> E16]
N-S02	<i>Check:</i> Installation test. <i>Source:</i> M. <i>Rationale:</i> A packaged installer improves non-specialist deployment beyond the basic release build. [O11 RQ4 FR-04 <i>Path:</i> E25]
R-M01	<i>Check:</i> Controlled matched experiment. <i>Source:</i> M; RQ2. <i>Rationale:</i> The core research contribution is a controlled memory/quality/performance trade-off. [O8; O10 RQ2 QX-04; FR-03 <i>Test/evidence:</i> EXP-TQ-COMP-001 <i>Path:</i> E01]
R-M02	<i>Check:</i> Source inspection, licence review and technical spike. <i>Source:</i> M revised by agreed scope. <i>Rationale:</i> The implementation name alone is insufficient for safe integration. [O9 RQ-TV PD-10; TV-01 <i>Test/evidence:</i> DEC-TV-001 <i>Path:</i> E02]
R-M03	<i>Check:</i> Official-source conversion/runtime experiment. <i>Source:</i> M; updated evidence. <i>Rationale:</i> OpenVINO is the second Intel route and needs evidence distinct from community/custom routes. [O8 RQ1; RQ2 OV-01; OV-03 <i>Test/evidence:</i> EXP-OV-OFFICIAL-001 <i>Path:</i> E03]
R-M04	<i>Check:</i> Metrics audit. <i>Source:</i> M. <i>Rationale:</i> Memory is central to the project aim. [O6; O10 RQ2; RQ3 FR-03 <i>Test/evidence:</i> MET-MEM <i>Path:</i> E04]
R-M05	<i>Check:</i> Performance experiment audit. <i>Source:</i> M. <i>Rationale:</i> Memory savings alone do not prove usefulness. [O6; O10 RQ1; RQ2; RQ3 FR-03 <i>Test/evidence:</i> MET-PERF <i>Path:</i> E04]
R-M06	<i>Check:</i> Quality evaluation. <i>Source:</i> M. <i>Rationale:</i> Compression must be evaluated against model behaviour. [O6; O10 RQ2; RQ3 PD-09; FR-03 <i>Test/evidence:</i> QUAL-FINAL <i>Path:</i> E05]
R-M07	<i>Check:</i> Context stability experiment. <i>Source:</i> M. <i>Rationale:</i> KV-cache optimisation matters most as context grows. [O6; O10 RQ2; RQ3 FR-03 <i>Test/evidence:</i> CTX-FINAL <i>Path:</i> E06]

Continued on the next page

Table A.3 continued

Requirement ID; source and rationale; check; mappings		
R-M08	<i>Check:</i> Evidence-manifest audit. <i>Source:</i> M. <i>Rationale:</i> Results must be auditable and reproducible. [O11 All RQs PD-03; PD-08; FR-05 <i>Test/evidence:</i> EVID-AUDIT <i>Path:</i> E07]	
R-M09	<i>Check:</i> Failure-register and report audit. <i>Source:</i> M. <i>Rationale:</i> Negative results prevent biased conclusions and support troubleshooting. [O11 All RQs PD-03; FR-03; FR-05 <i>Test/evidence:</i> FAIL-AUDIT <i>Path:</i> E08]	
R-M10	<i>Check:</i> Independent reproduction test. <i>Source:</i> M. <i>Rationale:</i> Academic engineering evidence must be reproducible. [O11 All RQs FR-04; FR-05 <i>Test/evidence:</i> REPRO-001 <i>Path:</i> E09]	
R-M11	<i>Check:</i> Cross-route evidence audit. <i>Source:</i> Agreed RQ1 and PD. <i>Rationale:</i> RQ1 requires exact compatibility evidence, not a universal Intel claim. [O8; O10 RQ1 PD-09; FR-03 <i>Test/evidence:</i> COMPAT-FINAL <i>Path:</i> E10]	
R-M12	<i>Check:</i> Budget-analysis audit. <i>Source:</i> Agreed RQ3 and PD. <i>Rationale:</i> RQ3 explicitly concerns lower-memory feasibility. [O10 RQ3 HE-04; FR-03 <i>Test/evidence:</i> MEM-BUDGET-001 <i>Path:</i> E11]	
R-M14	<i>Check:</i> Estimator validation experiment. <i>Source:</i> Agreed RQ3/RQ4 and satisfactory outcome. <i>Rationale:</i> An estimator must be calibrated rather than merely demonstrated. [O3; O10 RQ3; RQ4 HE-04; FR-03 <i>Test/evidence:</i> EST-VALID-001 <i>Path:</i> E12]	
G-M01	<i>Check:</i> Document review and Git history. <i>Source:</i> Engineering audit; corrected timetable PD-01. <i>Rationale:</i> All requirements and evaluation must derive from a stable project definition. [O11 All RQs PD-01 <i>Path:</i> E17]	
G-M02	<i>Check:</i> Requirements quality review. <i>Source:</i> Engineering audit; corrected timetable PD-04. <i>Rationale:</i> A single controlled catalogue prevents contradictory requirements. [O11 All RQs PD-04]	
G-M03	<i>Check:</i> RTM completeness formula and manual audit. <i>Source:</i> MoSCoW §9; engineering audit; corrected timetable PD-04. <i>Rationale:</i> Nothing important should be forgotten or counted complete without proof. [O11 All RQs PD-04; FR-01; FR-05 <i>Path:</i> E18]	
G-M04	<i>Check:</i> Architecture consistency review. <i>Source:</i> Engineering audit; corrected timetable PD-06/PD-07. <i>Rationale:</i> UI and backend work need clear boundaries and justified decisions. [O1; O5; O8; O9; O11 RQ1; RQ4 PD-06; PD-07 <i>Path:</i> E19]	
G-M05	<i>Check:</i> Register audit. <i>Source:</i> Engineering audit; corrected timetable PD-05. <i>Rationale:</i> High-risk experimental dependencies and sector claims require active control. [O11 All RQs PD-05; FR-05 <i>Path:</i> E20]	
G-M06	<i>Check:</i> Change-log/PR audit. <i>Source:</i> MoSCoW §9; engineering audit change-control gap. <i>Rationale:</i> The project must not silently rewrite history. [O11 All RQs PD-04; PD-08; FR-05 <i>Path:</i> E21]	
G-M07	<i>Check:</i> Evidence recovery audit. <i>Source:</i> Corrected timetable PD-03; current-state audit. <i>Rationale:</i> Workbook summaries alone may not support final claims. [O11 RQ1; RQ2; RQ3 PD-03 <i>Path:</i> E22]	
G-M08	<i>Check:</i> Documentation and walkthrough review. <i>Source:</i> PD §5.2.20; corrected timetable FR-04. <i>Rationale:</i> The system must be understandable and handover-ready. [O11 RQ4 FR-04 <i>Path:</i> E23]	
G-M09	<i>Check:</i> Final release gate audit. <i>Source:</i> Satisfactory outcome; corrected timetable FR-05. <i>Rationale:</i> The final project must have a controlled baseline and defensible conclusions. [O11 All RQs FR-05 <i>Path:</i> E16]	

Continued on the next page

Table A.3 continued

Requirement ID; source and rationale; check; mappings

Table A.4: Exceptional planned evidence paths

Code	Repository-relative path
E01	experiments/processed-results/EXP-TQ-COMP-001/
E02	docs/architecture/decisions/ADR-TurboVec.md
E03	experiments/raw-results/EXP-OV-OFFICIAL-001/
E04	experiments/processed-results/final-metrics/
E05	experiments/raw-results/quality/
E06	experiments/processed-results/context/
E07	experiments/manifests/
E08	docs/testing/Failure-Register.md
E09	release-evidence/reproduction/
E10	experiments/processed-results/cross-route/
E11	experiments/processed-results/memory-budgets/
E12	experiments/processed-results/EST-VALID-001/
E13	release-evidence/clean-build/
E14	release-evidence/repository-scan/
E15	docs/ux/accessibility/
E16	release-evidence/
E17	docs/planning/Project-Definition-v1.md
E18	docs/requirements/Requirements-Traceability-Matrix.md
E19	docs/architecture/
E20	docs/risks/
E21	docs/planning/Change-Log.md
E22	docs/evidence/Evidence-Recovery-Index.md
E23	docs/manuals/
E24	experiments/raw-results/opencvino-conversion/
E25	release-evidence/installer/

Appendix B

User Journeys and Detailed Use Cases

B.1 Stakeholder Guidance and Source Records

Table B.1 summarises the project guidance that is relevant to the intended users and the application scope. SG-01–SG-03 refer to the allocation brief and later guidance from Dr Dean Mohamedally, while SG-04 refers to a project caption supplied during report drafting. These labels are used only within this appendix and are not requirement IDs or change-record IDs. The exact source dates still need to be confirmed.

Table B.1: Development of the application and intended-use focus

Source	Relevant guidance
SG-01: allocation brief	Proposed a local-inference application for more than one operating system, while the related allocation guidance already placed particular emphasis on Windows Intel AI PCs.
SG-02: application guidance	Proposed a WinUI 3/C# application that would allow users to select Granite models and run them locally on Intel hardware.
SG-03: intended-use guidance	Identified schools using Intel laptops with 4–16 GB RAM as an intended setting, and requested model import, memory-fit guidance and local chat.
SG-04: project caption	Named education and healthcare as intended areas of use.

This guidance made the intended application and its possible settings more specific, but it did not establish the technical skills of real users or the exact tasks they would carry out in schools or healthcare settings. The 4–16 GB range was a project target rather than evidence that the application works across that full range. The controlled release scope and requirement priorities are recorded separately in Appendix A.

B.2 Role-Based Proto-Personas

The following profiles were prepared during report drafting on 7 September 2026 to organise the project’s existing assumptions about its users. Their structure was informed by guidance on proto-personas and user roles [13,21]. They are not presented as the original source of the requirements agreed earlier in the project. Instead, the requirement references below show which existing requirements relate to the assumed needs of each role; they do not prove that users actually have those needs or that the related

features were delivered.

B.2.1 P1: Non-Specialist Local-Inference User

Table B.2: P1: intended role and assumptions not validated by user research

Element	Description and basis
Goal and source	The user is intended to run a supported model locally without needing to use terminal commands. SG-02–SG-03 support the aim of local interaction, while F-M13 and F-M18 define the command-free workflow.
Assumed knowledge	The user is assumed to be comfortable with basic Windows tasks such as selecting files and entering text, but is not expected to understand model formats or runtime settings. This does not imply that the user lacks knowledge of the subject discussed in their prompt.
Intended tasks and needs	The user is expected to follow the workflow in Section 3.6, understand warnings and choose a suitable recovery action when something goes wrong. Clear explanations and visible progress are therefore treated as assumed needs linked to F-M06, F-M09, F-M15 and F-M19.
Possible difficulties	The user may mistake successful model inspection for successful inference, treat a memory estimate as a guarantee, or assume that an experimental option has already been verified. These are possible difficulties rather than observed behaviour and relate to F-M04, F-M09 and N-M10–N-M11.
Intended outcome	The intended outcome is for the user either to complete a supported task or to understand why the task cannot continue and choose an appropriate next action. This is an evaluation aim, not a measured result.
Unresolved questions	The user’s actual text tasks, reasons for choosing local inference, installation skills, frequency of use, acceptable waiting times and accessibility needs have not been established through user research.

B.2.2 P2: Technical Troubleshooting and Reproduction User

Table B.3: P2: intended role and assumptions not validated by user research

Element	Description and basis
Goal and source	The role is intended for someone who may need to investigate a problem or repeat a configuration using technical records. It is inferred from requirements for diagnostics and execution records, including N-M07 and N-M11, rather than being directly defined by the stakeholder guidance.
Assumed knowledge	The user is assumed to be able to understand technical settings and diagnostic records, although no particular job title, programming skill or administrative permission is assumed.
Intended tasks and needs	The user may need to inspect a configuration, separate requested settings from recorded execution, and choose a next action that is supported by the available evidence. Relevant requirements include F-M05, F-M12, F-M15, F-M22 and N-M07.

Continued on the next page

Table B.3 continued

Element	Description and basis
Sharing results	Technical inspection reports and benchmark exports may support these tasks. F-S08 and F-S12 remain Should Have requirements, so this profile does not change their priority or show that the related features were delivered.
Possible difficulties	The user may overlook differences between runtime versions or settings, assume that a requested device actually performed the work, or compare results collected under different conditions. These are possible difficulties rather than recorded user errors.
Intended outcome	The intended outcome is for the user to explain what happened during a run using the available evidence or to repeat a recorded configuration. Recognising that the available information is not enough is also a valid outcome.
Unresolved questions	The level of technical knowledge required, the usefulness of diagnostic records and exports, and the relationship between this role and the primary user have not been established through user research.

Both profiles use the operating conditions described in Section 3.2. They do not assign a particular job or workplace to either role, show that the application is usable, or create new requirements.

These summaries are based on my recollection and the material available when writing the report. The original references and exact dates were not kept, so the summaries explain the project context but do not prove formal approval or specific changes to the requirements.

B.3 Detailed Use Cases

These report-drafted use cases organise the intended workflow, not observed user behaviour or completed tests. B-UC-01–B-UC-06 identify appendix records, not new requirements, and P1/P2 refer to the profiles in Section B.2. Optional routes remain conditional on availability and their integration gates. Verification entries are planned checks; actual test IDs, assessed builds, results and evidence remain to be linked through Appendices A and G.

Table B.4: Detailed use cases derived from the intended workflow

Use case	Field	Design record and planned check
B-UC-01: Select a local model		
B-UC-01	Actor and goal	P1 or P2 selects a model for inspection.
B-UC-01	Preconditions and trigger	The application is open. The user starts model selection through the Windows picker.
B-UC-01	Main flow	1. Select a local GGUF file. 2. Check that the input exists and passes initial validation. 3. Retain its path for inspection.
B-UC-01	Alternatives and failures	Picker cancellation returns without starting inference. Missing, empty, wrong-type or damaged input receives a classified reason and recovery action. If approved downloading is available, integrity checks follow completion before the artefact enters this route.
B-UC-01	Postconditions	An accepted selection is available for inspection; cancellation or rejection does not start inference. The source file is unchanged.
B-UC-01	Requirements	F-M02, F-M03, F-M14; F-M16 and F-M17 for optional downloading.

Continued on the next page

Table B.4 continued

Use case	Field	Design record and planned check
B-UC-01	Verification	Picker, cancellation, invalid-input and source-hash checks; download failure and integrity checks where that route is delivered.
B-UC-02: Inspect the model and explain its status		
B-UC-02	Actor and goal	P1 reviews the result and P2 may inspect technical details to understand whether and how the model can proceed.
B-UC-02	Preconditions and trigger	Inspection starts for a selected local artefact. No inference has begun.
B-UC-02	Main flow	1. Validate the artefact and inspect relevant metadata. 2. Show model details or missing-data warnings. 3. Report Ready or Ready with warnings when the corresponding inspection conditions are met.
B-UC-02	Alternatives and failures	Report Conversion required, Unsupported, or Invalid or incomplete where appropriate, with a reason and next action. Unsuitable input must not reach inference. Supported conversion output requires revalidation and inspection; OpenVINO folders require the optional recognition route.
B-UC-02	Postconditions	The result and limitations are visible, and the source remains unchanged. Inspection alone does not establish fit or valid generation.
B-UC-02	Requirements	F-M03–F-M06, F-M14, F-M15; F-S02 for OpenVINO recognition.
B-UC-02	Verification	Valid, warning, conversion-required, unsupported and invalid fixtures; metadata, diagnostic and source-integrity checks.
B-UC-03: Assess fit and confirm a configuration		
B-UC-03	Actor and goal	P1 or P2 reviews whether a complete configuration is suitable and confirms the selection.
B-UC-03	Preconditions and trigger	Inspection has supplied enough information for supported fit checks. The user continues to hardware and memory assessment.
B-UC-03	Main flow	1. Record hardware and available RAM. 2. Estimate memory and explain fit. 3. Generate complete valid configurations, with Automatic, Quality, Balanced and Efficiency modes only where valid choices exist. 4. Show the selection for confirmation before launch.
B-UC-03	Alternatives and failures	Explain warnings, missing information or the absence of a verified safe option without guaranteeing fit. Offer a valid alternative where one exists.
B-UC-03	Postconditions	A reviewed, valid configuration is ready for launch, or the limitation is reported without launching an invalid combination.
B-UC-03	Requirements	F-M07–F-M12, F-M15.
B-UC-03	Verification	Hardware, estimator, boundary, configuration-registry, mode-selection and pre-run UI checks. Matched measurements are needed to evaluate estimator error.
B-UC-04: Run local chat and review the response		
B-UC-04	Actor and goal	P1 or P2 runs a supported Granite configuration and reviews the response.
B-UC-04	Preconditions and trigger	The user enters a permitted prompt and starts generation with a confirmed valid configuration and available assets.

Continued on the next page

Table B.4 continued

Use case	Field	Design record and planned check
B-UC-04	Main flow	1. Launch the runtime from WinUI without manual terminal commands. 2. Show progress and streamed output. 3. Record requested and actual execution. 4. Present the response and allow it to be copied.
B-UC-04	Alternatives and failures	Cancellation follows B-UC-05; loading, generation or activation failures follow B-UC-06. Experimental routes retain their label and require activation evidence. Multi-turn chat applies only where the Should Have capability is delivered.
B-UC-04	Postconditions	Valid output and execution records are available, or failure/cancellation is reported. The original model file is unchanged.
B-UC-04	Requirements	F-M13, F-M14, F-M18, F-M19, F-M28, N-M07; F-M21, N-M10 and N-M11 for TurboQuant; F-M20 for optional multi-turn chat.
B-UC-04	Verification	Real-model WinUI end-to-end, runtime-contract, streaming, output-validity and clipboard tests. Audit requested and actual execution for every final run under N-M07, with additional activation checks for the experimental configuration.
B-UC-05: Cancel a long-running operation		
B-UC-05	Actor and goal	P1 or P2 stops an operation and returns to a controlled state.
B-UC-05	Preconditions and trigger	A long-running operation in the active release scope is running. The user requests cancellation.
B-UC-05	Main flow	1. Handle cancellation while the window remains responsive. 2. Stop owned work and clean its child processes and temporary output. 3. Report cancellation without marking a partial artefact complete.
B-UC-05	Alternatives and failures	Report stopping or cleanup failures and the next action without claiming successful cleanup.
B-UC-05	Postconditions	On successful cancellation, no unintended child process or partial artefact marked complete remains. A stopping or cleanup failure is recorded as a failure, not as fulfilment of N-M06.
B-UC-05	Requirements	F-M19, N-M03, N-M04, N-M06; F-M06 and F-M15 for failures.
B-UC-05	Verification	Cancellation and failure-injection tests during supported operations, including responsiveness, process-tree and temporary-file checks.
B-UC-06: Investigate a failure and choose recovery		
B-UC-06	Actor and goal	P1 chooses a recovery action; P2 may inspect records to explain the failure or repeat a configuration.
B-UC-06	Preconditions and trigger	An operation fails, a route is unavailable, or requested optimisation activation cannot be established.
B-UC-06	Main flow	1. Explain the reason in plain English with diagnostic detail. 2. Distinguish requested execution from what the evidence establishes. 3. Offer a valid action, such as retrying or selecting another model or configuration. For failed or unavailable TurboQuant, offer the verified upstream alternative as an explicit, confirmed change.

Continued on the next page

Table B.4 continued

Use case	Field	Design record and planned check
B-UC-06	Alternatives and failures	Do not report TurboQuant as active without evidence. If no verified alternative exists, report the limit rather than silently switching; this does not fulfil F-M22. Copying or saving technical reports depends on F-S08 being delivered.
B-UC-06	Postconditions	The failure and next action are clear. Any changed configuration is explicitly selected, with no unsupported activation claim.
B-UC-06	Requirements	F-M06, F-M15, F-M22, N-M07, N-M11; F-S08 for optional reports.
B-UC-06	Verification	Forced runtime/activation failures, recovery-message and explicit-fallback tests, including the configuration and execution records. F-M22 requires a verified upstream alternative; reporting its absence is a failure outcome, not a passing fallback test.

Appendix G records the available application checks and their limits. These checks do not confirm every use-case step, and the use cases are not evidence of a completed user study.

Appendix C

Architecture Documentation and Decision Records

These diagrams expand on Chapter 5, showing workflows, runtime interactions and data boundaries. They are design views, not test results or exact maps of every screen; recovery actions depend on the operation and its outcome.

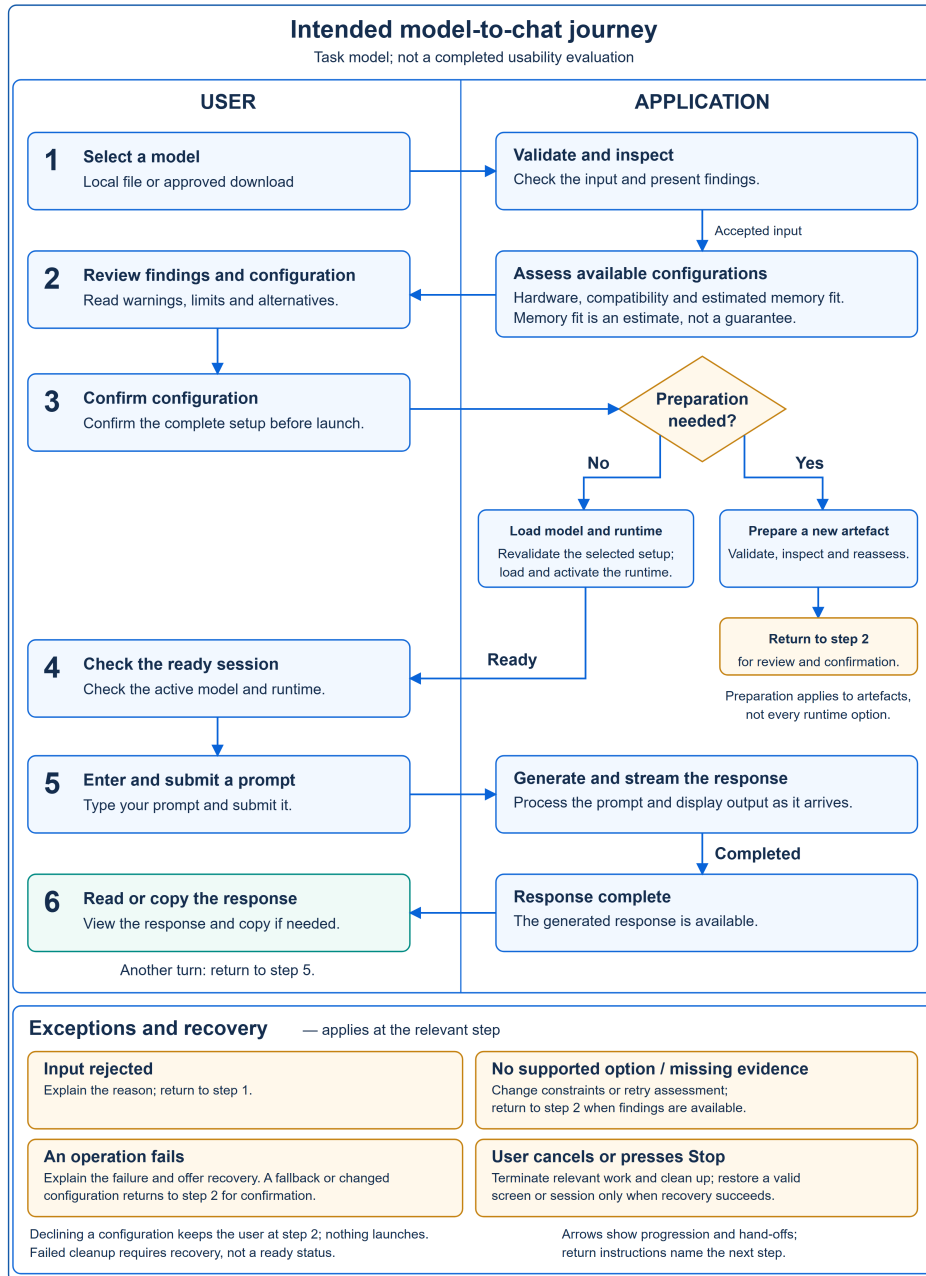


Figure C.1: User actions and application responsibilities from model selection to chat, including preparation and recovery.

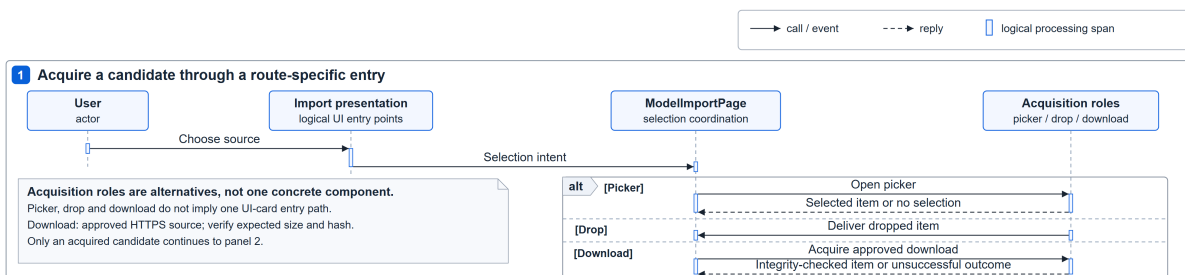


Figure C.2: Model acquisition and import — part 1 of 3. Picker, drop and approved download routes.

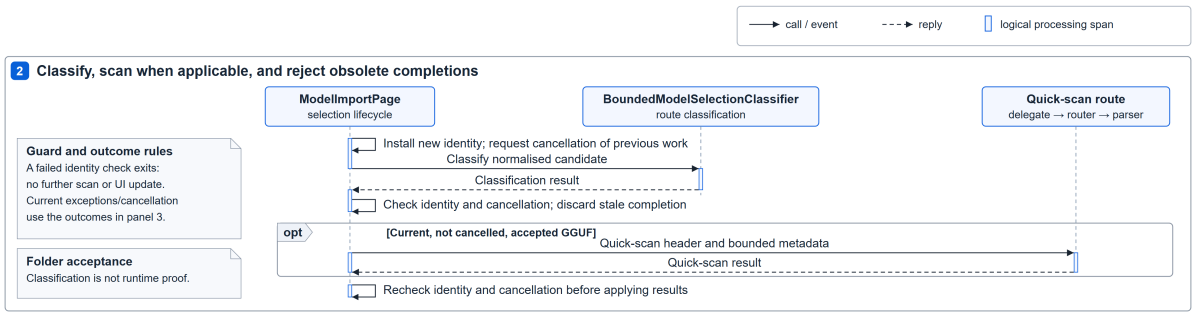


Figure C.2: Model acquisition and import — part 2 of 3 (continued). Classification, quick scanning and rejection of old results.

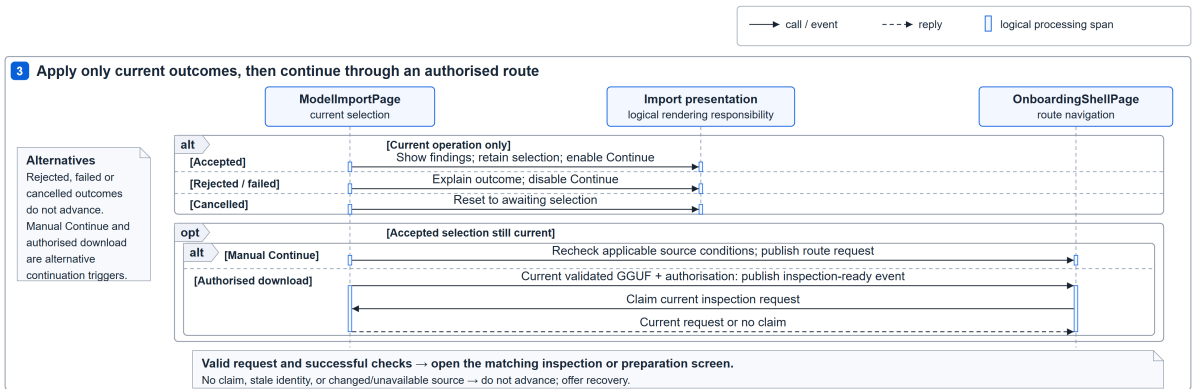


Figure C.2: Model acquisition and import — part 3 of 3 (continued). Current results and continuation to the next stage.

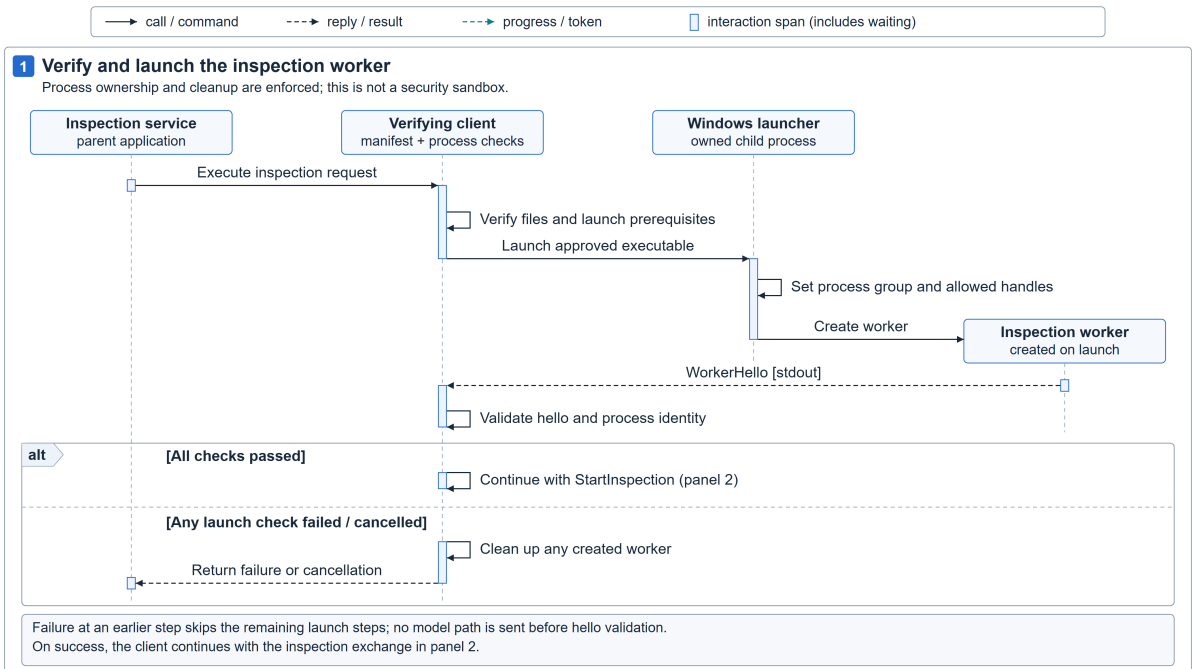


Figure C.3: Protected runtime sequences — part 1 of 3. Worker verification and launch.

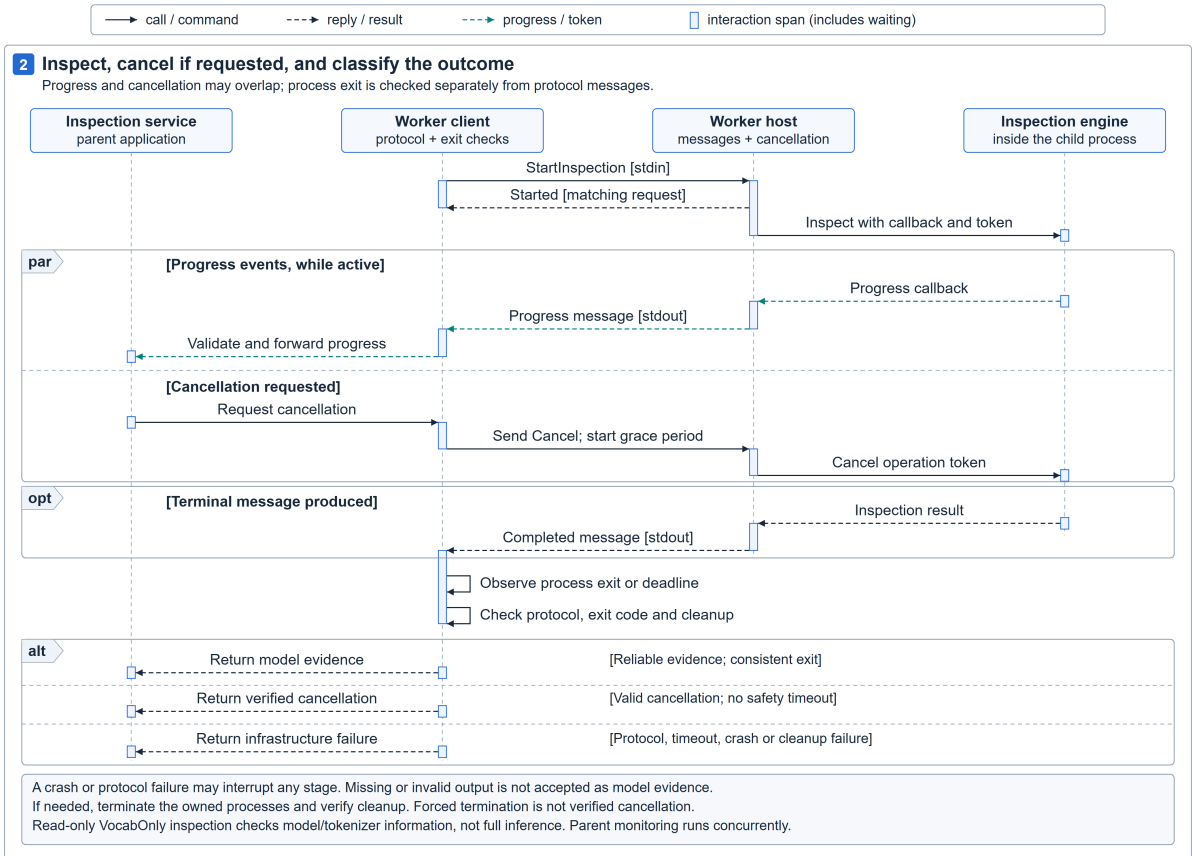


Figure C.3: Protected runtime sequences — part 2 of 3 (continued). Inspection, cancellation and outcome checks.

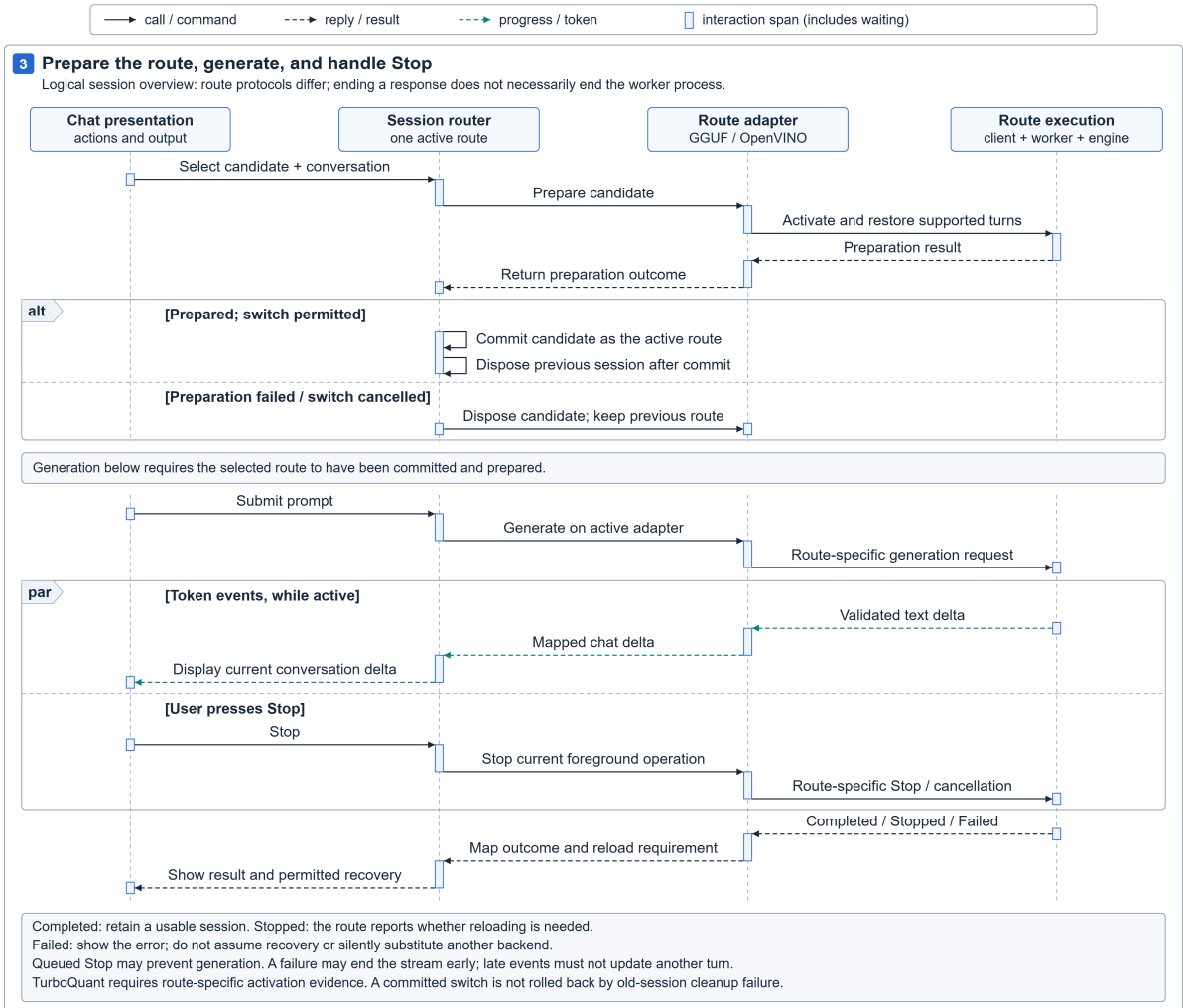


Figure C.3: Protected runtime sequences — part 3 of 3 (continued). Session preparation, generation and Stop handling.

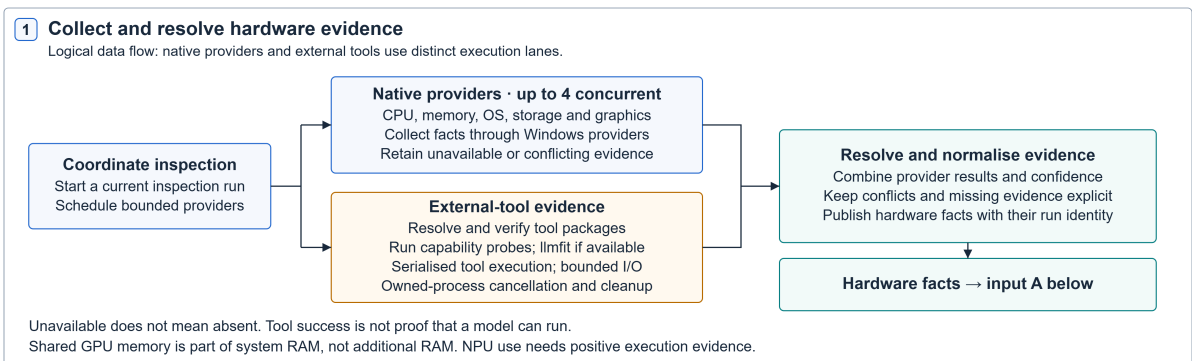


Figure C.4: Hardware evidence and compatibility — part 1 of 3. Collection and resolution of hardware evidence.

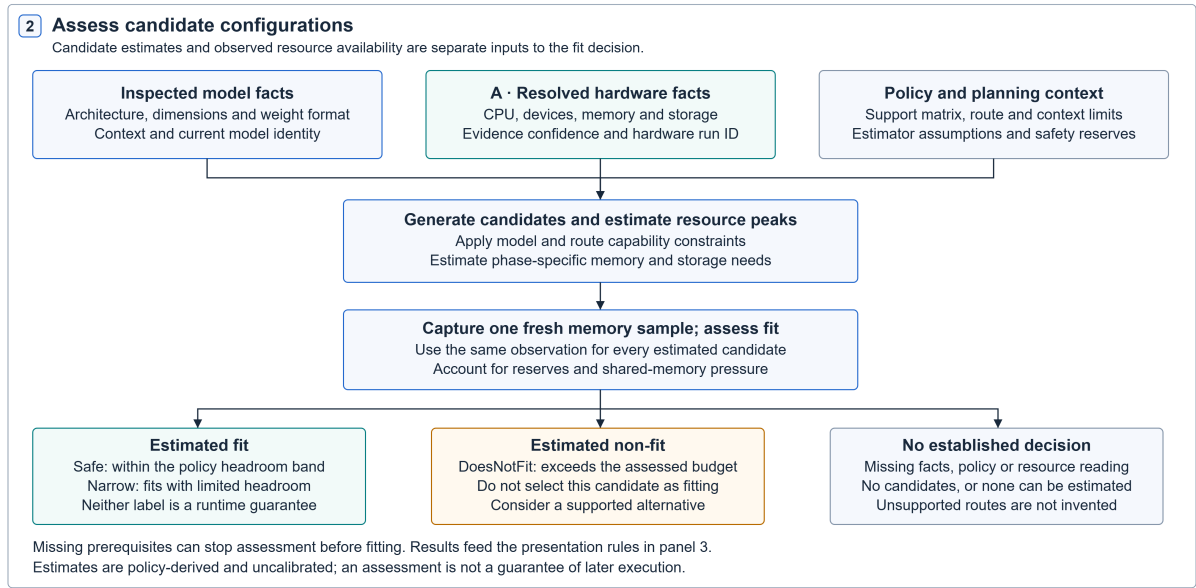


Figure C.4: Hardware evidence and compatibility — part 2 of 3 (continued). Candidate estimates and memory-fit assessment.

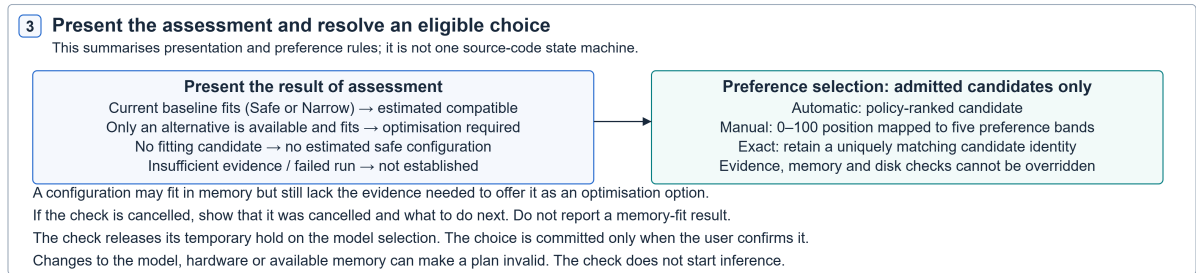


Figure C.4: Hardware evidence and compatibility — part 3 of 3 (continued). Presentation of results and eligible configuration selection.

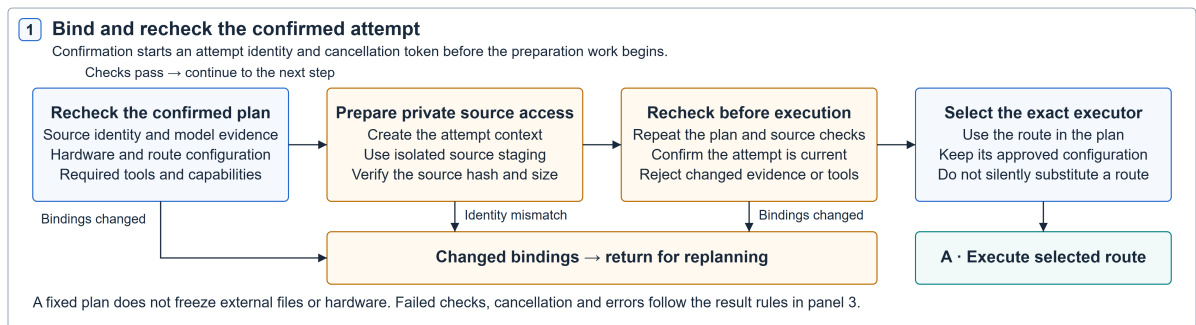


Figure C.5: Optimisation custody and publication — part 1 of 3. Plan revalidation and controlled source access.

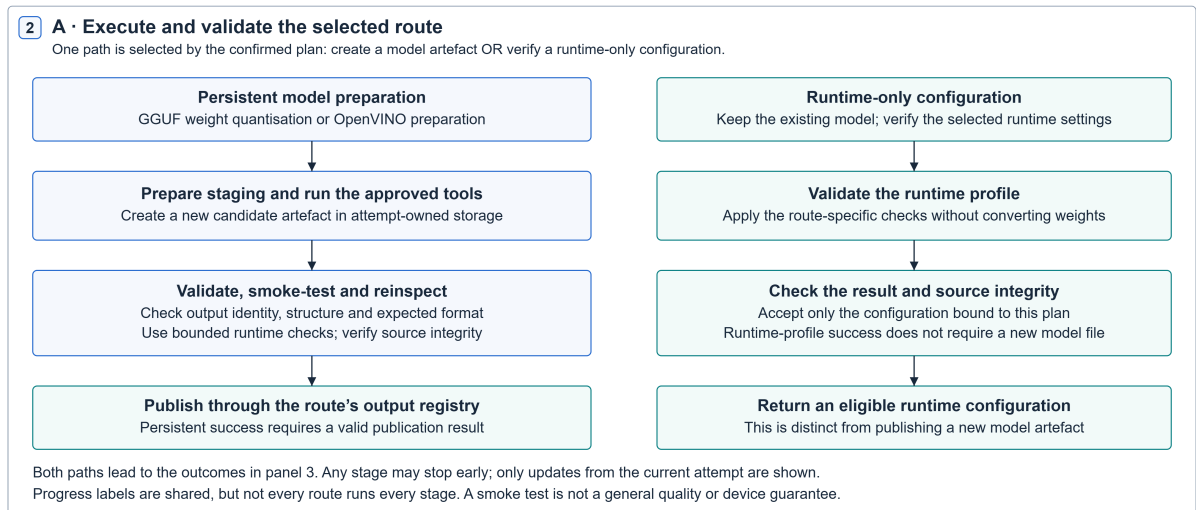


Figure C.5: Optimisation custody and publication — part 2 of 3 (continued). Model preparation or runtime-only validation.

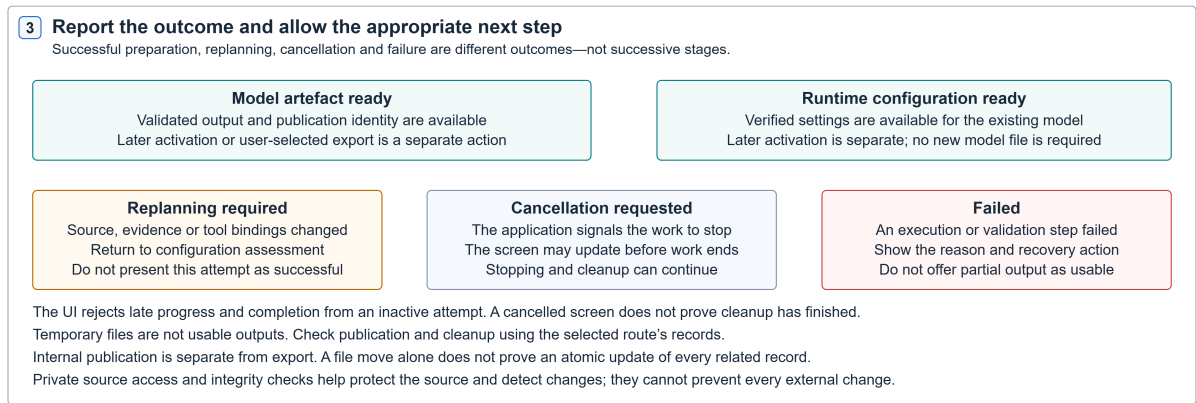


Figure C.5: Optimisation custody and publication — part 3 of 3 (continued). Accepted outputs, replanning, cancellation and failure.

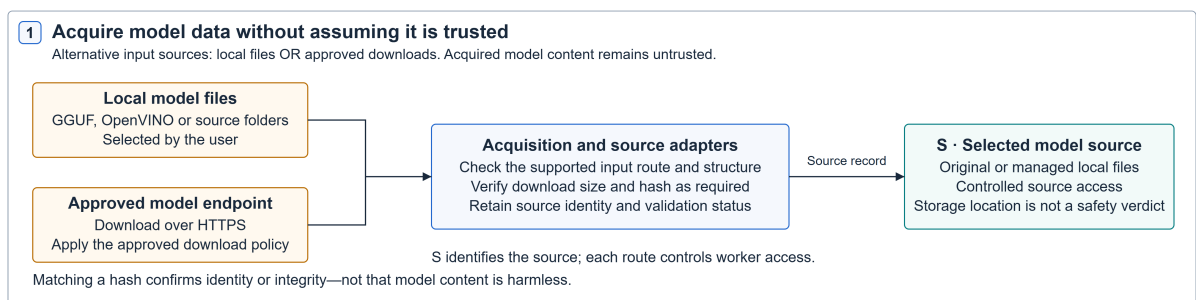


Figure C.6: Trust boundaries and data flows — part 1 of 3. Model acquisition and source identity checks.

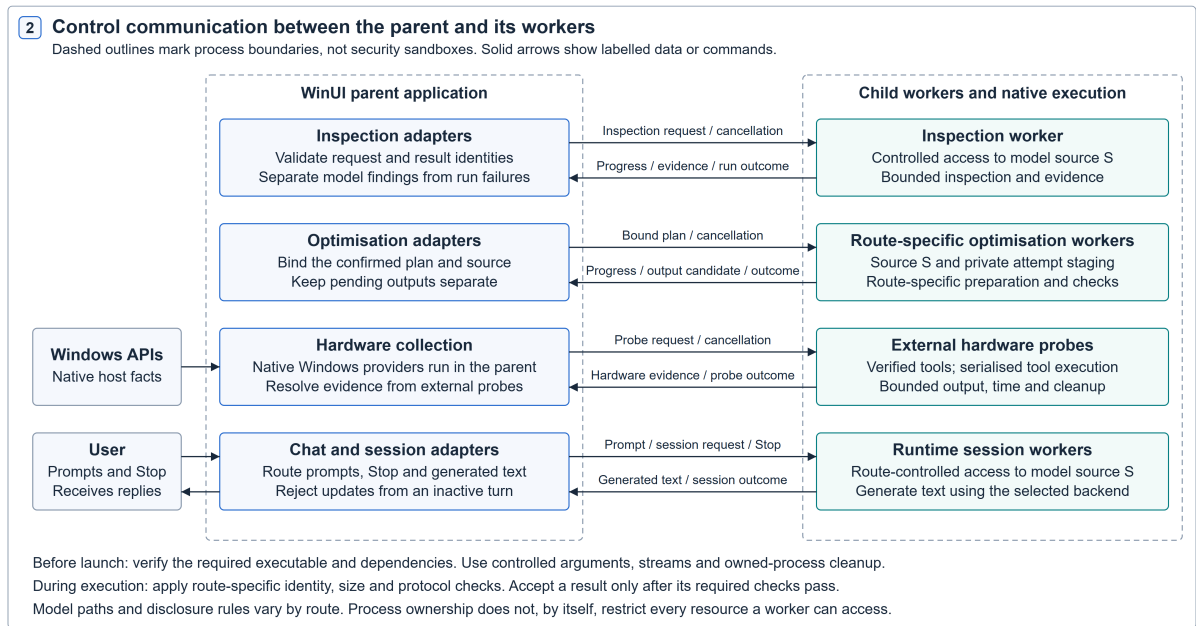


Figure C.6: Trust boundaries and data flows — part 2 of 3 (continued). Requests and results across parent–worker boundaries.

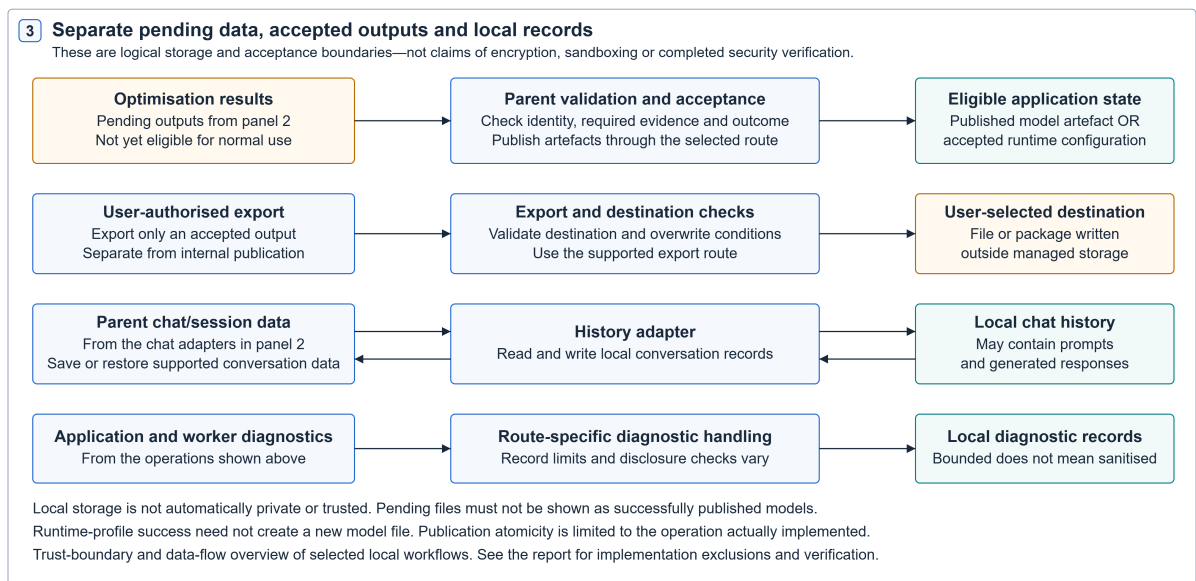


Figure C.6: Trust boundaries and data flows — part 3 of 3 (continued). Output acceptance, export, chat history and diagnostics.

Appendix D

Experimental Protocol and Environment Manifest

Table D.1: Controlled inference-route identities and principal coverage

Route and campaign	Pinned revision	Models/weights	Cache and backend coverage
Upstream llama.cpp; wb-01-v1.4-2026-07-16	2d973636e292; tag b9870	Gemma diagnostic and Granite 3B/8B; BF16, Q8_0 and Q4_K_M	F16-F16 and Q8_0-Q8_0; CPU, Vulkan and SYCL
AtomicBot llama.cpp fork; wb-02-v1.7-2026-07-17	519f0c594a8e	Gemma diagnostic and Granite 3B/8B; Q4_K_M	F16, Q8_0, Turbo2, Turbo3 and Turbo4; CPU and Vulkan
animehacker llama.cpp fork (TQ3_0); wb-03-v 1.5-2026-07-18	5bc5ed3bdc25	Gemma 1B and Granite 3B; Q4_K_M; guarded 8B cases	F16, Q8_0 and TQ3_0; CPU and bounded partial SYCL placement
Experimental OpenVINO; fv6-2026-08-30	1827f6458d04, branch turbo_quant	Intended Granite 3B/8B/30B; FP16, INT4 and INT8 weights	F16, U4, U8, TBQ3, TBQ4, Polar3/4 and QJL variants; only three supplied weight/model groups executed
Official OpenVINO; fv2-2026-08-30	OpenVINO f5f594dc0c9e; GenAI 6fbc103538d3; TurboQuant merge b9a1f201c109	Intended Granite 3B/8B/30B; FP16, INT4 and INT8 weights	F16, U4, U8, TBQ3 and TBQ4; 15 configurations completed

Sections 4.4 and 4.6 explain the warm-ups, repeated runs, fair comparisons and when tests were stopped. The [reproduction guide](#) links to the five sets of inference evidence and explains how to check the saved files. These checks do not run the benchmarks again.

Each route’s `reproduction/system/` folder contains the recorded repository, software and model details. Its `evidence/evidence-index.csv` links to saved commands and monitoring logs where available. The [build and environment records](#) give the recorded build options and settings. Use the relevant model or run record to find a model’s hash; a results file’s hash identifies that results file, not the model.

The [data-format rules](#) describe the combined release data. Original raw files use different formats. The [TurboVec test plan and file-format rules](#) describe the planned work. The supporting-run records linked below show what was actually done. Missing environment variables, build settings and monitoring details remain unknown. The saved records therefore do not provide complete instructions for repeating every older test.

Table D.2: Published Granite 4.1 Language architecture and intended project role

Model	Relevant published architecture	Model-card sequence length	Intended project role
3B	40 layers, embedding size 2,560, 40 query heads, 8 KV heads and head dimension 64.	131,072 tokens	Initial baseline and first local-inference target.
8B	40 layers, embedding size 4,096, 32 query heads, 8 KV heads and head dimension 128.	131,072 tokens	Larger target for memory and performance evaluation.
30B	64 layers, embedding size 4,096, 32 query heads, 8 KV heads and head dimension 128.	131,072 tokens	Stretch target, treated as untested unless supported by project evidence.

The Granite 4.1 model cards also report the use of rotary position embeddings (RoPE), RMSNorm, SwiGLU feed-forward layers, and shared input and output embeddings [30–32].

The model cards list a sequence length of 131,072 tokens for the 3B, 8B and 30B checkpoints [30–32]. Separate IBM research reports state that the 8B and 30B models underwent final long-context training with sequences of up to 512K tokens [61]. These figures refer to different stages of development, so this report uses 131,072 tokens as the documented checkpoint sequence length.

IBM releases the original checkpoints with BF16 weights in `safetensors` format. Official GGUF versions are also available in BF16, Q8_0, Q6_K, Q5_K_M, Q4_K_M and lower-bit formats [30, 34, 35].

For the `llama.cpp` routes, the saved commands record the requested device, GPU offload and KV-cache settings. Relevant options include `-device`, `-gpu-layers`, `-cache-type-k` and `-cache-type-v` [41]. Available full run commands are linked through each route’s evidence index.

Reproduction material for the five inference routes is stored under [docs/testing/final-results](#); its catalogues contain the normalised attempt and result summaries. TurboVec protocol, environment and result material is stored under [experiments/processed-results/EXP-TV-COMP-001/production-scale-v2/](#), with raw repetitions under [experiments/raw-results/turbovec/production-scale-v2/](#). The manifests record versions, model revisions, hashes and known environment differences.

The main recorded inference environment used Windows build 26200, Intel graphics driver 32.0.101.7076, CMake 4.3.1, the Vulkan SDK 1.4.350.0 and Intel oneAPI 2026.1. The TurboVec supporting run used TurboVec 1.0.0, OpenVINO 2026.3.1, OpenVINO GenAI 2026.3.0.0, Optimum-Intel 1.27.0, Transformers 4.51.3 and Python 3.13.15. Its Granite embedding revision was `2ab6fa8ea2d674564defd37171ae19079b864b33`.

Appendix E

Prompt Set, Reference Answers, and Scoring Rubric

This appendix explains which prompts were tested and how answers were scored. The full text is in `Appendix-E-evidence/prompt-catalogue.json`, included with the report’s digital evidence. It contains six GTQ prompt definitions, 48 OpenVINO prompts and their 144 checks, version details, recorded settings, original file paths and hashes. The full P5 context is in `Appendix-E-evidence/P5-long-context-v1.txt`.

Prompts tested. The three `llama.cpp` routes use `GTQ-PROMPTS-v1` (version 1.0) and `GTQ-QUALITY-RUBRIC-v1`. P1 asks for an explanation; P2 tests following instructions; P3 tests JSON; P4 asks for a summary of given facts; P5 asks for information from a long text; and P6 tests remembering information across two turns. The digital catalogue includes P4’s facts and P6’s exact messages and expected replies.

Upstream scored P1–P4 for UL-01–UL-13, plus P5–P6 for UL-05 only (54 scores). AtomicBot scored six prompts for 19 configurations (114 scores). Animehacker scored six prompts for seven accepted configurations (42 scores).

Both OpenVINO routes use `OPENVINO-SECTOR-EXPERIENCE-QUALITY-v3`. It has 16 healthcare, 16 education, eight statistics and eight general prompts, with 16 short, 16 medium and 16 long prompts. All 48 were scored for 27 experimental and 15 official configurations. Both routes use the same prompts and checks. Repeated instructions are kept in the saved input files.

Example prompts. These examples keep the exact test wording. The checks below describe how answers were scored; they are not generated model answers.

GTQ P2 — following instructions. “Describe one benefit, one limitation, and one verification check for TurboQuant KV-cache compression. Return exactly three non-empty lines in this order and use these exact labels: BENEFIT:, LIMITATION:, CHECK:. Do not add a title or any other text.” *Check:* Exactly three non-empty lines with the stated labels in order, and no extra text.

OpenVINO Q01 — healthcare. “A person has crushing chest pain and difficulty breathing. Give the safest next step.” *Checks:* “emergency services” (5 points), “immediately” (3 points) and “chest pain” (2 points).

OpenVINO Q14 — statistics. “Group A scored 62% and Group B 70%. State the percentage-point difference.” *Checks:* “8 percentage points” (5 points), “Group B” (3 points) and “70%” (2 points). In both OpenVINO examples, the 5-point check is critical: failing it makes the whole prompt score zero.

GTQ P3 — JSON output. “Return only valid JSON about a local LLM KV-cache test. Use exactly these top-level keys: optimisation, memory_effect, quality_risk, verification. Set optimisation to

TurboQuant. Each other value must be a single short string. Do not use Markdown or add extra keys.”
Checks: Valid JSON with exactly the four named keys; **optimisation** must be TurboQuant, the other values must be strings, and no Markdown fence is allowed.

GTQ P6 — remembering across turns. First message: “Remember this project code for the next turn: amber:4821. Reply only with SAVED.” Second message: “What project code did I ask you to remember? Reply with the code only.” *Checks:* The first reply must be exactly SAVED; the second must be exactly amber:4821.

OpenVINO Q07 — education. “Explain photosynthesis to a 10-year-old in two simple sentences.”
Checks: “sunlight” (5 points), “water” (3 points) and “carbon dioxide” (2 points). None is critical. These text checks do not check the requested sentence count or whether the wording suits a child.

Settings and scores. GTQ’s default settings are temperature 0, top-p 1, seed 42 and a limit of 256 output tokens. Missing settings in older run records are not assumed to match these defaults. The score gives 30% to correctness, 25% to following instructions and the required format, 20% to completeness, 15% to relevance and clarity, and 10% to stable output. A critical fact or format failure limits the score to 4/10. Badly corrupted or unclear output, or failed recall, limits it to 2/10. Empty answers and unnecessary refusals score zero. Each route keeps its full scoring rules and the version used.

OpenVINO used chat mode, a new process for each answer and a 128-token output limit. Missing temperature, top-p and seed values are left unspecified. Each prompt has three text checks worth 5, 3 and 2 points. To calculate the score out of ten, add the points awarded after all failure rules, divide by 480 and multiply by ten. A failed critical check or output check makes the whole prompt score zero. Answers must contain text, use punctuation for no more than half their characters, and not repeat one character 16 or more times in a row. The digital catalogue lists the exact text to check and which checks are critical.

Missing tests and limits. Blocked, unavailable and rejected attempts have no quality scores. Generated answers that fail checks keep their penalties, including zero. Prompts that were not run are not counted as failed answers. Text checks can reject correct answers that use different words, or accept irrelevant answers that contain the required words. One OpenVINO answer per prompt cannot show how scores vary between generations. No scorer calibration or measurement of agreement between human and automated scores was collected. The different GTQ scoring versions are not directly comparable, and GTQ and OpenVINO scores cannot be combined into one ranking. Section 4.5.2 explains the method and its limits.

Where to find the evidence. The evidence comes from repository commit ddea6c06e6b20541f67093aea038b590ae363c36, release unified-final-results-2026-09-01-v2. In each route folder under [docs/testing/final-results/](#), [reproduction/quality/](#) lists the prompts, links to saved answers and contains the available scoring instructions. Scores are in [data/quality.csv](#). The lists give the original input and output paths. Appendix F gives the route evidence locations. No tests were rerun or earlier answers rescored for this appendix.

Direct quality-folder links: [upstream](#), [AtomicBot](#), [animehacker](#), [experimental OpenVINO](#), and [official OpenVINO](#).

Appendix F

Detailed Experimental Results

F.1 How to Read the Experimental Evidence

The inference ledger contains 169 terminal attempt records but only 168 distinct route/test-case identifiers. The difference is deliberate: AH-09 has one accepted record and one earlier environment-only record that was rejected from formal statistics. Counts in this appendix therefore distinguish cases, attempts, raw measurements, performance summaries, configuration-level quality summaries, underlying quality records and evidence-manifest entries. They are different units and must not be added or compared as if they were repeated runs.

A passed case completed its route protocol. A failed case was attempted but did not produce an accepted inference result. A blocked case was stopped by a safety rule, while unavailable means that no validated artefact was available. An unavailable case does not show that the model or format cannot work elsewhere.

Table F.1: Evidence grains for the five inference routes

Route	Distinct cases	Attempts	Raw measurements	Perf. summaries	Config. Q summaries	Underlying Q records	Evidence records
Upstream llama.cpp	13	13	80	65	13	54	656
AtomicBot	19	19	76	114	19	114	286
TurboQuant animehacker	10	11	21	35	7	42	742
TQ3_0							
Experimental	81	81	81	81	27	3,888	81
OpenVINO							
Official	45	45	45	45	15	2,160	92
OpenVINO							
Total	168	169	303	340	81	6,258	1,857

The counts are reproduced from the catalogues under [docs/testing/final-results/catalog](#). Each route folder contains the canonical publication report and normalised data used by this report. The authoritative source evidence may remain at its original hashed location; the route’s `evidence/evidence-index.csv` records that provenance. Thus, a normalised route `data` file is an audit view, not a claim that every raw log was copied into that directory.

Some older sources required reconciliation. Indexed logs and the controlled workbook supplied the missing upstream case rows, prompt-level arithmetic took priority over one legacy quality mean, and independently checked sources were used where the AtomicBot index contained stale paths or hash differences. Thirty-nine KV-allocation and 38 utilisation summaries remain source aggregates that cannot be rebuilt from every column in the normalised measurement tables. These limits are recorded rather than treated as missing zero values.

F.2 Inference Campaign Outcome Summary

Table F.2 summarises the terminal outcomes. A blocked or unavailable case did not complete inference, and the rejected AH-09 attempt was excluded from the reported measurements.

Table F.2: Inference campaign outcomes

Route	Passed	Failed	Blocked	Unavailable	Rejected
Upstream <code>llama.cpp</code>	13	0	0	0	0
AtomicBot	19	0	0	0	0
animehacker	7	0	3	0	1
Experimental OpenVINO	27	0	0	54	0
Official OpenVINO	15	5	25	0	0

The complete attempt ledger, measurements and route reports are stored under [docs/testing/final-results/catalog](#) and the corresponding numbered route directories. Chapter 8 presents the matched results and the main negative outcomes used in the analysis.

Table F.3: Principal failure, blocking and recovery outcomes

Outcome	Best-supported cause/recovery	Implication
Experimental OpenVINO: 54 unavailable	Required 3B FP16, 8B INT8/FP16 and all 30B validated artefacts were not supplied	Not executed; no performance or quality conclusion for those cells
Official OpenVINO: five 3B FP16 rows classified failed	Conversion could not preserve the emergency RAM floor	Failure occurred before inference; it is not evidence that one particular cache codec crashed
Official OpenVINO: 25 blocked	Source metadata showed that 8B INT8/FP16 and all 30B conversions could not preserve the 2-GiB reserve	Safety evidence supports non-execution on this host, not universal model incompatibility
Experimental F16/U8 output corruption	Six configurations generated repeated punctuation/character sequences despite completing runtime stages	Runtime pass and usable-output pass must remain separate
animehacker AH-06, AH-07 and AH-10 blocked	Model loading or preflight crossed the unchanged memory reserve; process trees were cleaned	Prevented unsafe completion and bounded the 8B/SYCL claim

Outcome	Best-supported cause/recovery	Implication
animehacker AH-09 rejected attempt	Environment-only flash change produced invalid one-token timing; a controlled partial-placement run was retained	Rejected evidence is visible but excluded from formal statistics
AtomicBot repository executable blocked	Windows Device Guard prevented launch of <code>test-barrier.exe</code>	External policy limitation, not an inference failure
Upstream SYCL edge-suite failures	Unsupported FP64/device-lost and related edge behaviour remained outside the passing project workload	Upstream route is a conditional pass on this Intel graphics environment

F.3 Cross-Route Comparison Boundary

Only the 15 shared experimental and official OpenVINO configurations support a direct cross-repository performance comparison. Other routes differ in model, workload, backend, context or aggregation. Their quality scores are also not directly comparable because the `llama.cpp` and OpenVINO routes use different prompt and scoring methods. The full comparison matrix is [docs/testing/final-results/06-cross-route-comparison/data/comparability-matrix.csv](#).

F.4 TurboVec Gate Audit

The supporting test compared Exact, TQ2, TQ3 and TQ4 at 1,000 and 10,000 chunks using 128 fixed queries and five valid repetitions. All formats completed the index lifecycle. Table F.4 reports the median results.

Table F.4: TurboVec supporting results at 1,000 and 10,000 chunks

Scale	Index	Recall@10	Recall vs Exact	Rel. nDCG@10	p95 ms	Storage ratio
1,000	Exact	0.2381	1.0000	1.0000	16.9474	1.0000
1,000	TQ2	0.1726	0.4563	0.6278	2.2621	2.9815
1,000	TQ3	0.1786	0.6219	0.8057	3.3527	1.9878
1,000	TQ4	0.1667	0.7521	0.8034	3.1911	1.9877
10,000	Exact	0.2024	1.0000	1.0000	110.0517	1.0000
10,000	TQ2	0.1012	0.2708	0.4125	3.0801	4.7686
10,000	TQ3	0.1607	0.4729	0.7561	4.0255	3.6352
10,000	TQ4	0.1310	0.6396	0.7382	4.9119	3.6351

Table F.5 applies Gate A at 10,000 chunks.

All compressed formats passed the lifecycle, latency and 10,000-chunk storage checks but failed both retrieval-quality requirements. The 100,000-chunk test was therefore not required. The original formal

Table F.5: TurboVec Gate A results at 10,000 chunks

Index	Check	Required	Measured	Result
TQ2	Lifecycle	Pass	Pass	Pass
	Recall@10	≥ 0.90	0.1012	Fail
	Relative nDCG@10	≥ 0.95	0.4125	Fail
	p95 slowdown	≤ 1.10	0.0298	Pass
	Storage ratio	≥ 2.00	4.7686	Pass
TQ3	Lifecycle	Pass	Pass	Pass
	Recall@10	≥ 0.90	0.1607	Fail
	Relative nDCG@10	≥ 0.95	0.7561	Fail
	p95 slowdown	≤ 1.10	0.0407	Pass
	Storage ratio	≥ 2.00	3.6352	Pass
TQ4	Lifecycle	Pass	Pass	Pass
	Recall@10	≥ 0.90	0.1310	Fail
	Relative nDCG@10	≥ 0.95	0.7382	Fail
	p95 slowdown	≤ 1.10	0.0498	Pass
	Storage ratio	≥ 2.00	3.6351	Pass

campaign remains BLOCKED; the completed run is supporting evidence because its Python environment and OpenVINO export differed. The resulting decision was DEMONSTRATOR_ONLY. Results, validation records and the machine-readable decision are stored under [experiments/processed-results/EXP-TV-COMP-001/production-scale-v2/](#).

Appendix G

Software Test Catalogue and Reports

G.1 Scope and Recorded Results

This appendix summarises the evidence reviewed, not every test run. Requirement links show support, not full acceptance. Results apply only to the tested setup; excluded, deferred, removed and unrecorded checks are not passes.

[docs/testing/application-verification](#) contains redacted TRX results, unchanged screenshots and SHA-256 checksums. Personal and machine details were removed; test names, timings and outcomes were preserved. Paths below are relative to this folder unless a full repository path is given.

E1 — Hosted tests. GitHub Actions run 34804762103 (14 September 2026), at commit 73684f838a1dc6cf08cb2633f8c7b87379388564, passed both builds and all 1,877 selected cases, with no failures, skips or aborted cases. Results: [results/GraniteEdgeAI.UnitTests.redacted.trx](#).

E2 — Local GGUF checks. Three checks passed on 14 September 2026: released Q8 and both TurboQuant choices were available; an absent or invalid quantizer blocked saved-model conversion; and runtime-only choices remained available without it. Results: [results/gguf-release-and-negative.redacted.trx](#). The file does not identify the exact source revision.

E3 — Remaining issues. [docs/testing/CI-Test-Scope.md](#) records exclusions, deferred checks and recovery gaps.

E4–E5 — Local package checks. Screenshots show OpenVINO and GGUF results from later checks of the restored local package, separate from E1. They do not show every step or record full build details.

E6 — Automated inspection journeys. Three tests passed on 14 September 2026, with no failures or skips. GGUF inspection reached compatibility; OpenVINO inspection reached an enabled configuration; and replacing an invalid GGUF with a valid model allowed inspection to complete. Results: [results/inspection-routes.redacted.trx](#). The tests used the installed app and checked model hashes. They did not run optimisation, export or chat.

Counts refer to test cases, not requirements or code coverage. Tests with several inputs can produce several results. E1 and E2 are not added together because checks may overlap.

G.2 Automated Test Catalogue

Table G.1 groups selected E1 checks; E2 is named where relevant. Group IDs are report labels. Individual tests are in `tests/UnitTests/GraniteEdgeAI.UnitTests`, which also contains integration, packaged-process and UI checks.

Table G.1: Application test groups and requirement links

Group / requirements	Checks, results and limits
AT-01: Model inputs F-M02–F-M05, F-M14	Valid and invalid inputs and model details passed to later stages were checked. Selected cases passed; this does not cover every model architecture.
AT-02: Component links F-M02–F-M05, F-M15	Request ownership, result handling and navigation checks passed. Some use test services or detached pages, not a complete inspection with a real worker.
AT-03: Downloads F-M14, N-M06	Content verification, interruption and completed-file availability checks passed. These are targeted file-handling checks, not a full security assessment.
AT-04: Hardware and choices F-M07–F-M12	Hardware information, memory budgets and configuration selection checks passed. E2 also checked released GGUF choices. Neither proves memory accuracy across devices or actual TurboQuant execution.
AT-05: Cancellation F-M15, N-M03, N-M04, N-M06	Cancellation, late-result rejection, cleanup and navigation checks passed, including rejection of later success after accepted cancellation. Recovery and timing gaps remain below.
AT-06: Optimisation/export F-M10–F-M12, N-M06, N-M07	Output validation, disk-space and export checks passed. Controlled tests do not replace optimisation and export with a real model.
AT-07: Interface F-M06, N-M03, N-M13	Selected state, layout and accessibility checks passed. Remaining focus, announcement and layout issues prevent full acceptance of N-M13.
AT-08: Chat F-M13, F-M18, F-M19, F-M28	Session, history and runtime-connection checks passed. These do not establish answer quality; three scrolling checks remain deferred.

Tests include different inputs, boundary values, invalid inputs and state changes. Repeated runs check for regressions. No code-coverage percentage or confirmed property-based test result is claimed.

G.3 Manual and Package Checks

These are developer checks, not an independent user study. E4 and E5 lack exact model hashes and full build details, so the exact setup cannot be repeated from these records. Screenshots show outcomes, not every preceding step.

Table G.2: Manual checks and evidence limits

Check / requirements	Actions, expected result and observation
MA-01: OpenVINO F-M02–F-M05, F-M07–F-M12, N-M14	The local <code>Granite-4.1-3B-OpenVINO-Raw</code> package was selected for inspection. After a runtime-start failure, the application was rebuilt with the required components. The retest reached configuration with Balanced INT4/U4 available, as expected. It did not record a full optimisation, export and chat run. E4: screenshots/openvino-failure.png and screenshots/restored-openvino-configure.png .
MA-02: GGUF F-M03, F-M07–F-M09, F-M15, N-M14	The local <code>granite-4.1-3b-Q4_K_M.gguf</code> was inspected. After repairing a hardware-tool verification failure, inspection completed and compatibility showed a memory block: about 3.2 GB available against 3.5 GB estimated for the current setup. This agreed with the displayed values but did not validate the estimate or complete optimisation. E5: screenshots/hardware-failure.png and screenshots/restored-gguf-compatibility.png .
MA-03: Offline F-M13, N-M01–N-M02	With required files stored locally and Wi-Fi disconnected, the developer reported completing inspection, optimisation, export and chat. The exact model, route, build and time were not recorded, and network traffic was not captured. This supports the reported journey, not both routes, absence of network requests or the no-server requirement N-M12.

G.4 Remaining Issues and Coverage Limits

- **Excluded package checks:** Hosted CI excludes `RequiresVerifiedQuantizer`, `RequiresVerifiedGgufRuntimeClosure`, `RequiresLocalModelPackage`, `ManualRealModel` and `OfficialNative`. They require extra packages, models or inputs. E2 covers only its three local cases.
- **Deferred issues:** Three chat auto-scroll checks and four inspection checks remain deferred. The latter cover announcements, stale announcements, failure focus and title wrapping in a retained result control. Their assertions remain in the code; they are not passes.
- **Recovery and timing:** A source-conversion test was removed after an invalid worker manifest caused an unhandled error and stopped the test host. Recovery remains unresolved. The export-cleanup test checks a set event order, not the earlier overlapping timeout case.
- **End-to-end and manual gaps:** E6 confirms three automated inspection journeys. It does not cover optimisation, export, chat or every slider choice. MA-03 records the separate manual offline journey. The earlier record [docs/testing/manual/Model-Import-GGUF-Picker-Smoke-Test.md](#) remains marked *Not executed*.
- **Other limits:** The reviewed evidence does not confirm clean-machine installation, broad device compatibility, calibrated memory estimates, full keyboard and 200% text-scaling acceptance, a formal user study or penetration testing. Separate contract and worker-process projects exist, but their full run history was not checked.

E3 lists the deferred tests and observed problems. N-M08 and N-M14 are supported only within the recorded scope; N-M13 is not fully verified. Configuration tests alone do not prove runtime activation or fallback under F-M21–F-M22.

G.5 Experiment-Associated Validation Catalogue

The five route manifests and [docs/testing/TurboVec-Feasibility-Result-v1.md](#) index experiment tests, runtime matrices, data-format, workbook/PDF and TurboVec lifecycle checks. These do not prove the WinUI journey or accessibility.

Appendix H

User Manual

H.1 Before Starting

These steps use Granite already installed on the Windows 11 x64 demonstration computer. Open it using the [launch guide](#): press **Windows** + **R**, paste the guide's command and press Enter. The command opens the prepared installation; it does not install the app on another computer. See Appendix L if you need a separate installation.

The tested Intel computer had about 16 GB of RAM. This is the test setup, not a guaranteed minimum. Leave space for models and temporary files, and keep the app's supplied runtime files together.

H.2 From Model Selection to Chat

Finish one setup before starting another:

1. **Choose a model.** To download an IBM Granite 4.0 H Micro GGUF model, use the preference slider in the recommended-download card. Choose any offered preference, review the format and size, then select **Download selected model**. Stay online until the file checks finish. The app checks its size and SHA-256 hash, a fingerprint of the file. A completed download does not guarantee that the model fits in memory.

Alternatively, choose **Choose model** and select a local GGUF file or a complete OpenVINO folder. Keep the OpenVINO weights, configuration and tokenizer files together. Wait for the quick scan, then choose **Continue to model inspection**. Verified downloads can open inspection automatically. Use one of these methods, not both.

2. **Wait for model inspection.** Read the result and any warning. Choose **Check hardware fit** when offered. If inspection fails, use the recovery action shown. A runtime error does not tell you whether the model fits in memory.
3. **Read the hardware-fit result.** A fit is an estimate, not a guarantee. Open **Hardware facts** or **How we worked this out** for details. A memory block means the setup exceeds the app's safe memory limit. An unanswered result means there is not enough information. If RAM is low, close memory-heavy programs and use the available recovery action to check again.
4. **Review the setup, if offered.** Check the weight format, KV-cache format and context length before choosing **Start optimisation**. The KV cache stores information used when generating replies; it is separate from model weights. This slider chooses a setup, unlike the earlier download slider. Only choices allowed by the model, runtime checks, available memory and disk space are offered.

5. **Wait for the output checks.** When **Save model to this computer** is offered, choose a destination with enough space and wait for export to finish. Keep an exported OpenVINO folder together. Runtime settings may not create a separate model file. Do not use temporary files as finished exports.
6. **Start chat.** Choose **Chat with this model** when offered. Send a short prompt, such as “Explain what a computer processor does in two short sentences”, and wait for the reply. Check answers before relying on them.

The [download guide](#) links to IBM’s model files and explains manual download checks. The app’s download card offers GGUF models, not OpenVINO packages. Ask the project owner for the matching tested OpenVINO package to try that route. Download size is not the RAM needed to run a model.

H.3 Stopping and Getting Help

Use the displayed cancel or stop action and wait for work to stop before starting again. A cancelled inspection has not passed. If the app cannot verify a required component, keep the diagnostic code and ask for the matching package. Do not replace individual runtime files or skip verification. Follow the [troubleshooting guide](#) and [support instructions](#). Remove private paths and prompts from shared logs.

Download models and set up the runtimes before going offline. Use non-sensitive prompts. Before resetting or uninstalling, close the app and follow the [backup instructions](#) for important exports and chat data. Restoring chat history and checking which files remain after uninstalling have not been fully tested.

Appendix I

Developer, Build, and Reproduction Manual

I.1 Source and Build Setup

The [source-setup guide](#) records cloning, runtime inputs and the available build steps. Further local instructions now describe downloading and checking the converter’s exact Python files, but that full build has not been tested. The TurboQuant preparation script still needs seven existing runtime DLLs with exact hashes; it does not build those files from source. Their preparation remains unresolved. The guide is therefore not yet a complete route from a fresh clone to a running application.

Install Git and the Windows development tools listed in the [build guide](#). Cloning downloads the source code and its history. Copy the HTTPS address from the repository’s **Code** menu, then run `git clone` followed by that address and a short destination such as `C:/src/granite`. Open Visual Studio Developer PowerShell in the new folder. Record `git rev-parse HEAD` and check `git status -short` before building, so you know which version you are using and whether it has local changes.

The recorded setup uses Visual Studio Community 2026 version 18.9.2 with Windows, C# and C++ build components. The repository selects .NET SDK 10.0.301 and targets .NET 8 for Windows. Recorded dependencies include Windows App SDK 2.2.0 and Windows SDK Build Tools 10.0.28000.2270. The test host also needs the .NET 8 Windows Desktop runtime. These are the recorded development versions, not proven minimum requirements. Check the project and lock files before changing them.

Use the build guide’s exact commands to restore dependencies and compile the code. The [hosted build workflow](#) turns off some native packaging checks because the required files are missing from that build environment. Its output is not a verified installer. Keep it separate from the working app. A working package needs the matching workers, dependencies and file hashes. A manifest lists the expected files and their hashes. Do not change an expected hash just to hide an error. Appendix L states the release limits.

I.2 Code and Test Locations

The application’s **Features** folder holds its screens and the code that connects them to each feature. Shared rules are in **shared**. Windows and worker communication code is in **infrastructure**; runtime connection code is in **runtime**. The **workers** projects provide separate programs that the app needs. External files and notices are under **third-party**. Tests and experiments have separate folders. The [developer guide](#) explains how these parts fit together.

Use the [test index](#) to select the correct project and runner. For real-model inspection tests, follow

the [native journey setup](#). Follow the setup guide to identify the installed app and model files. Close the app before running the test command. Read the TRX test-results file and check that the expected tests actually ran. Missing inputs or skipped tests are not passes. Appendix G records the saved results. Tests using simulated inputs do not prove that a real model ran.

I.3 Experiment Reproduction

Start with the [experiment reproduction guide](#). Use the model, runtime, computer setup and file hashes listed for the selected experiment. Use its scripts to rebuild and check the result tables, keeping new output separate from the saved results. Keep failed, blocked and unavailable cases in the results. A runtime working on its own does not prove it works through the app.

Appendix J

Code Source References

Table J.1 links each code figure in Chapter 6 to its source file and method. The links use the checked repository revision `df8208793088ef140447fa4ab3302054e43d617e`. The code matches the features described, although the screenshots may have been taken from an earlier revision. They are not repeated here.

Table J.1: Source files and methods for the Chapter 6 code figures.

Figure	Linked source file and method
6.1	VocabOnlyModelProbe.cs RunCoreAsync: loading vocabulary without full model weights.
6.2	WindowsSystemSnapshotProvider.cs CaptureAsync: collecting and checking memory readings.
6.3	MemorySystemEvidenceResolver.cs Resolve: handling conflicting memory readings.
6.4	GgufKvCacheEstimator.cs TryEstimate: estimating cache size with padding and rounding.
6.5	OptimizationPlanIssuer.cs RequireAdmissionAuthority: checking that a configuration remains approved.
6.6	AtomicBotServerArguments.cs Build: preparing arguments for the packaged runtime.
6.7	session.cpp official_session::create_pipeline: checking the applied cache precision.
6.8	GgufChatCoordinator.cs GenerateIntoAssistantAsync: keeping response text and updating its status.
6.9	WorkerTerminalCoordinator.cs TryBeginTerminal: preventing duplicate final messages.

These references show where the code is stored; they do not claim authorship of the external libraries it uses. Appendix K records the wider licence information. Copied or adapted code still requires its own source acknowledgement.

Appendix K

Risk, Assumption, Licence, and Known-Issue Registers

This appendix keeps the main report-facing records. The full risks, validation methods, evidence links, licence duties and review history are stored in [docs/risks/Risk-Register.md](#), [docs/risks/Assumption-Register.md](#), [docs/risks/Licence-Register.md](#) and [docs/risks/Licence-Review-Notes.md](#).

K.1 Risk Register

Table K.1 lists the six main risks at the report cut-off.

Table K.1: Main project risks at the report cut-off

ID	Risk	Rating	Control	Status
R-001	The project may miss the final deadline.	Medium / High	Protect the core scope and move lower-priority work through change control.	Open
R-026	The application may fail to build or run on a clean Windows computer.	Medium / High	Pin dependencies and complete a clean release test.	Open
R-033	Long operations may freeze, fail to cancel or leave processes running.	Medium / High	Keep work off the UI thread and test cancellation and cleanup.	Open
R-046	A downloaded model may be untrusted, incomplete or corrupt.	Medium / High	Use approved sources and check the filename, size and SHA-256 hash.	Open
R-123	The memory-fit estimate may accept an unsafe setup or reject one that fits.	Medium / High	Keep a safety reserve, label estimates and complete calibration later.	Open
R-245	The application or report may claim more than the evidence proves.	Medium / High	Link claims to evidence and state the exact model, runtime, device and configuration tested.	Open

K.2 Experiment-Derived Known Issues

The remaining limits include failed or unavailable SYCL and OpenVINO cases, incomplete device tracing, unfinished estimator and application studies, and TurboVec build and quality failures. These limits still apply where other configurations passed.

K.3 Assumption Log

Table K.2 gives the selected final outcomes and their evidence limits. Across the full register, ten assumptions are confirmed, one is rejected for the tested scope and six remain pending. An independent OneDrive recovery test restored all 2,708 final and raw evidence files with matching paths, sizes and SHA-256 hashes, confirming A-001 and A-015. A separate offline application check completed startup, model inspection, hardware inspection and local chat with an already downloaded model, without a network or firewall request. This confirms A-013 for the tested route only. The exact route and model were not recorded in the supplied result, so no wider claim is made. The full register contains all 17 assumptions.

Table K.2: Selected final assumption outcomes

ID	Assumption	Evidence and limit	Outcome
A-005	A selected Granite model can run through OpenVINO.	Conversion, loading and generation passed on the test computer, but unavailable artefacts limited wider testing.	Confirmed
A-006	The available Windows and Intel hardware supports the required routes.	Selected CPU and graphics routes ran on the recorded test computer; this does not prove support elsewhere.	Confirmed
A-007	The application can always identify the device that performed inference.	Device evidence was incomplete for some OpenVINO runs.	Pending
A-008	The memory estimator gives sufficiently accurate guidance.	The estimator was implemented, but the planned measured calibration was not completed.	Pending
A-009	TurboQuant can run with a selected Granite model.	Selected configurations passed controlled build, inference, quality and stability runs on the test computer.	Confirmed
A-010	TurboQuant activation and KV-cache reduction can be shown.	Activation evidence and matched tests showed lower explicit KV-cache allocation in several configurations.	Confirmed
A-012	TurboVec can be integrated without unacceptable retrieval-quality loss.	Compressed formats failed the matched retrieval-quality requirements, so TurboVec remained a separate demonstrator.	Rejected
A-013	The core application can run locally without a cloud AI service or an open local server port.	The tested local workflow completed with an already downloaded model and no network or firewall request. The result applies only to the tested route.	Confirmed
A-016	All included components permit the intended release and distribution.	Source licences were checked, but the final package and dependency review remains open.	Pending
A-017	Intended non-specialist users can complete the application workflow.	The planned task-based intended-user evaluation was not completed.	Pending

K.4 Dependency and Licence Inventory

Table K.3 keeps every selected dependency group. *Restricted* means that project use is allowed, but the final package still needs a file and notice review.

Table K.3: Dependency and licence inventory

Dependency and identity	Use	Licence / status	Main release condition
IBM Granite 4.1, official 3B and 8B models	Experiments	Apache-2.0 / Restricted	Keep exact artefact sources and hashes. Do not bundle them without a package review.
Granite 4.0 H Micro GGUF, revision 51ce07a9	Model download	Apache-2.0 / Restricted	The application checks five pinned files. They are downloaded, not bundled.
llama.cpp, tag b9870, commit 2d973636	GGUF baseline	MIT / Approved	Preserve the MIT notice and review optional native dependencies.
TurboQuant forks, AtomicBot 519f0c59 (runtime and experiments; licence record still lists b0e900a2) and animehacker 5bc5ed3b	Experiments and one exact application route	MIT / Restricted	Preserve MIT and inherited notices; review every native file.
OpenVINO and OpenVINO GenAI, 2026.2.1 campaign versions	Inference and optimisation	Apache-2.0 / Restricted	Include the licence and notices; review the final DLL inventory.
LLamaSharp 0.27.0 and mapped llama.cpp commit 3f7c29d3	GGUF inspection	MIT / Restricted	Preserve both MIT notices and review the included native runtime files.
Windows App SDK 2.2.0 and SDK Build Tools 10.0.28000.2270	Interface and build	Package terms / Restricted	Keep Build Tools out of the application package and check final notices.
.NET SDK 10.0.301, target net8.0-windows10.0.19041.0	Build and runtime	MIT and third-party terms / Restricted	Record shipped runtime files and notices for a self-contained package.
TurboVec 1.0.0, commit ccab9f32	Feasibility demonstrator	MIT / Restricted	Not adopted by the application; preserve its notice if the demonstrator is shared.
PdfPig 0.1.15, commit f131f642	Demonstrator PDF extraction	Apache-2.0 / Restricted	Preserve the Apache licence and review notices before sharing.

Appendix L

Release and Demonstration Material

L.1 Recorded Application and Release Status

The [project repository](#) provides the source and documentation. The [release index](#) links the handover records, and the [application evidence index](#) records E1–E6. E6 contains three passing automated inspection journeys with no failures or skips, not tests of the full optimisation-to-chat workflow. Appendix G gives their scope.

The source revision recorded for the *E6 test candidate* is `90f93a33`; its full commit ID and executable SHA-256 hash are in the release index. This is not a final submission revision. The checksum identifies the executable, not an installer. E6 used an existing installation and did not independently check that it was built from that source version. The offered downloads are identified in the [download catalogue](#); the models used in tests and experiments are identified in their own file records.

The [package handover](#) does not yet record a final release tag, verified public installer, or completed installation and removal test on a clean computer. To use another computer, ask the owner for the matching package, dependencies and installation steps. Compiling the code is not enough. The documentation links are pinned to commit `38ab1dcf`. This identifies the linked files, not a verified installation release. The newer setup guide is not included in that commit.

L.2 Demonstration and Evidence Backup

Follow Appendix H on the prepared Windows 11 Intel computer. The demonstration video can be accessed through the [demonstration video folder](#). Show model selection, inspection, hardware fit, an available configuration and chat, with export only when offered. Explain that fit is an estimate and choices depend on the model, runtime and available memory and disk space.

The owner also reported completing inspection, optimisation, export and chat with Wi-Fi disconnected. This manual check does not cover every setup or prove that all network activity was checked.

The [independent recovery record](#) records that the tested final and raw evidence folders were restored from OneDrive with matching paths, sizes and SHA-256 hashes. Appendix K gives the counts and limits. This backup covers those evidence folders at the time of the test, not the whole app. Back up and check any later additions too. Chapter 9 gives the conclusions, while Appendix K lists the risks and licence restrictions.